Boosting Adverse Weather Crowd Counting via Multi-Queue Contrastive Learning

A Thesis
Submitted in Partial Fulfillment of the Requirements
for the Degree of Master of Engineering

Tianhang Pan

Supervisor: Prof. Xiuyi Jia

School of Computer Science and Engineering
Nanjing University of Science and Technology

October 2025

Abstract

Crowd counting supports a broad range of applications, including public safety, video surveillance and traffic control, and modern counting networks achieve remarkable accuracy on images captured under normal weather conditions. Real deployments, however, cannot choose the weather. Haze, rain and snow alter the texture, contrast and edge profile of pedestrian appearance, and the resulting domain gap between normal-weather and adverse-weather images severely degrades counting accuracy. The problem is aggravated by a second, structural obstacle: adverse-weather images are rare, so the weather classes of existing counting datasets are heavily imbalanced — in the JHU-Crowd++ benchmark, rainy images account for only 3% of the training data. This thesis formalizes crowd counting under diverse weather as an *imbalanced multi-domain learning* problem and identifies two key obstacles that limit counting accuracy: (i) the domain gap between adverse-weather and normal-weather images, and (ii) weather class imbalance in the training set.

To address both obstacles we propose Multi-queue Contrastive Learning (MQCL), a lightweight two-stage training method. In the first stage, Weather-aware Representation Learning (WRL), the counting backbone is equipped with weather-aware representations through a novel contrastive learning scheme, multi-queue MoCo, which replaces the single memory queue of vanilla momentum contrast with one queue per weather class. The class-wise queues restore the match between the data distribution held in memory and the uniform prior implicit in the contrastive objective, and thereby enable stable representation learning despite severe class imbalance. In the second stage, Contrastive-learning-guided Representation Refining (CRR), the learned queues are frozen and a lightweight refiner is trained with a second contrastive objective that treats every normal-weather vector in the queues as a positive example. The refiner thereby learns to convert adverse-weather representations into the normal-weather domain, so that the counting decoder operates on a single, consistent domain at inference time.

We instantiate MQCL on four backbones (VGG-16, CSRNet, Swin-T and ConvNeXt-T) and evaluate on two benchmarks with complementary characteristics, JHU-Crowd++ and GCC. On JHU-Crowd++, MQCL with a ConvNeXt-T backbone reduces the adverse-weather error by 22% in MAE relative to the baseline while improving normal-weather accuracy at the same time, and introduces only about 16% additional computational cost — achieving state-of-the-art overall accuracy among compared methods. Controlled imbalance experiments show that MQCL degrades far more gracefully than the baseline and its single-queue variant as training data becomes more imbalanced, and comparisons against four representative imbalanced-contrastive-learning strategies confirm the advantage of the multi-queue design. Beyond crowd counting, the multi-queue MoCo scheme is a generic remedy for contrastive learning under class imbalance and is expected to transfer to other imbalanced multi-domain tasks.

Keywords: crowd counting; adverse weather; contrastive learning; class imbalance; momentum contrast; representation learning

Acknowledgments

This thesis was completed at the School of Computer Science and Engineering, Nanjing University of Science and Technology. I owe my deepest gratitude to my supervisor, Prof. Xiuyi Jia, for her guidance throughout this work — from the initial framing of adverse-weather crowd counting as an imbalanced multi-domain learning problem, through the design of the multi-queue memory, to the writing of this manuscript. I also thank the maintainers and annotators of the JHU-Crowd++, GCC and NWPU-Crowd datasets, without whose labeled efforts an empirical study of weather imbalance would not be possible, and the open-source community behind PyTorch and the numerical Python stack. Finally, I thank my family and colleagues for their patience and support during the long process of writing.

Table of Contents

Abstract 2

Acknowledgments 3

Notation and Abbreviations 7

Mathematical Symbols 7

Abbreviations 7

Chapter 1: Related Research and Objective Setting 8

1.1 Background and Motivation 8

1.2 Related Research 11

1.2.1 Crowd Counting under Normal Conditions 11

1.2.2 Crowd Counting under Adverse Conditions 14

1.2.3 Contrastive Learning and Class Imbalance 17

1.3 Problem Formulation and Objective Setting 21

1.3.1 Task Formulation 21

1.3.2 The Two Obstacles 21

1.3.3 Scope and Assumptions 24

1.3.4 Objectives and Requirements 24

1.4 Research Approach and Thesis Organization 24

Chapter 2: The Proposed Method: Multi-Queue Contrastive Learning 27

2.1 Method Overview and Design Rationale 27

2.1.1 Architecture 27

2.1.2 Why two stages, and why contrastive learning 28

2.1.3 Notation 28

2.1.4 Component lifecycle across stages and deployment 28

2.1.5 Task output 30

2.2 Stage 1: Weather-Aware Representation Learning via Multi-Queue MoCo 30

2.2.1 Counting supervision: the Bayesian loss 30
2.2.2 Baseline machinery: momentum contrast 31
2.2.3 The vanilla objective and why it fails under imbalance 32
2.2.4 Multi-queue MoCo 33
2.2.5 Stage objective 39
2.3 Stage 2: Contrastive-Learning-Guided Representation Refining 39
2.3.1 The stage interface 39
2.3.2 Refiner mapping 39
2.3.3 Domain conversion by contrastive guidance 40
2.3.4 Stage objective 41
2.4 Algorithm Summary, Plug-and-Play Design and Complexity 42
2.4.1 Training procedures 42
2.4.2 Plug-and-play properties 42
2.4.3 Cost model 42
2.4.4 Roles of the objective hyperparameters 43
2.4.5 Label usage and deployment semantics 44
2.4.6 Requirements check 44
2.5 Chapter Summary 45
Chapter 3: Implementation, Testing and Analysis 46
3.1 Implementation Details 46
3.2 Datasets and Evaluation Protocol 47
3.2.1 JHU-Crowd++ 47
3.2.2 GCC 48
3.2.3 Metrics 49
3.3 Experimental Results 50
3.3.1 Comparison with the state of the art 50
3.3.2 Ablations and analyses 53
3.3.3 Computational complexity and model size 57
3.4 Analysis, Limitations and Conclusion 58
3.4.1 Why does the multi-queue win? 58
3.4.2 Threats to validity 58
3.4.3 Requirements verification 59
3.4.4 Limitations 59
3.4.5 Conclusion and future work 59
List of Publications 62
Chapter A: Supplementary Material 63
A.1 NWPU-Weather: A Synthesized Adverse-Weather Benchmark 63

A.2 Is the Refiner Useful Without Contrastive Guidance? 64
A.3 Training Configuration Reference 64
A.4 Reproduction Checklist 65
Appendix B: Editable Diagrams 66
References 68

List of Figures

Figure 1.1: (a),(b) Weather condition distributions of the JHU-Crowd++ and GCC datasets: rain, snow … 10
Figure 2.1: The architecture of MQCL. Upper half: the WRL stage learns weather-aware … 28
Figure 2.2: Multi-queue MoCo. Projection heads map representations to 1-D vectors; the memory … 38
Figure 3.1: Weather class distributions of the two benchmarks. Both are strongly imbalanced, with … 49
Figure 3.2: Example counting scenes under normal (a) and adverse (b) weather. Adverse weather …49
Figure 3.3: Qualitative comparison on JHU-Crowd++ test images (normal, haze, rain and snow). Rows: … 52
Figure 3.4: t-SNE [58] visualization of the projected anchor vectors after the WRL stage on … 55
Figure 3.5: Counting accuracy under varying degrees of weather-class imbalance (horizontal axis: … 56
Figure A.1: Example images of the synthesized NWPU-Weather dataset. 63
Figure B.1: The MQCL two-stage pipeline 66
Figure B.2: Multi-queue MoCo memory 66
Figure B.3: Weather-class composition of JHU-Crowd++ 67

List of Tables

Table 1.1: Positioning of this thesis among related research lines. MQCL is the only method that …20
Table 1.2: Observed weather-class imbalance in the evaluation data. is the ratio of the largest to …23
Table 1.3: Requirements derived from the objectives. Each requirement is checked by a specific …24
Table 1.4: Contributions mapped to objectives and chapters. 26
Table 2.1: Component lifecycle: which components are trained, frozen or absent in each phase. ... 29
Table 2.2: Central notation of Chapter 2. 29
Table 2.3: Properties of contrastive memories. Balance is the property that Eq. (2.9) requires … ...34
Table 2.4: The two contrastive objectives side by side. 40
Table 2.5: Requirements versus design elements. 44
Table 2.6: The equation chain of Chapter 2 and the obstacle each part addresses. 45
Table 3.1: Summary of the evaluation benchmarks. 49

Table 3.2: Quantitative comparison with state-of-the-art methods on the JHU-Crowd++ dataset. Best … 50

Table 3.3: Quantitative comparison with state-of-the-art methods on the GCC dataset (cross-location … 52

Table 3.4: Ablation of the two stages on JHU-Crowd++ (ConvNeXt-T backbone). 53

Table 3.5: Positive/negative selection strategies on JHU-Crowd++. Strategy 1: same weather label as … 54

Table 3.6: Storage strategies for key vectors on JHU-Crowd++ (JHU-Crowd++, ConvNeXt-T). ... 54

Table 3.7: MAE on JHU-Crowd++ under controlled weather-class imbalance (normal/haze/rain/snow … 55

Table 3.8: Multi-queue MoCo versus representative imbalanced contrastive learning strategies on … 57

Table 3.9: Computational complexity (FLOPs at input) and number of parameters. 57

Table 3.10: Verification of the requirements defined in Chapter 1. 59

Table A.1: Quantitative results on the NWPU-Weather dataset. 63

Table A.2: Effect of removing contrastive learning from both stages (JHU-Crowd++). 64

Table A.3: Complete training configuration. 64

Notation and Abbreviations

Mathematical Symbols

X	input image	I	mini-batch (set of anchors)
$F(\cdot\,;\theta)$	encoder (counting backbone)	Q_i	anchor vector (projection of $\mathrm{R}_{\mathrm{Q}_i}$)
θ	encoder parameters	K_p	key vector from a memory queue
$\tilde{\theta}$	momentum-encoder parameters	τ	temperature of the contrastive loss
μ	momentum coefficient	$P(i)$	positive index set of anchor i
$\mathrm{E}_Q, \mathrm{E}_K$	on-line / momentum encoder	$\boldsymbol{A}$	index set of all queued keys
$\mathrm{P}_Q, \mathrm{P}_K$	projection heads	$\boldsymbol{N}$	index set of normal-weather keys
D	counting decoder (crowd head)	B	number of weather classes
$g_\phi(\cdot)$	refiner network	L	length of each sub-queue
$R, \mathrm{R}_\mathrm{Q}, \mathrm{R}_\mathrm{K}$	representations	C_1, C_2	representation / embedding dims
$D_\sigma(\cdot)$	rendered density map	d	embedding dimension (bytes calc.)
c_j	count mass of head j	λ_1, λ_2	loss weights of the two stages
$\delta(\cdot)$	Dirac delta	$\mathcal{N}(x\|m, \sigma^2)$	Gaussian density

Abbreviations

MQCL	Multi-queue Contrastive Learning	MAE	Mean Absolute Error
WRL	Weather-aware Representation Learning (stage 1)	RMSE	Root Mean Squared Error
CRR	Contrastive-learning-guided Representation Refining (stage 2)	SOTA	state of the art
MoCo	Momentum Contrast	t-SNE	t-distributed Stochastic Neighbor Embedding
CL	contrastive learning	FLOPs	floating-point operations
KCL	k-positive contrastive loss	BCL	balanced contrastive learning
TSC	targeted supervised contrastive learning	SBCL	subclass-balanced contrastive learning
GCC	GTA5 Crowd Counting dataset	nAP	density-normalized Average Precision

Chapter 1: Related Research and Objective Setting

1.1 Background and Motivation

Crowd counting — estimating the number of people in a single image — has attracted sustained attention because it underpins a wide range of applications such as public safety, video surveillance, traffic control and urban planning. Driven by large-scale benchmarks and increasingly powerful backbones, modern counting networks estimate crowd sizes with remarkable accuracy on images recorded under *normal* weather conditions, and the field has largely treated clear, well-lit scenes as the default operating point.

The practical stakes of weather robustness are easy to underestimate. In surveillance and public-safety settings, precisely the scenes that matter most — accidents, gatherings during storms, fog-bound traffic — tend to occur in degraded conditions, so a model whose error grows exactly when the scene becomes adverse fails asymmetrically: it is least reliable when it is most needed. Counting errors also compound downstream: occupancy estimates feed flow control and alerting systems, and a sudden accuracy drop at weather onset would manifest as spurious alarms or missed crowd-risk events. Counting (rather than full detection) is the task of choice in such pipelines precisely because it degrades gracefully in crowds — but only if its representation is weather-robust, which is the property this thesis investigates. Historically, crowd sizes were estimated by hand — sampling density on aerial or news photographs and extrapolating — a practice accurate enough for journalism but unusable for continuous operations. The shift to learning-based counting (reviewed in Section 1.2.1) made per-camera, per-frame estimation feasible, and in doing so changed the failure mode: hand estimation failed on lack of information, whereas learned estimators fail on *distribution mismatch* — they are exact where their training distribution reaches and silently unreliable where it does not. Weather is the largest, most predictable axis of that mismatch for outdoor cameras, and the one axis whose shifts arrive on a schedule anyone can observe. This is the sense in which adverse-weather robustness is not a niche concern but the leading edge of the deployment question for the entire field.

A related design question is why *counting* rather than full detection is the right task formulation for crowded, weather-degraded scenes. Detection frameworks traverse the same degraded images but must localize every head with box-level precision; in dense crowds, heads shrink to a few pixels, overlap heavily, and lose exactly the contour evidence that weather corrupts first. Density-based counting sidesteps the localization requirement — aggregate evidence is harder to destroy than any single instance's evidence. Localization-based counting retains this advantage only partially: its proposals are still per-head decisions, and its matching-based training (Hungarian or proposal-assignment schemes) is sensitive to exactly the per-head evidence that weather perturbs.: it only needs the *integral* of a smooth field to be correct, so it degrades more gracefully with density, occlusion and blur. This tolerance is not unconditional, however — the density still has to be produced from features that survive the weather, which is the representation question at the center of this thesis. In extreme-density regimes the two formulations also differ in annotation cost: point annotations scale better than boxes, and all major weather-tagged benchmarks (JHU-Crowd++, GCC, NWPU-Crowd) provide point supervision, which suits the density formulation adopted here.

Worse, weather and crowd density interact multiplicatively rather than additively. The scenes where crowds gather in bad weather — stations during storms, festivals in fog — are precisely high-density scenes, where heads are smallest and most occluded to begin with; the degradation lands on the already hardest inputs. A counting error of a few percent on a sparse clear scene is a rounding issue; the same relative error on a dense rainy scene is hundreds of missed people. This is why the

thesis insists on the adverse-weather *subset* metrics rather than totals, and why a method that improves sparse-weather accuracy while collapsing on dense adverse scenes has not solved the problem. The experiments of Chapter 3 therefore read MAE and RMSE jointly: MAE for typical per-image behavior, RMSE specifically to catch the heavy-tailed failures that dense adverse scenes produce.

The application profile of crowd counting explains why this robustness matters economically, not only academically. Transportation authorities use counts to gate platform crowding at hubs; city-scale event management relies on density estimates to trigger crowd-control interventions before conditions become dangerous; retail analytics use footfall counts to plan staffing and layouts; and epidemiological practice has renewed interest in gathering-size estimation. In all of these, the operating assumption is that the camera cannot be moved indoors and the weather cannot be scheduled: the estimator must simply work, year-round. A weather-fragile model therefore does not merely lose accuracy on a benchmark slice — it loses entire days of operation, which for safety applications is the difference between having a measurement and having none. The two error directions are also not symmetric in consequence. Under-counting at a transit hub during a storm means crowd-control systems believe the platform is safer than it is — the dangerous direction. Over-counting wastes resources (staff, closures) but fails safe. A method whose errors are systematically directional under specific weather is therefore worse than its MAE suggests: the per-weather analysis of Chapter 3 checks not only the magnitude but the *distribution* of errors across conditions, and part of the qualitative evaluation inspects whether residual errors under adverse weather remain unbiased. Robustness, in operational terms, is the stability of both the magnitude and the sign of errors across the weather axis.

Beyond weather, counting “in the wild” poses challenges that any robustness claim must respect: extreme variation in crowd scale (from tens to tens of thousands per frame), arbitrary camera viewpoints and distances, heavy occlusion, and distribution shift across scenes and cities. These factors interact with weather — a distant, tiny crowd under haze is harder than either condition alone — so a fair robustness study must hold the non-weather factors as constant as the benchmarks allow, and vary the weather axis deliberately. That principle shapes the experimental design of Chapter 3: matched baselines, per-weather test subsets, and a synthetic benchmark in which scene content is identical across weather conditions.

Real deployments do not enjoy this luxury. Cameras for surveillance, transportation hubs and smart-city monitoring operate around the clock and through the seasons, so a non-trivial fraction of the incoming frames is affected by haze, rain or snow. Such adverse weather is not a superficial nuisance: it alters the texture and edge features of objects, suppresses local contrast, and physically changes the appearance of people themselves, who wear raincoats, carry umbrellas or wrap themselves in winter clothing. Umbrellas and raincoats additionally *occlude* neighbouring pedestrians, so the degradation is not limited to low-level appearance but propagates into the spatial arrangement of the crowd. The resulting domain gap frequently causes counting difficulties, and the limited robustness of counting networks to adverse weather fundamentally bottlenecks their deployment.

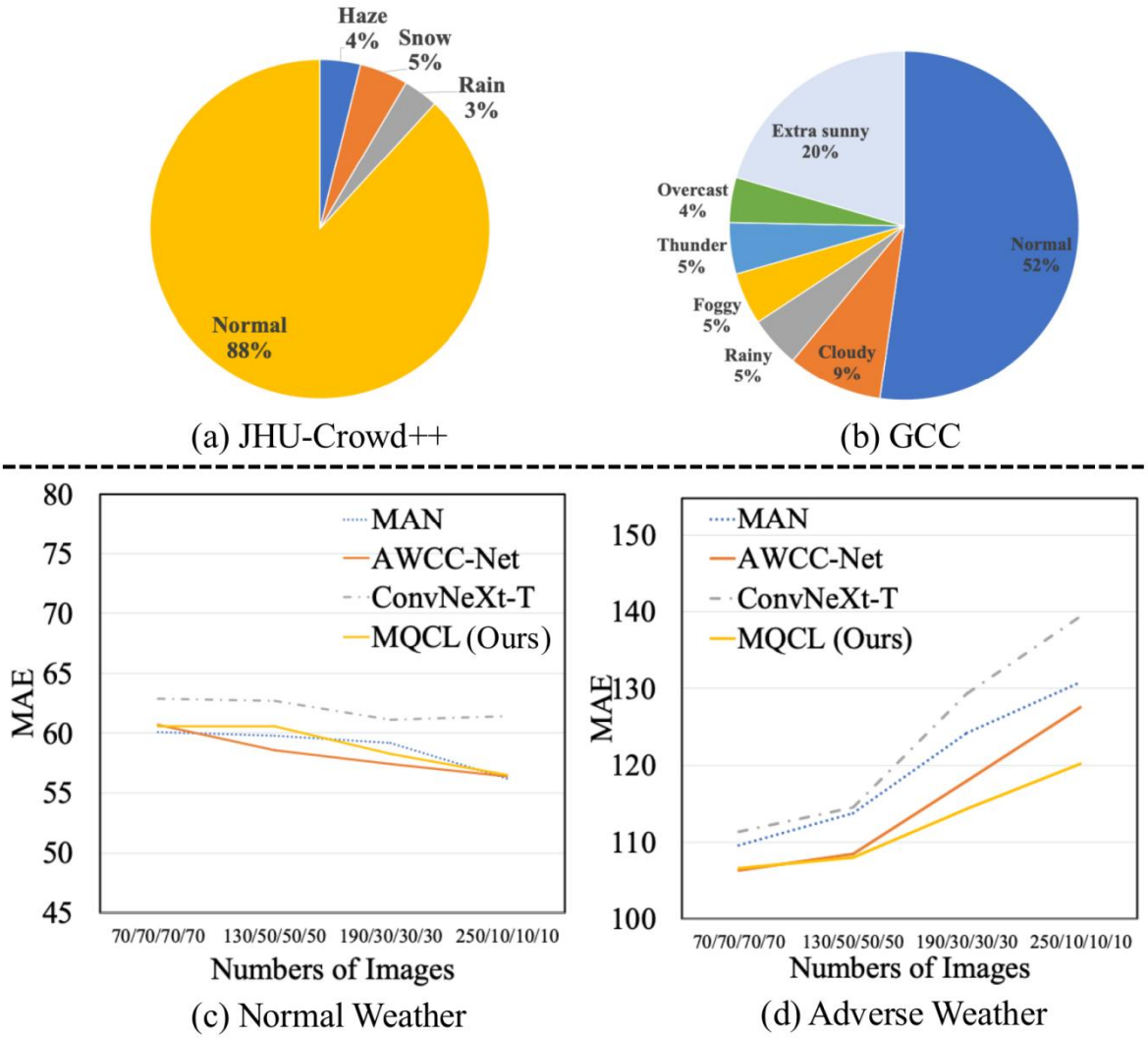


Figure 1.1: (a),(b) Weather condition distributions of the JHU-Crowd++ and GCC datasets: rain, snow and haze images account for only a small fraction of JHU-Crowd++. (c),(d) Counting accuracy (MAE) of different models on normal-weather and adverse-weather images under varying degrees of weather-class imbalance in the training data. As imbalance intensifies, existing models improve only marginally on normal weather yet degrade severely on adverse weather, whereas MQCL remains robust.

The problem is aggravated by a structural property of the available data: labeled adverse-weather crowd images are exceedingly rare. Collecting and annotating crowds in rain or snow is difficult and sometimes dangerous, so benchmark datasets inherit the natural skew of the weather itself. Figure 1.1(a) shows the weather distribution of the JHU-Crowd++ dataset [1]: images under rain account for only 3% of the data, and haze and snow images are similarly scarce (4% and 5%). The synthetic GCC dataset [2], shown in Figure 1.1(b), is imbalanced as well, spanning seven weather types of very unequal frequency.

Such imbalance is not a benign inconvenience. When the training distribution is dominated by one class, models trained with conventional objectives silently fit the majority and under-serve the minority. Figure 1.1(c) and (d) make this concrete for crowd counting: when the training data becomes progressively more imbalanced, established models such as MAN [3] show only negligible improvement on normal-weather images while degrading severely on adverse-weather images. Even AWCC-Net, the Transformer-based method specifically designed for adverse weather counting [4], still suffers clear degradation when class imbalance intensifies, despite carrying a heavy computational burden.

A natural first remedy is to *restore* the image before counting: apply an image restoration module to remove rain streaks, haze or snow, and then count on the restored image. Unfortunately, even when

restoration mitigates the obscuration caused by weather, the restored images still exhibit significant domain differences from genuinely normal images — restoration can remove a rain veil, but it cannot eliminate the occlusion caused by the raincoat or umbrella itself. Moreover, restoration modules, weather classifiers and enhancement blocks all add parameters and computation to the deployed model. Training a separate model per weather category is equally unattractive: the few available adverse-weather images cannot support training a high-capacity model in isolation, and an additional weather classifier is needed at test time, whose misclassifications propagate into the prediction.

This thesis therefore pursues a method that satisfies three objectives:

- **[O1]** achieve high counting accuracy under adverse weather *even when the training data is weather-class-imbalanced*;
- **[O2]** maintain high accuracy under normal weather conditions;
- **[O3]** remain lightweight and adaptable to diverse backbone architectures (plug-and-play), rather than relying on heavy task-specific modules.

The route we take is to view crowd counting over normal *and* adverse weather as an *imbalanced multi-domain learning* task, and to let the counting backbone itself learn weather-aware, class-balanced representations via contrastive learning. The resulting method, Multi-queue Contrastive Learning (MQCL), is previewed in Section 1.4 and developed in Chapter 2.

1.2 Related Research

This section reviews the three research lines that MQCL builds upon: crowd counting under normal conditions (Section 1.2.1), crowd counting and image restoration under adverse conditions (Section 1.2.2), and contrastive learning with its treatment of class imbalance (Section 1.2.3). Each subsection closes with the gap that remains open and that this thesis addresses.

1.2.1 Crowd Counting under Normal Conditions

Most single-image crowd counting methods fall into two families: density-map-based methods and localization-based methods.

Density-map-based methods

Density-based methods regress a density map whose pixel values represent local crowd density; the estimated count is the sum over all pixels of the map. MCNN [5] pioneered this formulation with a multi-column architecture whose columns of different receptive-field sizes handle inputs of arbitrary resolution and perspective. SANet [6] aggregates multi-scale features with an isotropic grid and transposed convolutions to preserve spatial resolution. CSRNet [7] replaces the back end with dilated convolutions, enlarging the receptive field while avoiding the resolution loss of pooling. CAN [8] encodes multi-scale contextual information with a single-column network and soft pyramid pooling, and SFCN [2] scales the backbone while introducing a Bayesian loss for supervision. The Bayesian loss itself [9] reformulates the density target as a per-anchor Gaussian mixture, eliminating the need for a hand-tuned fixed Gaussian kernel and substantially improving robustness; it is the counting supervision adopted in this thesis.

Beyond purely convolutional designs, DM-Count [10] matches the unnormalized distributions of predicted and ground-truth densities with optimal transport, and MAN [3] augments a Transformer backbone with a multifaceted attention network that encodes local spatial relations and appearance features. Du et al. [11] redesign the multi-scale network as a hierarchical mixture of density experts, and Gramformer [12] models sample relations with a contrastive Gram matrix for crowd

transformers. Composition-loss ensembling [13] also contributes the observation that global context, density level and head count are complementary signals.

These methods continually push accuracy on clear-weather benchmarks, but their supervision and architectures are domain-agnostic in an unfortunate sense: they assume that all training and test images are drawn from one visual domain. Nothing in the density-regression objective distinguishes a rain-soaked pedestrian from a dry one, so weather-induced appearance shifts translate directly into counting error.

Localization-based methods

Localization-based methods sidestep the density regression and predict head positions directly. Early work framed detection as the counting primitive: Stewart et al. [14] formulated end-to-end people detection in crowded scenes with a convolutional formulation of R-CNN, and Liu et al. [15] proposed the point-in-box detection scheme that weakly supervises head points from box annotations. P2PNet [16] regresses a set of point proposals that directly matches human annotations and introduces the density-normalized average precision (nAP) metric for a more precise evaluation of localization quality. CLTR [17] brings the Transformer to crowd localization with a Hungarian matcher that reduces ambiguous assignments, and APGCC [18] provides clearer guidance for proposal selection and optimization, addressing matching uncertainty in point-based counting. Across these designs the trade-off is consistent: localization methods buy interpretable outputs and tighter evaluation with a harder matching problem, while density methods buy simplicity and scale with an indirect output. For the weather-robustness question both families reduce to the same prerequisite — features whose semantics survive the domain shift — which is why this thesis develops its contribution at the shared representation level and instantiates it on the density formulation, whose supervision is annotation-adaptive and battle-tested on the very benchmarks used here. A second, practical reason to prefer the density formulation for this study concerns *output resolution*: density heads can be attached to low-resolution feature maps and upsampled (by transposed convolution or bilinear interpolation) with little penalty on the integrated count, whereas localization quality is directly coupled to spatial resolution. Since the refiner of Chapter 2 operates at representation resolution, a formulation tolerant of that resolution is required for the two-stage design to remain lightweight; the density formulation provides this tolerance natively. The formulation itself has a history worth recalling, because it defines what “learning to count” means. Lempitsky and Zisserman [19] reframed counting as density estimation learnable from weak point annotations — a regression problem solvable with per-image constraints — and thereby made counting a machine-learning task rather than a detection-by-other-means. Everything since has refined the function class and the supervision, not the problem statement: regress a non-negative field whose integral is the count. The stability of that problem statement is an asset for this thesis: a training-scheme contribution can be evaluated against a fixed target semantics across architectures, datasets and years, so improvements are attributable to the scheme rather than to drift in the task definition. Idrees et al. [13] combine local detection scores with global context in a multi-loss formulation, and machine-EDA style augmentations have also been explored to regularize localization training.

Benchmarks under normal conditions

The benchmarks that shaped these methods deserve a brief review, since this thesis inherits two of them and their design explains what the field could and could not see. ShanghaiTech (Parts A and B), introduced alongside MCNN [5], established the standard train/validation/test protocol on internet and street images of widely varying density. UCF-QNRF [13] pushed scale to 1.5 million annotations over 1,535 dense images and demonstrated how supervision quality — rather than architecture — can bottleneck counting. WorldExpo [20] pioneered the cross-scene evaluation protocol, testing whether a counter trained on some scenes transfers to unseen ones — an early,

scene-axis version of the generalization question this thesis asks along the weather axis. NWPU-Crowd [21] scaled the benchmark to 5,109 images with ordinary-minority diversity and a withheld test set. What *none* of these datasets provides, with the sole exception of JHU-Crowd++ [1], is a weather label; the benchmarks were constructed to measure progress on density, occlusion and scale, and their weather composition is an accident of collection. The consequence for research practice is structural: methods are selected and tuned on clear-weather-dominated averages, so the field's reported progress and its weather robustness can diverge without anyone noticing. This thesis's evaluation protocol (per-weather columns, imbalance-controlled training subsets, and a weather-synthesized benchmark) is designed to close exactly that measurement gap.

Two aspects of this line matter for the design choices of Chapter 2. First, supervision has evolved from fixed-kernel density regression towards annotation-adaptive targets: the Bayesian loss [9] renders each anchor's count mass from its neighbours, removing the hand-tuned kernel bandwidth that earlier losses required, and this robustness under crowding and occlusion is why this thesis adopts it as the counting objective in both training stages. Second, the architecture of the counting head has become deliberately decoupled from the backbone: MFA's upsampling head [3] (two transposed convolutions) works across CNN and Transformer encoders alike. This decoupling is what makes a training-scheme contribution such as MQCL meaningful — if the supervision and the head are fixed, any measured improvement is attributable to the representation, which is exactly the variable this thesis manipulates.

The evolution of the *supervision* for density regression deserves a closer look, because it parallels the evolution of the architectures. The earliest density targets placed a fixed-variance Gaussian kernel on every annotated head and minimized pixel-wise mean squared error; the kernel bandwidth had to be chosen by hand, and a single choice is simultaneously too wide for congested regions (where heads are a few pixels apart) and too narrow for sparse ones. Perspective-aware variants estimated the bandwidth from the geometry of the scene, but required perspective maps that real datasets rarely provide. The Bayesian loss [9] dissolved the problem by making the bandwidth *per head* and *data-derived* (Eq. (2.1) of Chapter 2), and DM-Count [10] went one step further by abandoning pixel-wise targets altogether, matching the total mass of predicted and ground-truth densities with optimal transport plus total-variation regularizers. The trajectory — from hand-tuned pixel targets to annotation-adaptive and distribution-level objectives — is a steady retreat from assumptions that dense and occluded scenes violate, and the Bayesian point-anchored formulation is the one this thesis builds on because its per-head adaptivity is exactly what rain, snow and haze (which blur and merge heads) stress most.

On the *architecture* side, the field progressed from multi-column designs (MCNN [5]) through dilated single-column networks (CSRNet [7]) to attention- and Transformer-based backbones (MAN [3], Gramformer [12]). Two observations from this progression inform the present work. First, accuracy gains have increasingly come from *representation quality* rather than from the counting head itself, which has remained a lightweight upsampling module across generations; the representation, not the decoder, is the sensitive variable. Second, every generation of backbones has been validated almost exclusively on clear-weather benchmarks, so the representations that made them accurate are optimized — through large-scale classification pretraining and counting fine-tuning alike — for the statistics of clear images. Nothing in this pipeline is broken for clear-weather evaluation and everything is unexamined for weather shift, which is the opportunity this thesis takes up.

Evaluation practice reinforces the blind spot. Standard practice reports a *single* MAE/RMSE pair over the whole test set, which averages away the weather: if 88% of test images are clear, a method can shed 30 MAE points on rainy scenes and move the headline number by only a few tenths. The JHU-Crowd++ weather annotations make per-weather reporting possible — normal, adverse and total columns — and this thesis treats the per-weather columns, not the total, as the primary evidence throughout. Localization work adds the nAP metric [16], which normalizes precision by

local density; it is orthogonal to the counting metrics used here and equally weather-agnostic, so the same averaging concern applies to localization benchmarks. The blind spot is easy to quantify. Suppose a test set is 88% clear-weather and a modification improves adverse-weather MAE from 105 to 82 while leaving clear-weather accuracy untouched. The headline (total) MAE moves by only $0.12 \times 23 \approx 2.8$ points — a change a reviewer could dismiss as noise — even though the modification more than halved the failure rate on exactly the scenes that matter for deployment. Averages are not merely incomplete evidence; under class imbalance they are actively misleading evidence, which is why every results table of Chapter 3 carries explicit normal/adverse columns.

A third, subtler axis of variation is the *output parametrization*: density maps integrate trivially to a count but blur under uncertainty; patch/classification counters bin images into coarse cells; point-based methods localize but need matching during training. The methods reviewed above span all three choices, and none of the choices interacts with weather directly — which is why the contribution of this thesis is located at the representation level, upstream of all of them, and is compatible with each.

Localization-based methods are attractive for their interpretable outputs, yet under adverse weather they inherit the same fragility: the visual evidence for a head proposal (a small texture patch) is precisely what rain streaks, haze veils and umbrellas corrupt first. Neither family, in their standard form, models weather as a factor.

Gap

Normal-weather counting research provides powerful backbones and mature supervision (density regression with the Bayesian loss, point localization), but treats the visual domain as homogeneous. No mechanism in these methods ensures that representations are *discriminative with respect to weather*, which, as we argue in Section 1.3, is the prerequisite for counting robustly across domains.

1.2.2 Crowd Counting under Adverse Conditions

A second line of research confronts weather explicitly. JHU-Crowd++ [1] contributed the first counting dataset containing adverse-weather scenes with weather labels, and augmented the backbone with additional class-conditioning blocks trained by cross-entropy on those labels. The conditioning supplies weather information as a side input, but the backbone features themselves remain weather-entangled.

AWCC-Net [4] is, to our knowledge, the only work that directly targets counting under *diverse* adverse weather. It learns adaptive query vectors — a set of learnable queries that attend to degradation-specific cues so that the decoder can compensate for the detected weather type without an explicit enhancement module. The design is elegant, but it concentrates the entire weather-handling burden in a Transformer module that accompanies the network into deployment, and its adaptive queries are only as reliable as the degradation cues available at test time; neither the training scheme nor the memory addresses what happens when adverse classes are rare. Both limitations are visible empirically: AWCC-Net carries roughly twice the FLOPs of the ConvNeXt-T baseline used in this thesis (Table 3.9), and its adverse-weather advantage erodes as class imbalance intensifies (Figure 1.1(d)). that extract weather information according to the image-specific degradation, avoiding customized image-enhancement modules. However, its Transformer-based design significantly increases model weight and computational cost, and — critically for this thesis — it does not address performance degradation under weather-class imbalance: as Figure 1.1(d) shows, its adverse-weather advantage erodes as imbalance intensifies.

A complementary family improves visibility first and counts second. Weather removal itself has a long lineage: dark-channel-prior dehazing [22] established single-image haze removal, and deep joint rain detection and removal [23] brought convolutional models to rain streaks; subsequent

unified models handle several degradations in one network [24]. Heavy-weather synthesis pipelines [25] generate rainy or hazy training data on clear images; direction-aware attention aggregation [26] targets hazy scenes in a single-stage manner; RestoCrowd [27] leverages image restoration coadjutant learning to count accurately in haze without weather labels; and unified all-in-one restoration models [24] remove multiple weather types with a single network. These restoration-based approaches, however, suffer from a target mismatch: restoration is optimized for perceptual fidelity, not for counting, and restored images still differ from genuinely clear images (raincoats and umbrellas remain, wet ground reflections remain). When plugged in front of a counting network, such modules yield only modest gains — an observation we quantify in Section 3.3 with the "Restore" variant of our baseline. Domain generalization techniques [28] have likewise been applied to counting, but they aim at scene-level shifts rather than the combination of weather shift and class imbalance, and NWPU-Crowd [21] provides a large-scale benchmark whose normal-scene bias motivated our synthesized NWPU-Weather variant (Appendix A.1).

A further structural observation unifies these methods: almost all of them treat the weather either as a nuisance to be *removed* (restoration) or as a category to be *classified* (conditioning). Neither treatment changes the geometry of the representation space itself — after training, normal-weather and adverse-weather features still occupy different, uncoordinated regions, and the decoder must cope with all of them. The multi-domain learning perspective adopted in this thesis differs precisely here: instead of removing or labeling the domains, it learns a representation in which domains are *explicitly separated and then explicitly aligned*, using the first contrastive stage for separation and the second for alignment. The separation stage additionally has to survive the class skew that real weather statistics impose — which leads directly to the contrastive-learning literature on imbalance reviewed next.

It is useful to organize the adverse-weather counting literature into three mechanism families, because their costs and failure modes differ structurally. *(i) Conditioning-based* methods (weather-conditioning blocks [1], adaptive query vectors [4]) feed weather information into the network as a side signal; they add a classification or encoding sub-module, and their deployed accuracy depends on the fidelity of that signal at test time. *(ii) Restoration-based* methods [24], [26], [27] pre-process the image; they inherit the whole restoration literature's machinery and its latency, and they optimize a perceptual objective that is only loosely coupled to countability. *(iii) Degradation-adaptive counting* methods learn features that absorb the degradation; this family is the least explored, and MQCL belongs to it — but, as argued above, adaptation without balance simply re-introduces the skew at the feature level. A fourth axis, orthogonal to the three families, is the *severity continuum*: weather is not categorical but graded (light haze to dense fog), and any method whose internals are organized around hard classes must place a threshold somewhere on that continuum. One of the quieter advantages of the representation-level approach of Chapter 2 is that nothing in it thresholds severity: the refiner maps representations continuously, so graded conditions are handled by graded representations rather than by a nearest-class decision.

The three reviews also demarcate what this thesis does *not* attempt. It does not propose a new counting architecture — four off-the-shelf backbones are used as-is. It does not synthesize weather (the NWPU-Weather variant exists only to broaden *evaluation*). And it does not require per-pixel restoration — no operation in the pipeline touches the input image beyond the standard training augmentations. The entire contribution lives in how the training signal is composed: which pairs are pulled apart, against which memory, in which order — a deliberately narrow intervention point whose smallness is what makes the plug-and-play claim (O3) credible.

With the three literatures mapped and the narrowness of the intervention stated, the design questions of Chapter 2 can be posed precisely. If the representation is the sensitive variable, *what* objective makes it weather-discriminative? If imbalance is the corrupting channel, *where* must balance be injected so that it reaches that objective? And if deployment economics forbid extra

machinery, *how little* new structure achieves both? The answers — a contrastive objective, a class-structured memory, and a single refiner — occupy the three sections that follow.

The thesis's contributions, stated once and referenced throughout, are:

- **C-a (formulation):** a formalization of all-weather crowd counting as imbalanced multi-domain learning, separating the domain-gap and class-imbalance obstacles and deriving acceptance gates for each objective.
- **C-b (method):** multi-queue MoCo — a class-partitioned contrastive memory with a push-time balance invariant — and the two-stage MQCL scheme that reuses the frozen memory as the anchor distribution of a representation-conversion stage.
- **C-c (analysis):** an analytical account, at the level of the objective's gradient and memory composition, of why memory balancing removes the majority-class dominance that skew otherwise imposes.
- **C-d (evidence):** a multi-benchmark empirical study (two real/ synthetic benchmarks, four backbones, seven experiment families) with requirement-level verification, including controlled imbalance stress tests.
- **C-e (reproducibility):** a complete configuration reference and reproduction checklist (Appendix), so that the results can be audited and rebuilt independently.

A tempting alternative to all three families is to *manufacture* balance: synthesize adverse weather on clear images (as GCC does in an engine [2], as heavy-weather pipelines do [25], and as the NWPU-Weather variant of Appendix A.1 does for evaluation) until the classes are even. Synthesis is genuinely useful — it is how this thesis obtains a controlled third benchmark — but as a *training* remedy for real deployments it has two limits. First, the sim-to-real gap: rendered rain and fog differ statistically from measured ones, and a model balanced on synthetic weather is balanced on the wrong statistics the moment it meets real weather. Second, and more fundamentally for counting, synthetic scenes inherit the crowd statistics of their source images; balancing the *weather* axis does not balance the joint distribution of weather, crowd density and scene type that actually drives the counting error. Synthesis complements the memory-level remedy of this thesis (it created the very benchmarks used for evaluation), but it does not replace it.

The restoration route also illustrates a general principle about *objective mismatch* that recurs throughout robustness research. Restoration networks are trained to maximize perceptual similarity (PSNR, SSIM, or adversarial fidelity) to clean images; counting accuracy is nowhere in their objective. When two tasks are related but not identical, the auxiliary model can be excellent at its own task and still deliver little to the downstream one — the restored image may look right to a human while differing from a true clear image in precisely the statistics (counter features) that matter. The empirical gap between the Restore baseline and MQCL in Chapter 3 is a measured instance of this principle, and it motivates the thesis's choice to never touch the pixels: if the representation can be made *invariant* to the weather factor, no intermediate "denoised picture" needs to exist at all.

A final distinction clarifies the problem class. *Domain adaptation* [28] assumes access to unlabeled (or labeled) target-domain data during training and seeks to transfer a model across a *single* domain boundary; *domain generalization* removes the target data but typically still assumes one source-to-target shift. Adverse-weather counting matches neither setting exactly: there are *multiple* simultaneous target conditions (rain, snow, haze of unknown severity, plus mixtures), the labeled source is skewed, and the shift occurs at test time on every frame. The appropriate framing is therefore multi-domain learning under imbalance — which is the formulation adopted in

Section 1.3 — and the remedy must be built into the model's training rather than into an adaptation phase that cannot anticipate every test condition.

Gap

Existing adverse-weather counting methods either (i) inject weather information as an auxiliary input rather than learning weather-aware features, (ii) restore images and accept the residual domain gap, or (iii) specialize to a single weather type. None of them treats *class imbalance across weather domains* as a first-class design constraint, and most add substantial computation to the deployed model. This double gap — weather awareness *and* imbalance awareness at low cost — is precisely where this thesis positions itself.

1.2.3 Contrastive Learning and Class Imbalance

Contrastive learning [29], [30] learns representations by maximizing the similarity of representations from positive pairs while minimizing that of negative pairs, and has become a cornerstone of unsupervised and supervised representation learning. Instance-discrimination frameworks differ mainly in how the negative pool is maintained: the memory bank of Wu et al. [31] stores one vector per training sample, SimCLR [30] draws negatives from the current large batch, and MoCo [32] maintains a *queue* of keys produced by a momentum-updated encoder, combining a large, consistently encoded negative pool with low memory cost. The dictionary view of MoCo deserves emphasis, because it is the lever this thesis pulls: the queue is a large, slowly evolving *sample* of the data as encoded by recent versions of the network, and the contrastive loss treats it as a stand-in for the data distribution itself. Everything the loss "knows" about the data, it knows through this sample — its size sets the resolution of the negative signal, and its *composition* sets which classes the encoder is asked to tell apart. None of the three maintenance schemes controls composition: the bank mirrors the dataset, the batch mirrors the sampler, and MoCo's queue mirrors the stream. Composition is the open variable the multi-queue design closes. There is a conceptual bridge worth making explicit: weather can be read as a natural, uncontrolled *augmentation* across images — the same street under drizzle and downpour are, semantically, the "same" scene rendered through two domain conditions. Standard contrastive learning creates such pairs artificially (two crops of one image); adverse weather provides them for free across the dataset, except that they arrive at frequencies the practitioner does not choose. The multi-queue can be understood as a way of harnessing this free augmentation axis in a balanced way: it does not manufacture domain pairs, it merely guarantees that the domain axis is represented uniformly in the negative pool, so that whatever natural augmentation the environment provides is actually learned from. MoCo v2 [33], BYOL [34], SimSiam [35] and DINO [36] subsequently refined the recipe toward stronger augmentation and negative-free objectives, and supervised contrastive learning [37] generalized the loss to exploit label information. Wang and Isola [38] characterized contrastive learning through alignment and uniformity, and showed a useful tolerance of the objective to *semantically similar* negatives — a property MQCL deliberately relies on when samples of the same weather but different images are treated as negatives (Section 2.2.4).

Crucially for this thesis, Assran et al. [39] showed that the contrastive formulation carries a hidden prior toward learning features that *uniformly cluster* the data. When pretraining data is class-imbalanced, this uniform prior clashes with the skewed data distribution and hampers the learned features. The imbalance-aware contrastive literature proposes remedies along three directions. KCL [40] draws k same-class instances into the positive set, balancing the *positive* side but leaving the negative pool imbalanced. BCL [41] balances the *gradient* contribution across classes and enforces class balance within each batch. TSC [42] injects cluster-center priors into the supervised contrastive loss. SBCL [43] splits head classes into similarly sized subclasses so that all learned clusters are of comparable size. MAK [44] addresses imbalance at the data level by augmenting tail classes with realistic samples. Beyond contrastive objectives, class-balanced re-weighting [45] is a

standard data-level baseline. An intuitive data-level remedy for our task would be to supplement adverse-weather images, but counting supervision requires head-position labels for every added image, which is exactly what is scarce — so loss- and memory-level remedies are the practical route.

Formally, Wang and Isola [38] decompose the contrastive objective into *alignment* — pulling positive pairs together — and *uniformity* — spreading representations over the hypersphere, and show that the InfoNCE family optimizes a bound that trades the two terms off. The uniformity term is where imbalance enters: uniformity is defined with respect to the *sampling distribution* of the negatives, so a skewed negative pool yields a geometry that is uniform over the majority and compressed over the tail. Assran et al. [39] sharpened this into the uniform-cluster-prior result. Read through this lens, the design space of remedies is exactly the space of ways to change the sampling distribution seen by the uniformity term: re-weight gradients (BCL), re-sample positives (KCL), inject priors (TSC), split clusters (SBCL) — or, as in this thesis, repair the sampling distribution itself at the point of memory maintenance. The last option is the least invasive: the loss is untouched, the theoretical guarantees of the objective continue to apply verbatim, and the repair is a data-structure change rather than a new objective.

A second, often overlooked channel of skew is the *pretraining* inheritance. Counting backbones are initialized from large-scale classification pretraining (ImageNet [46] and successors), and those corpora are themselves dominated by clear, well-lit photography; the encoder therefore starts its counting fine-tuning from features already tuned to the majority domain. Fine-tuning on a skewed counting dataset then reinforces, rather than corrects, the initial imbalance of attention. This double skew — pretraining skew compounded by dataset skew — is why the adverse-weather failure of conventional pipelines is so persistent, and why a training-scheme remedy (which acts on both stages of the pipeline) is better placed than an architectural patch (which acts only after pretraining has already shaped the features). A related subtlety concerns *which* labels the contrastive signal consumes. Supervised contrastive learning [37] assumes semantic class labels and pulls all same-class instances together; applied naively to weather labels, it would flatten each weather domain to a point — exactly the over-averaging that Strategy 1 of Section 3.3 exposes empirically (same-label positives underperform same-image positives by 6.1 adverse MAE). The multi-queue design consumes labels only to *route* keys into sub-queues, never to define positives; the positives remain instance-level (same image), preserving within-domain variation while the memory-level balance fixes the cross-domain signal. This division — labels for memory organization, instance identity for the objective — is what lets the method benefit from weather annotations without over-committing to them. Beyond classification, contrastive objectives have been transplanted into dense prediction and low-level vision — depth, segmentation and restoration — where they typically align pixel- or region-level features across augmented views. Those uses, however, are domain-agnostic in exactly the sense this thesis criticizes: they enforce view-consistency, not domain structure. MQCL's use of contrastive learning is different in kind, not only in degree: the negative pool is given an explicit *domain-axis* structure (one queue per weather class), and the second stage exploits that structure as an alignment target. To our knowledge, using a class-structured memory both for imbalance-robust pretraining *and* as the frozen anchor distribution of a subsequent conversion stage is novel to this thesis.

Because the imbalance argument of Chapter 2 rests on the gradient structure of the contrastive objective, it is worth writing that structure out now. For an anchor Q, the gradient of the InfoNCE loss with respect to the anchor takes the form $\partial\mathcal{L}/\partial \mathrm{Q} = \tau^{-1}\left(\sum_a \; p_a \, \mathrm{K}_a - \mathrm{K}_+\right)$, where $p_a = \mathrm{softmax}_a(\mathrm{Q}\cdot \mathrm{K}_a/\tau)$ is the distribution the objective fits and K_+ is the positive key. The learning signal is thus a *weighted average over the memory*: every queued key exerts force proportional to its softmax mass, and the composition of the memory decides which classes appear in that average at all. With a skewed memory, the majority class occupies most of the mass $\sum_a \; p_a$, so the "push"

component of the gradient is mostly majority-to-anchor; minority keys are present but their collective softmax mass is small, and the encoder is never forced to separate them. This is the precise, gradient-level statement of the failure that the multi-queue repairs: by equalizing the *counts* $|Q_c|$ across classes, it equalizes the opportunity of each class to appear in $\sum_a \ p_a$, regardless of n_c. The loss function, the temperature, and the augmentation policy are all left exactly as the theory assumes them. Finally, the contrastive remedies should be read against the broader long-tailed learning toolbox. Data-level remedies (oversampling, class-balanced sampling, synthesized minorities [45]) act before the model; loss-level remedies (re-weighting, logit adjustment) act on the objective; memory-level remedies act on what the objective *sees*. For crowd counting the data-level route is largely closed — adverse-weather images cannot be synthesized with credible head-position labels at scale, which makes the memory-level route of this thesis, and the loss-level alternatives it is compared against in Section 3.3, the practically relevant families. A complementary axis of variation is *what plays the role of the compared unit*. Instance-discrimination methods compare individual embeddings; prototype- and center-based variants compare against class means or learned cluster centers, which reduces variance but fixes the class geometry earlier in training. The multi-queue design deliberately stays instance-level *inside* each sub-queue (keys remain individual samples) and applies class structure only to the *partition* of the memory: per-class balance without per-class prototypes. This preserves the fine-grained negative diversity that prototype methods average away — important for counting, where the within-class variation of a weather domain (drizzle to downpour) is itself informative — while still obtaining the balance that motivated prototype methods. The data-economics argument deserves its own paragraph, because "just collect more adverse data" is the most common external suggestion. Three costs make it impractical at the required scale. *Acquisition*: adverse events are rare and geographically uneven; a camera network accumulates them at the weather's schedule, not the dataset's. *Annotation*: every collected image needs head positions, and dense adverse scenes are the slowest, hardest images to annotate — inter-annotator disagreement rises exactly where the data is scarcest. *Coverage*: the taxonomy of degradations is open-ended (drizzle, sleet, fog at dusk, snow with glare), so even a large collected set leaves the long tail long. These costs do not argue against collecting adverse data where it exists — JHU-Crowd++ is precious precisely because someone did — but they rule out collection as the *primary* remedy, and they shift the burden to methods that extract more robustness from the data already labeled. Naive intermediate fixes fail in instructive ways. The failure modes are instructive enough to generalize: any remedy that re-uses scarce minority data *more* amplifies memorization, while any remedy that discards majority data discards representation quality. A third family worth naming is *hard-negative mining*: prioritizing the negatives with the largest current similarity so that the gradient budget is spent where the loss is. Mining interacts with imbalance in a particularly unfortunate way — under a skewed memory, the hardest negatives are overwhelmingly same-class (majority) instances, so mining amplifies exactly the within-majority pressure the memory skew already over-provides. This is one more argument for fixing the composition *before* optimizing over it, and it is consistent with the finding of [38] that the objective's tolerance is greatest for negatives that are semantically near the anchor — which, in our data, are precisely the balanced adverse-weather keys the multi-queue guarantees. Over-sampling the few adverse-weather images balances the frequencies but repeats each minority image so often that the encoder memorizes it — the model becomes robust to the *training rainstorms* and blind to new ones. Down-weighting the majority loss restores balance but discards most of the training signal, since the majority carries the scene-diversity the encoder needs for counting in general. Both failures point to the same conclusion: the imbalance must be repaired in a channel that does not consume the scarce data more aggressively — which a class-structured memory accomplishes, because it re-uses the *same* minority samples as balanced negatives without amplifying their counting gradients.

The later self-supervised line — BYOL [34], SimSiam [35] and DINO [36] — removes the negative pool altogether, avoiding collapse through architectures and stopping gradients instead. A negative-free objective, however, cannot express the structure this thesis needs: our second stage

requires an explicit pool of normal-weather keys to serve as the conversion target, and the first stage's remedy for imbalance (Section 1.2.3's gap) is precisely to restructure the negative pool by class. The queue-based substrate of MoCo is therefore retained deliberately: the queue is not merely a memory optimization but the component that the multi-queue design turns into a class-balanced, stage-to-stage stable summary of the weather domains.

Reading the imbalance remedies against the requirements.

Although the four imbalanced-contrastive strategies will only be compared experimentally in Chapter 3, their mechanisms already indicate different degrees of fit with the counting setting, and it is worth making the comparison precise here. KCL [40] expands the positive set to k same-class instances; its *positive* balancing is well matched to long-tailed classification, where every instance carries a reliable class label, but in our task it would couple the objective to the weather label at every gradient step and — more importantly — leave the negative pool untouched, so the dominant failure mode identified above persists. BCL [41] balances gradients and enforces class-balanced batches; this attacks the frequency mismatch directly, but batch balance for a 25:1 skew means replicating scarce images many times within a batch, which concentrates the counting gradients on a handful of scenes. TSC [42] injects cluster priors estimated from labeled data; the priors are only as good as the clusters, and weather clusters must be learned rather than given. SBCL[43] redistributes cluster sizes by splitting head classes into subclasses; the fit problem is subtler, since a downstream stage (ours: the refiner) may depend on the unsplit cluster structure. Memory-level balancing — our proposal — is the only remedy that requires neither new losses, nor label consumption inside the gradient, nor batch restructuring: it changes *what the existing loss sees*, which makes it simultaneously cheap, orthogonal to the other remedies, and composable with them should a future method want both.

Gap

Imbalance-aware contrastive methods were developed for *classification* pretraining, where the class of every instance is known and the representation feeds a linear classifier. Their behavior in an *imbalanced multi-domain regression* setting — where a class-conditional memory must remain consistent across two training stages, and the representation feeds a counting decoder — is unexplored. This thesis fills that gap: multi-queue MoCo transplants the class-balancing idea into the memory queue itself (rather than the loss gradients or batch composition), keeps the queues stable between stages so that the second stage can exploit them as anchors, and couples the objective with a counting loss. Table 1.1 positions MQCL among the reviewed methods.

Table 1.1: Positioning of this thesis among related research lines. MQCL is the only method that simultaneously provides weather-aware representations, explicit imbalance handling, and a lightweight plug-and-play design.

Method family	Weather-aware	Imbalance-aware	Lightweight
Density / localization counting [3], [7], [9], [16]	✗	✗	✓
Restore-then-count [24], [27]	partial	✗	✗
Weather-conditioned counting [1], [4]	✓	✗	✗
Imbalanced CL [40], [41], [42], [43]	✗	✓	✓
MQCL (this thesis)	✓	✓	✓

Synthesis: three gaps, one design response

The three reviews converge on three concrete gaps, which this thesis states explicitly because the method of Chapter 2 answers them one for one. **G1 — the awareness gap:** counting networks are trained and validated as if the visual domain were homogeneous, so their representations carry no

explicit structure for weather; yet multi-domain recognition is the stated prerequisite for any multi-domain regression. **G2 — the balance gap:** wherever representations are learned with a contrastive objective, the class-composition of the negative memory silently re-imports the weather skew of the dataset into the learning signal; no architecture change can compensate for a biased curriculum. **G3 — the deployment gap:** methods that do model weather (conditioning, restoration) spend their budget at inference time and depend on labels or modules that real systems cannot reliably provide.

The design response of this thesis maps directly: a contrastive pretraining stage gives the encoder explicit weather structure (G1), a class-partitioned memory gives the objective a balanced curriculum independent of data frequencies (G2), and the entire apparatus is training-time scaffolding — at deployment only the backbone, a small refiner and the counting head remain (G3). The remainder of the thesis is the derivation, the evidence, and the boundary conditions of that response.

1.3 Problem Formulation and Objective Setting

1.3.1 Task Formulation

Let $\mathcal{D} = \{(X_i, Y_i, w_i)\}_{i=1}^{M}$ be a training set of M images, where $X_i \in \mathbb{R}^{H\times W\times 3}$ is an image, $Y_i = \{(x_j, c_j)\}_{j=1}^{n_i}$ contains the annotated head positions $x_j \in \mathbb{R}^2$ with associated count masses c_j, and $w_i \in \{1, ..., B\}$ is the weather class label (e.g., normal, haze, rain, snow, so $B = 4$ for JHU-Crowd++). A counting model is a mapping $F(X;\theta) = \widehat{D} \in \mathbb{R}^{H'\times W'}$ that produces a density map whose total mass is the predicted count:

$$\widehat{Y} = \sum_{u=1}^{H'} \sum_{v=1}^{W'} \widehat{D}(u, v). \qquad (1.1)$$

The weather label w_i is available *only during training*; at inference time the model must handle images whose weather is unknown. This asymmetry is what rules out solutions that condition the deployed model on a predicted weather class. The asymmetry induces three constraints on any acceptable solution, used as design filters throughout this thesis:

- **C1 (no test-time labels).** The weather class may organize training but must not be an input at inference — neither as a hard decision nor as a soft prior.
- **C2 (bounded test cost).** Any added module runs on every frame, so its cost is multiplied by the deployment's lifetime traffic; it must be an order of magnitude smaller than the backbone it augments.
- **C3 (unknown conditions).** Test weather may be absent from the training taxonomy (mixtures, unseen severity); the method must degrade gracefully rather than fail on out-of-taxonomy inputs.

Restoration pipelines strain C2; classifier-conditioned models violate C1 and are fragile under C3; per-weather model banks violate both. These constraints are the practical content of objective O3, and the design of Chapter 2 is checked against them before any experiment is run.

1.3.2 The Two Obstacles

P1 — Domain gap. Adverse weather induces a covariate shift between $\mathcal{D}_{\text{adv}}$ (rain/snow/haze images) and $\mathcal{D}_{\text{nor}}$ (normal images). A decoder trained on the union of both domains receives statistically inconsistent features and produces degraded density maps. Formally, for the same

underlying crowd configuration, the feature distribution $p_\theta(R|\text{adv})$ induced by the encoder differs from $p_\theta(R|\text{nor})$, while the desired counting behavior is identical for both.

P2 — Class imbalance. The classes of $\mathcal{D}$ are far from uniform: in JHU-Crowd++ the training split is dominated by normal-weather images, with rain at ∼3%. We quantify imbalance by the ratio $\rho = \max_c n_c / \min_c n_c$ over per-class training counts n_c; in our controlled stress subsets ρ ranges from *1* (balanced) to *25* (Section 3.3). Imbalance breaks contrastive representation learning in a specific way: the uniform prior implicit in the contrastive objective — the queue is, in MoCo's own terms, a *dictionary* of the data: large (decoupled from batch size), consistent (momentum-encoded), and therefore an operating definition of what the objective believes the data to be. When the dictionary's composition matches the true class balance, uniformity is a benign inductive bias; when it matches the skewed frequencies instead, uniformity becomes a distortion — the encoder is taught to spread a nine-tenths-majority sample space evenly, which it can only do by entangling the minority. The queue is thus not infrastructure but *epistemology*: it is where the objective's view of the data lives, and it is the view this thesis repairs. [39] mismatches the skewed sampling distribution, so the memory content — and hence the negative pool — is dominated by the majority class. Both obstacles compound: the domain gap of P1 means that the majority-dominated features are also *inconsistent features for the same semantic content*, so a decoder trained on their union learns an average that is faithful to neither. Any remedy must therefore act on the representation layer — loss re-weighting at the density output cannot undo an entangled encoder.

The imbalance obstacle can now be stated in the same language. The idealized multi-domain objective would minimize the class-balanced risk $\sum_{c=1}^{B} \mathbb{E}_{(X,Y)\sim p_c}[\ell(F(X;\theta), Y)]$ with equal weight on every weather domain. What ordinary empirical risk minimization actually minimizes is $\sum_c (n_c/M)\,\mathbb{E}_{p_c}[\ell]$, in which the weight of domain c is its frequency n_c/M — and, as reviewed above, the same skew leaks into the contrastive memory through the streaming fill of the queue. P2 is therefore not merely "few examples of some classes"; it is a systematic mismatch between the weighting the practitioner wants (uniform over domains) and the weighting that both the loss and the memory quietly apply (proportional to frequency). Concretely for P1: when rain coats a pedestrian, the annotation Y is unchanged — the same person still stands there — but the feature distribution $p_\theta(R|\text{rain})$ shifts because every cue the encoder uses (texture, contrast, contour) has been perturbed. The count supervision is therefore *domain-invariant by ground truth* while the features are *domain-dependent by construction*; the decoder is asked to map two different feature distributions to one counting semantics. That is the formal content of the domain gap, and it explains why the remedy must act before the decoder: aligning features (making $p_\theta(R|\text{rain}) \approx p_\theta(R|\text{nor})$) dissolves the problem, whereas re-training the decoder on both domains merely averages two incompatible regression problems. Any remedy must re-balance *both* channels, which is why the method of Chapter 2 keeps the counting loss as designed but restructures the memory that feeds the contrastive channel.

Finally, the formulation fixes what any remedy must *preserve*: the counting semantics. Whatever transformation is applied to features, the decoded quantity must remain the integral of a density whose meaning is anchored to head positions — invariance of the output under feature manipulation is what separates a robustness method from a shortcut. This is the deep reason the thesis couples every contrastive objective to a counting loss (Eqs. (2.10) and (2.13) of Chapter 2) rather than treating representation alignment as a free-standing pretraining exercise: the alignment is only valuable insofar as the count semantics survive it, and the paired losses are the enforcement mechanism.

A concrete scenario makes both obstacles visible in a single image. Consider a street photographed during heavy rain: pedestrians wear dark raincoats (low contrast against wet asphalt), carry umbrellas that occlude neighbors' heads and shoulders, and are surrounded by rain streaks whose high-frequency appearance resembles texture. A feature tuned on clear images responds to this

scene with a systematically different activation pattern — edge energy drops, texture statistics rise, color distribution darkens (P1). Now place that scene in a training set where 88% of images are clear: the counting loss averages the gradient over a distribution in which this image is an outlier, and the contrastive memory — filled by the same distribution — offers mostly clear-weather negatives (P2). The image is thus doubly disadvantaged: its features are shifted by the weather and its learning signal is diluted by its rarity. MQCL's two stages attack the two disadvantages in order, and the experiments of Chapter 3 are arranged so that each obstacle's contribution to the final improvement can be read off separately.

One further formalization detail matters for what follows. Weather and scene are *correlated* confounders: in real datasets, rainy images are also, say, more often street scenes shot at night. A method that merely overfits scene cues could masquerade as weather-robust on a given split. This is why the empirical protocol of Chapter 3 evaluates on weather subsets *across* multiple scene types (the JHU-Crowd++ test scenes span both conditions within each weather label) and on a synthetic set whose scene content is held fixed while weather varies — the two together separate the weather axis from the scene axis. The formal object of study throughout is the weather conditional feature distribution $p_\theta(R|w)$ and its behavior under balanced versus skewed memory, never a scene-conditional surrogate. It is also worth stating why weather is a *harder* shift than many studied in domain-adaptation literature. First, it is partly *semantic*, not only photometric: rain changes what people *wear and carry* (raincoats, umbrellas), so the shift enters the object appearance itself and cannot be inverted by photometric normalization. Second, it is *open-set*: the space of degradations is a continuum of type–severity mixtures, so any method organized around a closed set of weather categories inherits a taxonomy risk. Third, it is *entangled with density*: adverse scenes in benchmarks skew towards certain locations and times, so weather co-occurs with density extremes. The design of Chapter 2 answers each property with a structural choice — representation-level learning for the semantic part, a balanced memory rather than fixed categories for the open-set part, and per-weather evaluation for the entanglement.

Table 1.2: Observed weather-class imbalance in the evaluation data. ρ is the ratio of the largest to the smallest class count.

Dataset / subset	Classes (B)	Minority share	ρ
JHU-Crowd++ (full training set)	4	rain ≈ 3%	≈ 25–30
JHU-Crowd++ stress subsets (§3.3)	4	10/280 ≈ 3.6%	1 to 25
GCC (training locations)	7	rarest of 7 types	> 10
NWPU-Weather (Appendix A.1)	3	5% haze, 5% rain	≈ 18

Table 1.2 collects the imbalance ratios of the datasets used throughout the thesis; every benchmark exhibits ρ well above the regime where the uniform prior of contrastive learning can be taken for granted.

The position of this thesis can be stated in one sentence: *counting robustly under adverse weather is less a perception problem to be repaired at the image level than a representation problem to be solved at the feature level, and the feature level itself is untrustworthy unless the class imbalance of the training weather is repaired first.* Everything that follows — the two-stage design, the multi-queue memory, the contrastive refiner — is the working-out of that sentence. The remaining sections of this chapter ground each clause: Sections 1.2.1 and 1.2.2 review what counting and restoration methods assume about the input domain; Section 1.2.3 reviews what contrastive learning assumes about the data distribution; and Section 1.3 formalizes both assumptions and their failure under imbalance.

1.3.3 Scope and Assumptions

Three assumptions bound the problem as studied in this thesis. First, weather labels are assumed to exist at *training* time (as provided by JHU-Crowd++ or by synthetic generation as in GCC); they are not needed at inference, and this asymmetry is exploited rather than assumed away. Second, adverse weather is treated as a *domain* factor — a systematic, class-level shift — rather than as arbitrary corruption; per-pixel noise unrelated to weather is not modeled. Third, the method targets counting accuracy; localization quality (nAP) is out of scope, although the representation learned here could plausibly support localization heads. Within those bounds, the problem is stated openly: no assumption is made about *which* weather classes exist (the method takes B from the data), about the severity distribution within a class (the memory balances classes, not severities), or about the ratio of normal to adverse data (experiments span balanced to 25:1 skew). The one hard dependence — training-time weather labels for the queue push — is treated as part of the annotation contract of the benchmark, on a par with the head positions themselves, and its removal is identified in Section 3.4 as a natural next step rather than assumed away here.

1.3.4 Objectives and Requirements

The three objectives stated in Section 1.1 are made testable through five requirements:

Table 1.3: Requirements derived from the objectives. Each requirement is checked by a specific design element (Chapter 2) and a specific experiment (Chapter 3).

ID	Requirement	O	Verified by
R1	Adverse-weather accuracy improves substantially over the baseline	O1	Tables 3.2–3.4
R2	Robustness holds as imbalance intensifies ($\rho \uparrow$)	O1	Fig. 3.5, Table 3.7
R3	Normal-weather accuracy does not degrade (ideally improves)	O2	Tables 3.2–3.4
R4	Adaptable across backbones with unchanged hyperparameters	O3	Table 3.2 (four encoders)
R5	Added parameters/FLOPs are small relative to the baseline	O3	Table 3.9

For the requirements to be checkable rather than aspirational, each is given a quantitative acceptance gate, fixed before the experiments of Chapter 3: R1 requires a reduction of at least 15% in adverse-weather MAE over the matched baseline; R2 requires that the adverse-weather degradation across the imbalance stress sweep be at most half that of the baseline; R3 requires no regression on normal-weather MAE for any backbone; R4 requires the same hyperparameters for all backbones with improvements on each; R5 requires the inference-time overhead to stay below 20% in both FLOPs and parameters. Chapter 3 returns to this list and reports the measured value against each gate.

1.4 Research Approach and Thesis Organization

The analysis above suggests that the counting model needs an internal, weather-aware representation before any counting refinement makes sense, and that learning such a representation requires a memory mechanism immune to class imbalance. We therefore split training into two stages. In the first stage, *Weather-aware Representation Learning* (WRL), the backbone is trained with a counting loss and a contrastive loss based on our multi-queue MoCo, which stores class-balanced keys in one sub-queue per weather class and thereby restores agreement between the memory distribution and the contrastive prior. In the second stage, *Contrastive-learning-guided Representation Refining* (CRR), the encoder is frozen, the queues are retained as stable anchors, and

a lightweight refiner is learned that converts adverse-weather representations into the normal-weather domain; the counting decoder then operates on a single domain. At inference only the encoder, refiner and decoder run — no weather classifier, no restoration module — so the method is plug-and-play on any backbone and adds only a small refiner to the deployed cost.

The mapping between contributions, objectives and chapters is summarized in Table 1.4. The rest of this thesis is organized as follows. Chapter 2 develops the method formally: it fixes notation, derives the multi-queue MoCo objective from its vanilla counterpart, presents the refining objective, and analyzes complexity. Chapter 3 describes the implementation, the experimental protocol, and the complete empirical study — comparisons with the state of the art, ablations of every design decision, imbalance stress tests, representation visualizations and computational cost — and closes with limitations and conclusions. In detail: Chapter 2 states the architecture and notation (§2.1), derives the WRL stage including the multi-queue objective and its imbalance analysis (§2.2), derives the CRR stage (§2.3), and packages both stages as algorithms with a cost model and a requirements check (§2.4); Chapter 3 documents the implementation (§3.1), datasets and metrics (§3.2), the full empirical study (§3.3), and closes with analysis, threats to validity, limitations and conclusions (§3.4). Appendices provide a synthesized third benchmark (NWPU-Weather), supplementary ablations, the complete training configuration and a reproduction checklist. The accompanying artifacts follow the same discipline: the bibliography separates the works that define the task from those this thesis introduces; the figures are generated from vector sources where available; and each results table carries its backbone and dataset in the caption so that no claim depends on an unlabeled default. Readers who wish to verify a single claim can do so through three hops at most — claim, table, configuration.

Chapter summary.

This chapter established the ground the thesis stands on. Adverse-weather crowd counting was motivated operationally (weather is when counting matters most and works worst), positioned scientifically (normal-weather counting is mature, restoration-based and conditioning-based robustness exist, but neither handles imbalance nor cost), and connected to its theoretical substrate (contrastive learning, whose uniformity prior breaks under skewed memories). The problem was formalized as imbalanced multi-domain learning with two obstacles — the domain gap P1 and class imbalance P2 — and three objectives were operationalized as five checkable requirements with quantitative gates. The research approach — two contrastive stages around a class-balanced memory — was previewed with its design rationale and contribution structure. Chapter 2 now carries the full derivation. The thesis closes with a synthesis chapter-level argument rather than a summary alone: the requirements of Table 1.3 are verified one by one, the failure modes the design pre-empts are stated explicitly, and the limitations are given with the same prominence as the results — a structure intended to let the reader accept or challenge each link of the argument independently. Two conventions are used throughout. First, equations are numbered per chapter and are meant to be read as a chain — Chapter 2 provides a map (its Table 2.6) of which equation addresses which obstacle. Second, every experimental claim in Chapter 3 is tied to a numbered table or figure, and every requirement of Table 1.3 to an explicit verification entry, so that the argument can be audited requirement by requirement rather than taken on trust.

Relation to the author's published work.

The core of this thesis — the MQCL method, the multi-queue MoCo scheme and the principal experiments — was published in condensed form as a conference paper at ECAI 2024 (arXiv:2408.05956). The thesis extends that work in three directions: the method is derived in full formal detail, with an explicit imbalance analysis (Eq. (2.9)) and design-alternative discussion that the page limit excluded; the empirical study is enlarged with the controlled imbalance stress test, the supplementary significance analysis and the synthesized NWPU-Weather benchmark presented as

first-class results (Section 3.3, Appendix A.1); and the whole is positioned against a broader review of the counting, restoration and imbalanced-contrastive-learning literatures.

Table 1.4: Contributions mapped to objectives and chapters.

Contribution	Objective	Chapter	Key evidence
Multi-queue MoCo: class-balanced memory for contrastive learning	O1	§2.2	Tables 3.5–3.7; Fig. 3.4
Contrastive-guided representation refining (CRR stage)	O1, O2	§2.3	Table 3.4; Fig. 3.3
Two-stage plug-and-play training scheme MQCL	O1, O2, O3	§2.1, §2.4	Tables 3.2–3.4, 3.7
Extensive empirical study incl. imbalance stress tests	O1, O3	Ch. 3	Figs. 3.1–3.5

Chapter 2: The Proposed Method: Multi-Queue Contrastive Learning

This chapter develops MQCL as one continuous derivation. Section 2.1 fixes the architecture and the notation; Section 2.2 derives the first training stage from the counting supervision up to the multi-queue contrastive objective; Section 2.3 derives the refining stage; Section 2.4 summarizes both stages as algorithms and analyzes the computational cost. Equations are numbered (2.1) through (2.15) and are deliberately chained: every equation exists because a term, a set or an assumption of a previous equation breaks down under weather class imbalance. Each key equation is discussed with a fixed pattern — *symbol walkthrough*, *why this form*, *consequence* — and is closed by a *bridge* to the next one.

2.1 Method Overview and Design Rationale

2.1.1 Architecture

The architecture of MQCL is shown in Figure 2.1. It consists of five learnable components and one memory structure:

- an on-line encoder E_Q with parameters θ, which extracts a representation $R_Q \in \mathbb{R}^{H\times W\times C_1}$ from an augmented view I_Q of the input image;
- a momentum encoder E_K with parameters $\tilde{\theta}$, which extracts $R_K \in \mathbb{R}^{H\times W\times C_1}$ from a second augmented view I_K of the same image;
- two projection heads P_Q and P_K, which pool and project representations to embedding vectors $Q, K \in \mathbb{R}^{C_2}$;
- a multi-queue memory $\mathcal{Q}$ of B equal-length sub-queues, one per weather class, storing key vectors;
- a crowd counting head (decoder) D that produces a density map from R_Q; and
- a refiner $g_\phi(\cdot)$, trained only in the second stage, which converts representations towards the normal-weather domain.

At inference time only E_Q, g_ϕ and D execute; the momentum branch, the queues and the projection heads are training-time machinery.

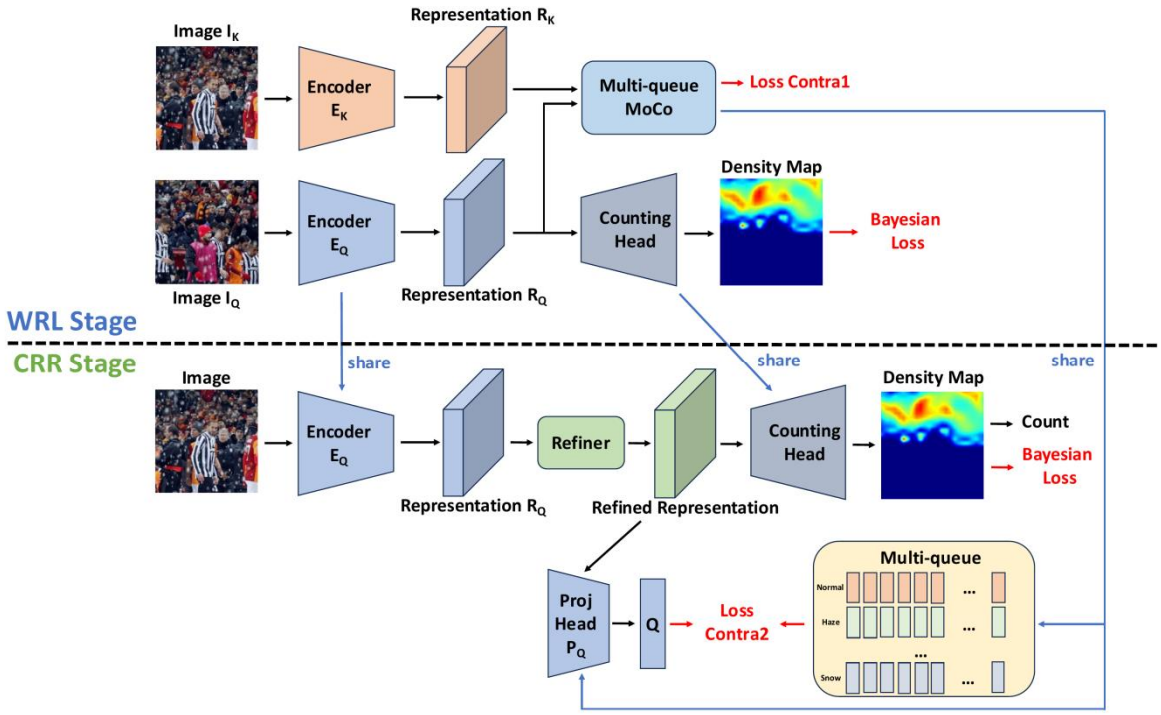


Figure 2.1: The architecture of MQCL. Upper half: the WRL stage learns weather-aware representations via multi-queue MoCo contrastive learning and Bayesian-loss supervision. Lower half: the CRR stage freezes the encoder, re-uses the frozen queues as anchors, and trains a lightweight refiner g_ϕ that pulls adverse-weather representations towards the normal-weather domain.

2.1.2 Why two stages, and why contrastive learning

The design answers two questions. *Why not one stage?* The refiner is premised on stable and consistent target representations: converting representations that are still drifting is ill-posed, because the domain the refiner must approach changes under its feet. Splitting training into a representation-learning stage (WRL) and a representation-converting stage (CRR) separates these two targets and lets the queues act as a stable interface between them — the queues are the only component that survives both stages unchanged.

Why contrastive learning rather than resampling or re-weighting? The prerequisite for multi-domain counting is that representations of different weather domains be *discriminative*, i.e., images with similar weather conditions map to similar representations and images with dissimilar conditions map to distant ones. Contrastive learning optimizes exactly this geometric property, and it does so without requiring the class label at inference. Re-sampling or re-weighting the counting loss would balance gradient contributions but would not, by itself, impose any structure across weather domains; supervised conditioning [1] injects weather as an input but leaves the representation itself entangled. The price of contrastive learning is its sensitivity to class imbalance — which is precisely the obstacle addressed by the multi-queue structure of Section 2.2.4.

2.1.3 Notation

Table 2.2 summarizes the symbols used throughout this chapter. All sub-queues are written Q_c for class $c \in \{1, ..., B\}$, and $Q = \{Q_1, ..., Q_B\}$ denotes the multi-queue.

2.1.4 Component lifecycle across stages and deployment

Because the two training stages and the deployment phase use different subsets of the components, it is worth fixing their lifecycle explicitly (Table 2.1). Three properties deserve emphasis. First, the *queues are the only state that crosses the stage boundary*: they are written during WRL, frozen and

read during CRR, and discarded at deployment. They act as a frozen summary of the training domains, which is what allows the CRR objective, Eq. (2.12), to define its positive set without consulting the dataset. Second, the projection heads have asymmetric fates: P_K exists only while keys are being produced (WRL), whereas P_Q is re-positioned behind the refiner in CRR so that anchors and frozen keys remain comparable in one embedding space — without this re-positioning, the contrastive term of Eq. (2.12) would compare refined representations against keys in a projection it never sees. Third, the deployed pipeline is *strictly a subset* of the baseline stack plus the refiner: encoder, refiner, decoder. Everything contrastive is scaffolding.

To make the machinery tangible, it helps to trace the *life of an image* through the three phases of the system. *During a WRL iteration:* the image is cropped twice; the two crops travel through E_Q and E_K; the on-line branch continues through P_Q and D, producing both the anchor vector and a density map; the momentum branch produces one key, which is pushed into the sub-queue of the image's weather class; the anchor then reads the *entire* multi-queue — all $B \cdot L$ keys — to compute its loss against Eq. (2.7); both losses are summed as in Eq. (2.10) and gradients reach θ, the projection head and D. *During a CRR iteration:* the image is cropped once; only the frozen encoder runs; the refiner edits the representation; the frozen P_Q projects the refined representation and reads the *frozen* queues, whose normal-weather sub-queue is now the positive set (Eq. (2.12)); the refined representation is decoded and supervised by the Bayesian term; gradients update only g_ϕ and D. *At inference:* the image (any weather) is encoded once, refined once, decoded once; Eq. (1.1) yields the count. Three passes in, one count out; no queue, no momentum branch, no labels. The reader is encouraged to keep these three traces in mind — every equation in this chapter is exercised by exactly one of them.

Table 2.1: Component lifecycle: which components are trained, frozen or absent in each phase.

Component	WRL	CRR	Deployment	Trained by	Equation
Encoder $E_Q(\theta)$	trained	frozen	used	both losses	(2.10)
Momentum encoder E_K	updated	absent	absent	EMA (no grad.)	(2.4)
Proj. head P_Q	trained	frozen	absent	$\mathcal{L}_{\text{contra}}$	(2.7)
Proj. head P_K	trained	absent	absent	$\mathcal{L}_{\text{contra1}}$	(2.7)
Multi-queue $\mathcal{Q}$	written	frozen	absent	push rule	(2.8)
Decoder D	trained	trained	used	$\mathcal{L}_{\text{bayesian}}$	(2.3)
Refiner g_ϕ	absent	trained	used	$\mathcal{L}_{\text{CRR}}$	(2.13)

Table 2.2: Central notation of Chapter 2.

Symbol	Meaning
X, (x_j, c_j)	input image; position and count mass of annotated head j
$F(X;\theta)$, $\widehat{D}$	encoder network; predicted density map
d_j, D_σ	rendered anchor density; assembled density target
$E_Q, E_K, \tilde{\theta}, \mu$	on-line encoder, momentum encoder, its parameters, momentum coefficient
$R_Q, R_K \in \mathbb{R}^{H\times W\times C_1}$	encoder representations of the two augmented views

Symbol	Meaning
P_Q, P_K	projection heads (pooling + MLP)
$Q_i, K_a \in \mathbb{R}^{C_2}$	anchor (query) vector; memory key vector
$\mathcal{Q}_c \in \mathbb{R}^{L \times C_2}, \mathcal{Q}$	sub-queue of class c with length L; the multi-queue
I, τ	mini-batch of anchors; temperature
$P(i), \boldsymbol{A}, \boldsymbol{N}$	positive index set; all queued indices; normal-weather indices
g_ϕ, R', Q'	refiner; refined representation; its projection
λ_1, λ_2	weights of the Bayesian loss in the two stages

2.1.5 Task output

The quantity the deployment actually consumes is the predicted count, obtained by integrating the predicted density map (Eq. (1.1)): *Why this form.* Density integration makes the output insensitive to the spatial resolution $H' \times W'$ of the decoder and turns counting into a smooth regression problem. *Consequence.* Every design decision in this chapter ultimately serves the fidelity of $\widehat{D}$; the contrastive objectives never appear at inference — they only shape the representation from which D produces $\widehat{D}$. *Bridge.* To supervise $\widehat{D}$ we need a target density; the Bayesian loss provides one without hand-tuned kernels, and is derived next.

2.2 Stage 1: Weather-Aware Representation Learning via Multi-Queue MoCo

2.2.1 Counting supervision: the Bayesian loss

Before the contrastive machinery, the counting pathway must be pinned down — both because it is half of the stage objective and because its robustness under crowding determines how much of the weather problem remains after representation learning. A remark specific to this thesis: the rendering bandwidths of Eq. (2.1) are computed from *annotations*, not from image appearance, so the target density is weather-blind by construction. Whatever the weather does to the image, the target says the same thing about the same heads. This is a quiet strength of the Bayesian formulation for the adverse-weather setting: the supervision cannot degrade along with the pixels, and all weather sensitivity is concentrated in the features — which is precisely the quantity the contrastive stages of this chapter are designed to fix. the counting pathway must be pinned down — both because it is half of the stage objective and because its robustness under crowding determines how much of the weather problem remains after representation learning.

Each training image carries head annotations $\{(x_j, c_j)\}_{j=1}^{n}$. Following the Bayesian formulation [9], instead of placing a fixed Gaussian kernel on every head (whose bandwidth is ill-defined under heavy occlusion and perspective), each anchor point x_j is rendered from *all* annotated points with a Gaussian whose mass equals the annotated count:

$$d_j(x) = \sum_{i \neq j} c_i \, \mathcal{N}\left(x \,|\, x_i, \sigma_j^2 \boldsymbol{I}\right), \qquad \sigma_j^2 = 1/m \sum_{i \neq j} \left\| x_i - x_j \right\|^2, \qquad (2.1)$$

where σ_j is the empirical average distance to the m nearest neighbouring points. The target density map assembles the rendered masses at the anchor locations:

$$D_\sigma(x) = \sum_{j=1}^{n} d_j(x)\,\delta(x - x_j), \qquad (2.2)$$

with $\delta(\cdot)$ the Dirac delta. The decoder output $\widehat{D} = D(F(X;\theta))$ is then supervised by normalizing the residual with the target mass:

$$\mathcal{L}_{\text{bayesian}} = \sum_{i \in \widehat{D}} \left\| \frac{\widehat{D}(i) - D_\sigma(i)}{\|D_\sigma\|_1} \right\|_2^2 . \qquad (2.3)$$

Symbol walkthrough.

Eq. (2.1) renders each head's count mass c_j as a mixture over its neighbours' Gaussians; Eq. (2.2) collapses the rendered field back onto the anchor positions, producing a sparse but locally calibrated target; the normalization $\|D_\sigma\|_1$ in Eq. (2.3) makes the loss invariant to the absolute count scale of the image, so that sparse and crowded images contribute comparably gradients.

Why this form.

A fixed-kernel density target mis-estimates crowded regions (kernel too large) and sparser regions (kernel too small); the Bayesian rendering adapts the effective bandwidth per head from the data itself. *Consequence.* The counting head receives a target that is trustworthy in exactly the crowded, occluded regions where adverse weather also bites. *Bridge.* This loss constrains *what* the decoder counts, but nothing yet constrains *how* the encoder represents weather — the representation could mix all domains arbitrarily and the density target would still be satisfiable on the

A three-point rendering example.

To see Eq. (2.1) at work, consider three annotated heads with masses $c = (1,1,2)$ at positions forming a tight cluster plus one outlier at distance r. For the outlier, the nearest-neighbour distance is r, so its Gaussian bandwidth is wide ($\sigma \approx r$) and its rendered mass d_j is small — the target says "about one person somewhere in this large region". For the tight cluster, bandwidths are narrow and the rendered masses are close to the true counts, so the target says "two people exactly here". The target density of Eq. (2.2) thus adapts its precision to the local evidence, and the normalization in Eq. (2.3) scores the decoder on how well the *shape* of its density matches the rendered masses.

Why normalize by $\|D_\sigma\|_1$*?* Without the normalization, an image of 100 people would dominate the batch gradient ten times as strongly as an image of 10 people, purely because its residuals are larger in absolute terms; the normalized residual measures per-capita misfit, so both images teach the decoder equally about *relative* placement. This scale invariance also stabilizes the interplay with the contrastive term in Eq. (2.10): the two losses then vary on comparable scales across images, rather than the counting term swamping the contrastive term exactly on crowded (often adverse-weather) scenes. majority domain. The contrastive objective below closes that gap.

2.2.2 Baseline machinery: momentum contrast

The contrastive branch follows the MoCo template [32]. Each image is augmented twice, yielding views I_Q and I_K. The on-line encoder produces $R_Q = E_Q(I_Q;\theta)$ and the momentum encoder produces $R_K = E_K(I_K;\tilde{\theta})$, whose parameters evolve slowly by the convex update

$$\tilde{\theta} \leftarrow \mu\,\tilde{\theta} + (1-\mu)\,\theta, \qquad (2.4)$$

so that keys entering the memory across iterations are encoded by a smoothly drifting function rather than a jumping one. Representations are flattened by the projection heads: P_Q pools $R_Q \in \mathbb{R}^{H\times W\times C_1}$ to a vector and maps it through a two-layer MLP to the anchor vector $Q_i \in \mathbb{R}^{C_2}$; P_K has the same structure (without shared parameters) and maps R_K to keys K.

Symbol walkthrough (Eq. (2.4)).

The update is a convex combination: with μ close to 1 the momentum encoder changes by only a fraction $(1-\mu)$ of the on-line update per iteration, so a key written into the queue at iteration t was produced by parameters that are a low-pass filtered version of the recent training trajectory. *Why this form.* Contrastive negatives must be encoded by a *consistent* function: if negatives were re-encoded by the fast-moving on-line encoder each iteration (as in a memory bank without EMA), the same stored key would silently change meaning across iterations and the objective would chase a moving target. *Consequence.* The queue behaves like a large, temporally smooth batch — the property the memory bank of [31] lacks and which the storage ablation (Table 3.6) isolates experimentally. *Consequence (projections).* The projection decouples the contrastive geometry from the representation that feeds the decoder, so the contrastive loss cannot directly distort the counting features — it only shapes them indirectly through the encoder. *Bridge.* With anchors and keys defined, the vanilla contrastive objective can be stated — and its failure mode under imbalance exposed.

A parenthetical on *where negatives should come from*. In-batch schemes (SimCLR [30]) draw negatives from the current large batch, which couples the negative-pool size to the batch size and demands very large batches for a rich pool. The queue decouples the two: the pool size BL is a design constant, not a memory-budget accident, and — decisively for this thesis — it is a *controllable* distribution. In-batch negatives would re-introduce the data frequencies through the sampler; even a perfectly balanced batch sampler would fluctuate with the shuffle order. The queue is the natural place to enforce a distribution, because it is where samples accumulate over time rather than in one draw.

2.2.3 The vanilla objective and why it fails under imbalance

Instance discrimination treats the keys of *different* images as negatives. With a single memory queue $\mathcal{Q} \in \mathbb{R}^{L\times C_2}$ and anchor Q_i, the vanilla objective is the InfoNCE form

$$\mathcal{L}_{\text{nce}} = \sum_{i\in I} -\log \frac{\exp\ (Q_i \cdot K_{i^+}/\tau)}{\sum_{a\in\mathcal{Q}} \exp\ (Q_i \cdot K_a/\tau)}, \qquad (2.5)$$

where K_{i^+} is a key from the same image as anchor i, $\mathcal{Q}$ is the single queue, and τ is the temperature.

Two design constants do the quiet work in Eq. (2.5). The queue size sets how many negatives the objective sees — in-batch alternatives see only $|I|$, which forces the very large batches of SimCLR-style training. The temperature rescales the similarities before the softmax and thereby sets the *hard-negative emphasis*: small τ concentrates the gradient on the highest-similarity negatives, large τ spreads it evenly. Both constants will reappear in the imbalance analysis, because the class composition of the memory multiplies into exactly these terms.

Why this form breaks under P2.

Assran et al. [39] showed that contrastive objectives carry a hidden prior towards uniformly clustered features. Under class balance this prior is harmless; under weather-class imbalance the mismatch between prior and data becomes harmful in a specific, quantifiable way: the queue is

filled by streaming samples from $p_{\text{data}}(w)$, so the fraction of negatives of weather class c inside the queue approaches the *data frequency* $n_c/\sum_{c'} n_{c'}$. In JHU-Crowd++ the majority (normal) class approaches nine-tenths of all data, so the denominator of Eq. (2.5) is overwhelmingly computed from normal-weather keys: the learning signal that pushes representations *apart* is almost exclusively *within*-class for the majority and leaves the minority classes without discriminative pressure. The representation remains weather-entangled exactly where robustness is needed. The gradient statement makes the failure exact. For anchor Q, $\partial\mathcal{L}/\partial \mathrm{Q} = \tau^{-1}\left(\sum_a p_a \mathrm{K}_a - \mathrm{K}_+\right)$ with $p_a = \mathrm{softmax}_a(\mathrm{Q}\cdot \mathrm{K}_a/\tau)$: the anchor is pushed away from a *softmax-weighted average of the memory* and towards its positive. If normal-weather keys occupy nine-tenths of the queue, the pushed-away average is nine-tenths a normal-weather direction — the encoder learns mainly "not-normal", and the separation between rain, snow and haze keys is never demanded. The composition of the denominator is thus not a detail of implementation; it *is* the curriculum. *Bridge.* The failure is a *memory distribution* problem, not an objective problem. The repair is therefore not a new loss but a new memory: decompose the negative pool by class.

2.2.4 Multi-queue MoCo

The two-view regime.

Each image is augmented exactly twice per iteration, once per branch. This regime deserves a remark because it fixes the meaning of the positives. With two views, every anchor Q_i has its same-view counterpart key K_i written into the queue in the same iteration, and possibly the keys of the same image from a handful of previous iterations (small relative to $L = 1024$, so mostly evicted). The positive set $P(i)$ is therefore small (typically one or two) and tightly related to the anchor, while the negative pool is large and — crucially, by Eq. (2.8) — class-balanced. The asymmetry is deliberate: contrastive learning benefits from few, tight positives and many, diverse negatives; the failure of the vanilla scheme was never the number of positives but the skewed composition of the negatives. Weight sharing between $\mathrm{P_Q}$ and $\mathrm{P_K}$ is likewise avoided (identical architectures, separate parameters): sharing would tie the anchor geometry to the slowly moving key geometry and is known to be unnecessary when the projection heads are small.

Set definitions

Let each queued key carry the weather label of the image it came from. Define

$$P(i) = \{\, p \in \mathcal{Q} : \mathrm{img}(p) = \mathrm{img}(i) \,\}, \qquad \mathcal{Q} = \bigcup_{c=1}^{B} \mathcal{Q}_c, \ \boldsymbol{A} = \{1, \ldots, B\} \times \{1, \ldots, L\}, \qquad (2.6)$$

where $P(i)$ is the positive index set of anchor i (keys from the same image), $\mathcal{Q}_c$ is the sub-queue of weather class c with fixed length L, and $\boldsymbol{A}$ indexes all $B \times L$ queued keys. The multi-queue is a tensor $\mathbb{R}^{B\times L\times C_2}$; each sub-queue stores *only* keys of its own class, so $|\mathcal{Q}_1| = \ldots = |\mathcal{Q}_B| = L$ by construction.

The multi-queue contrastive objective

The objective below is stated on the multi-queue; its relation to Eq. (2.5) is exactly one substitution — the negative pool $\mathcal{Q}$ becomes the balanced union of sub-queues — plus a multi-positive normalization. Everything else (temperature, projection heads, momentum branch) This is deliberate isolation, and it mirrors how the experiments later verify the design: because the objective is unchanged, any performance difference between storage strategies (Tables 3.6 and 3.7) is attributable to the memory structure alone — a single-variable comparison in a field where multi-

variable comparisons are the norm. contribution is the memory structure, and isolating it makes the later ablations (Table 3.6) a clean single-variable comparison.

$$\mathcal{L}_{\text{contra1}} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp\left(\mathrm{Q}_i \cdot \mathrm{K}_p / \tau\right)}{\sum_{a \in \boldsymbol{A}} \exp\left(\mathrm{Q}_i \cdot \mathrm{K}_a / \tau\right)}. \qquad (2.7)$$

Symbol walkthrough.

The outer sum iterates over the anchors of the mini-batch ($|I|$ = batch size). For each anchor, the inner sum averages the log-likelihood ratio over *all* its positives $P(i)$ — under two random augmentations of the same image this set typically contains two or more keys, including keys of the same image pushed into the queue earlier. The denominator sums over $\boldsymbol{A}$, i.e., over *all* queued keys of *all* weather classes. τ rescales the cosine similarities before the softmax; smaller values sharpen the distribution and strengthen gradients on hard negatives.

Why this form.

Three departures from Eq. (2.5) are deliberate. First, positives are *same-image* keys rather than same-weather keys: even when two images share a weather label their conditions differ in severity, so collapsing all same-label pairs would over-average; same-image positives additionally share scene content, which strengthens scene recognition (verified empirically in Section 3.3, Table 3.5). Second, treating same-weather different-image keys as negatives is a deliberate misuse tolerated by the alignment–uniformity analysis of [38]: contrastive learning tolerates semantically similar negatives. Our experiments (Table 3.5) confirm that this strategy outperforms label-based positive/negative partitioning. Third, and central to this thesis, the negative pool is drawn from *equal-length per-class sub-queues*, which restores the uniformity prior: the fraction of class-c negatives in the denominator is exactly $L/(BL) = 1/B$ by construction, independent of n_c.

Consequence.

The encoder now receives discriminative pressure *across* weather domains for every class, including the tail: rainy keys must separate from a balanced pool rather than drown inside it. This is what grants the representation its weather-awareness under P2.

Bridge.

Eq. (2.7) presumes that each sub-queue contains what its label promises. The update rule that guarantees this is formalized next.

Properties of the memory, side by side

Table 2.3 condenses the discussion so far into the three properties a memory must have for this task: *consistency* (keys encoded by a stable function), *capacity* (enough negatives for a rich signal) and *balance* (class composition independent of data skew). The memory bank achieves capacity but neither consistency nor balance; single-queue MoCo achieves consistency and capacity but not balance; the multi-queue achieves all three, which is the structural root of the empirical gaps measured in Section 3.3 (Tables 3.6 and 3.7).

Table 2.3: Properties of contrastive memories. Balance is the property that Eq. (2.9) requires under weather-class imbalance.

Memory	Consistency	Capacity	Balance	Extra state	Source
Memory bank [31]	✗	✓	✗	$M \cdot C_2$	one vec./sample

Memory	Consistency	Capacity	Balance	Extra state	Source
Single queue [32]	✓	✓	✗	$L \cdot C_2$	FIFO
Multi-queue (ours)	✓	✓	✓	$BL \cdot C_2$	per-class FIFO

Queue update rule

Immediately after a key K is computed by the momentum branch, it is pushed into the sub-queue of its weather class $c = w(\mathrm{K})$ with a FIFO shift:

$$\mathcal{Q}_c \leftarrow \left[\mathcal{Q}_c[L - 1], \ldots, \mathcal{Q}_c[1], \mathrm{K}\right], \qquad \mathcal{Q}_c[0]\text{evicted}, \qquad (2.8)$$

i.e., the oldest key is dropped and the freshest key occupies the head position. Eq. (2.8) is the single mechanism through which the memory distribution is controlled: because class assignment uses the ground-truth weather label (available in training only), the invariant $|\mathcal{Q}_1| = \ldots = |\mathcal{Q}_B| = L$ holds at every iteration regardless of the data frequencies n_c. Because adverse-weather images are few, the same image may contribute multiple keys to one sub-queue over time; to prevent such views from being treated as negatives of each other, every image in the dataset is assigned a unique index and all keys descending from the same index are placed in $P(i)$ of the corresponding anchors (Eq. (2.6)).

Why FIFO: staleness analysis of Eq. (2.8)

The FIFO discipline is the simplest rule that keeps every stored key *recent enough*. A key evicted from position 0 of $\mathcal{Q}_c$ was pushed L same-class pushes ago; with per-class arrival rate $|I|_c$ images per iteration, that is $L/|I|_c$ iterations of staleness. In the balanced regime each class receives $|I|/B$ keys per iteration on average, so staleness is bounded by $\approx BL/|I|$ iterations for *every* class simultaneously — on JHU-Crowd++, $4 \cdot 1024/16 = 256$ iterations. Combined with the momentum smoothing of Eq. (2.4), keys older than that window differ from the current encoder by a bounded parameter distance,

Symbol walkthrough (Eq. (2.8)).

The rule has three operands: the sub-queue $\mathcal{Q}_c$ selected by the key's ground-truth weather label, the shifted contents (every position moves one slot towards the tail), and the incoming key at the head. Note what the rule does *not* involve: no similarity test, no threshold, no learning — the push is unconditional, so the memory cannot develop blind spots through a learned gate. Unconditional pushes are also what make the balance invariant of Eq. (2.9) structural rather than statistical: each sub-queue grows by exactly one key per same-class image processed, and shrinks by exactly one at length L. Alternative disciplines change the trade-off: *reservoir* sampling (keep a uniform sample of all past keys) maximizes diversity but lets arbitrarily stale keys survive; *recency-weighted* eviction complicates the push rule for marginal gain. The storage-strategy experiments of Section 3.3 (Table 3.6) evaluate the extreme points of this design space (memory bank vs. queue) and confirm the FIFO-plus-momentum choice. The class-balanced storage strategies compared there (random replacement, class-balanced sampling) are intra-queue variants that leave Eq. (2.8)'s balance property untouched — only the eviction order changes.

Warm-up phase and early-iteration behavior

Eq. (2.8) presumes full sub-queues, which do not exist at iteration zero. The multi-queue is filled by streaming: with batch size $|I| = 16$, the first $L/|I| = 64$ iterations populate each sub-queue from empty to full, and during warm-up the denominator of Eq. (2.7) simply sums over the keys present so far. Two properties make this transient harmless. First, all sub-queues fill *at the same rate per class by construction* (each batch contributes one key to exactly one sub-queue per image in it), so the balance property $|\mathcal{Q}_1| = \ldots = |\mathcal{Q}_B|$ holds at every iteration, empty or full — the invariant is about

equality, not fullness. Second, early keys are encoded by an encoder close to its initialization, whose representations are undifferentiated anyway; by the time the contrastive signal becomes informative, the memory has long stabilized. In practice we observed no sensitivity to warm-up details, which is consistent with the momentum branch smoothing the key distribution (Eq. (2.4)).

A worked example

To make Eqs. (2.7)–(2.8) concrete, consider a miniature setting with $B = 4$ weather classes (normal, haze, rain, snow), sub-queue length $L = 8$, temperature $\tau = 0.05$, and embedding vectors reduced to scalars for readability. Let the anchor of a *rainy* image be $\mathrm{Q}_i = 0.4$, and let its sub-queues after several iterations hold the (stylized) keys

$\mathcal{Q}_{\text{nor}}$:	-0.5	-0.4	-0.6	-0.3	-0.5	-0.2	-0.4	-0.5
$\mathcal{Q}_{\text{haze}}$:	0.1	0.0	0.2	0.1	-0.1	0.2	0.0	0.1
$\mathcal{Q}_{\text{rain}}$:	0.5	0.4	0.6	0.3	0.5	$\mathbf{0.4}$	0.5	0.6
$\mathcal{Q}_{\text{snow}}$:	-0.2	-0.1	0.0	-0.2	0.1	-0.1	0.0	-0.2

where the bold key is from the *same image* as the anchor (same-image keys survive FIFO eviction only briefly, since $L = 1024$ in practice and the same-image key is pushed once per iteration). The positive set is $P(i) = \{\text{position of } \mathbf{0.4} \text{ in } \mathcal{Q}_{\text{rain}}\}$, $|P(i)| = 1$. The numerator of Eq. (2.7) is therefore $\exp(0.4 \times 0.4/0.05) = \exp(3.2) \approx 24.5$. For the denominator, the anchor contributes a similarity of 0.16 against its own class ($\exp(3.2) \approx 24.5$ per key, 8 keys), but only $\exp(0.4 \times (-0.5)/0.05) = \exp(-4) \approx 0.018$ per normal key, $\exp(0.08/0.05) = \exp(1.6) \approx 5.0$ per haze key and $\exp(0.4 \times (-0.15)/0.05) = \exp(-1.2) \approx 0.30$ per snow key. The pooled negative evidence is $8 \times (24.5 \cdot 7/8 + 0.018 \cdot 8/8 \cdot 0 + 5.0 + 0.30) \approx 8 \times (21.4 + 5.3) \approx 214$, dominated on the positive side by same-class keys and on the negative side by the haze cluster — exactly the discriminative pressure Eq. (2.9) predicts: the anchor is pulled towards its own class and pushed from the *nearest* other class, while the majority class contributes almost nothing to the denominator because its keys are already far away. Under a *single* queue filled by data frequencies, eight of the roughly 24 stored keys would be normal-weather keys whose contribution remains negligible — i.e., most of the memory would be spent storing near-zero evidence, the formalization of "wasted memory" in Section 2.2.4. The example can be pushed one step further to quantify the imbalance effect. Suppose the memory had instead been filled at JHU-like frequencies $\pi = (0.88, 0.05, 0.04, 0.03)$ with the same per-key evidence: of the ≈ 24 stored keys, about 21 would be normal (-0.4-ish, evidence ≈ 0.018 each), roughly one each of haze (evidence ≈ 5) and snow (≈ 0.3), and one rain key. The pooled negative evidence would then be $\approx 21 \times 0.018 + 5 + 0.3 \approx 5.4$ instead of ≈ 213: the denominator collapses onto the single nearest-class cluster, and the softmax mass available to punish confusions between *adverse* classes (rain vs. snow, which the encoder must ultimately distinguish for the refiner to have clean targets) shrinks by two orders of magnitude. The multi-queue restores those factors structurally, before any training dynamics can trade them away.

Analysis: why balancing the memory fixes the gradient signal

The quantitative core of the multi-queue argument is the class composition of the denominator of Eq. (2.7). Let $S_c(\mathrm{Q}_i) = \frac{1}{L}\sum_{a \in \mathcal{Q}_c} \exp(\mathrm{Q}_i \cdot \mathrm{K}_a/\tau)$ denote the mean negative evidence contributed by class c. The expected denominator evidence is

$$\mathbb{E}\left[\sum_{a} \exp\left(\mathrm{Q}_i \cdot \mathrm{K}_a/\tau\right)\right] = L\sum_{c=1}^{B} \pi_c \, S_c(\mathrm{Q}_i), \qquad \pi_c$$

$$= \begin{cases} n_c/\sum_{c'} n_{c'} & \text{single queue (streamed fill)} \\ 1/B & \text{multi-queue (Eq. (2.8))} \end{cases} \qquad (2.9)$$

where n_c is the number of training images of class c.

Reading Eq. (2.9).

With a single queue, π_c *is* the data frequency: for JHU-Crowd++-like skews the majority class receives $\pi_{\text{nor}} \approx 0.88$ and each adverse class $\pi_c \leq 0.05$. The pooled negative evidence is therefore a majority-class quantity: gradients through the denominator teach the encoder "what normal weather is *not*". Tail classes contribute almost no negative evidence, so their representations receive little separation pressure and remain entangled with the majority. With the multi-queue, $\pi_c = 1/B$ *irrespective of* n_c: the denominator is a balanced mixture over weather domains, every class exerts an equal share of repulsive force, and the encoder is forced to make all domains mutually separable. This is the analytical counterpart of the uniform-cluster prior of [39]: we do not change the prior, we change the memory so that the data seen through the memory *matches* the prior.

The same decomposition clarifies the two forces inside Eq. (2.7). The *pull* term (numerator) is class-local: it acts on $P(i)$, keys of the anchor's own image, so its strength is independent of n_c. The *push* term (denominator) is where imbalance acts, and its per-class mass is governed by π_c. A useful way to summarize Eqs. (2.6)–(2.9) is: *the vanilla objective lets the data decide which classes the encoder must separate; the multi-queue decides once, uniformly, at memory level.* The decision is one-time, training-only, and costs a reshaped tensor rather than a new loss.

Computational note.

Eq. (2.7) requires, per anchor, similarities against all $B \cdot L$ keys — a matrix product of shape $|I| \times C_2$ by $C_2 \times BL$, negligible next to the encoder convolution ($C_2 = 128$, $BL = 4096$ on JHU-Crowd++). The log-sum-exp in the denominator should be evaluated in its stabilized form to avoid overflow at small τ; the gradient of the stabilized form is the familiar softmax-weighted average of keys, which is what standard implementations differentiate.

Design alternatives and why they were rejected

The multi-queue was designed against four plausible alternatives, each of which fails a specific requirement:

- **Class-conditional input features** (à la JHU-Crowd++ conditioning blocks [1]): inject the weather label as a side input to the decoder. Rejected because the label is unavailable at inference; using a predicted label couples the system to a classifier whose errors propagate (cf. the MultiModel rows of Table 3.2).

- **Restore-then-count** [24]: restore first. Rejected because restoration optimizes perceptual fidelity, not countability; the residual domain gap (raincoats, umbrellas, wet reflections) persists, and the restoration network is a heavy inference-time component (cf. the Restore rows of Table 3.2).

- **Re-weighted or re-sampled counting loss**: balance the counting gradient per class. Rejected because it balances *gradient magnitude* but imposes no *geometric* structure across domains; the encoder remains free to represent all weather entangled as long as the weighted average loss decreases.
- **Class-balanced loss modifications** (KCL [40], BCL [41], TSC [42]): keep the single queue, modify the loss. These are viable — and are compared empirically in Section 3.3 — but they operate *after* the imbalance has already contaminated the memory; the multi-queue removes the imbalance at the source. They also require the class label inside the loss at every step, whereas the multi-queue needs it only at push time.

Bridge. The contrastive term shapes the representation; the counting loss must still be served. The stage objective combines both.

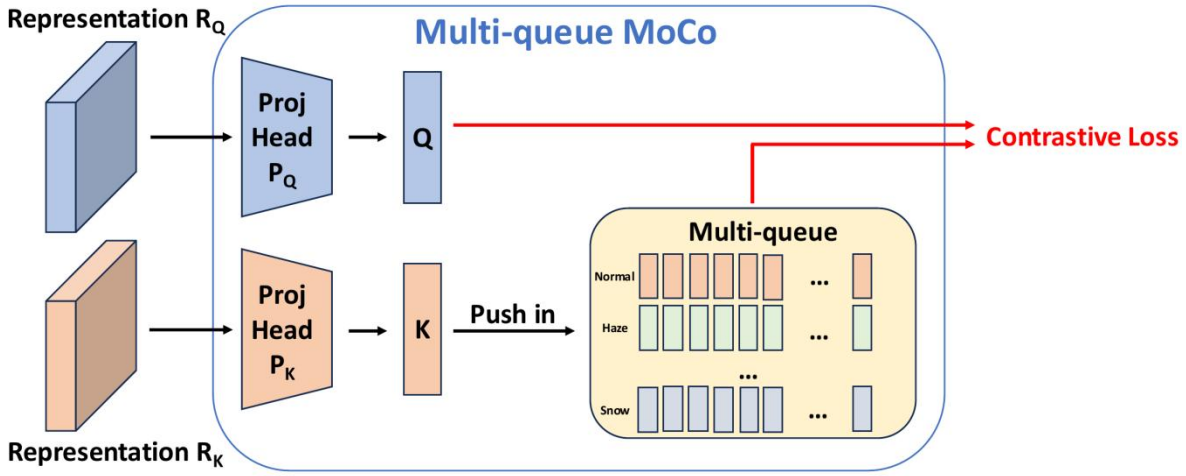


Figure 2.2: Multi-queue MoCo. Projection heads map representations to 1-D vectors; the memory consists of B equal-length sub-queues, each exclusively storing keys of one weather class, so the class composition of the contrastive denominator is uniform by construction.

Why three blocks?

The depth of g_ϕ is a deliberate middle point. Too shallow a refiner (a single block) cannot express the cross-domain appearance edit — weather artifacts are entangled with object texture across the whole receptive field. Too deep a refiner risks re-learning the encoder: with the counting loss flowing through it, a powerful g_ϕ can reproduce a private encoder, bypassing the weather-aware geometry and re-introducing domain entanglement below the contrastive signal. Three ConvNeXt blocks at matched dimensionality provide enough composition for the appearance edit while keeping the module (and its FLOPs, Eq. (2.15)) an order of magnitude below the encoder, preserving the plug-and-play economics of objective O3.

What if the queues were not frozen?

Freezing is a design choice worth testing in thought before in code. If the queues continued to be pushed by the (now frozen) encoder, their content would still drift in composition as old keys expire — harmless, since new keys equal old keys in distribution. If instead the encoder were also fine-tuned while the refiner trains, two couplings appear at once: the keys age against a moving encoder (breaking the consistency of Eq. (2.4)), and the conversion target $\mathcal{Q}_{\text{nor}}$ moves with the encoder — the objective of Eq. (2.12) then chases a distribution that the network itself controls, which is precisely the unstable-refiner pathology that the two-stage split was designed to avoid. Freezing is therefore not an engineering shortcut but the condition under which Eq. (2.12) has a stationary optimum. A related consistency requirement concerns the density map: because $\mathcal{L}_{\text{bayesian}}$ in

Eq. (2.13) is computed from the decoded *refined* representation, the decoder is re-calibrated to the converted domain during CRR. If the conversion drifted across iterations, the decoder would chase a moving input distribution; the frozen encoder removes this second source of non-stationarity, leaving a single, well-posed learning problem for g_ϕ and D.

One degenerate solution deserves explicit exclusion: *could the refiner simply learn the identity?* If $g_\phi \approx$ id, then Q′ equals the unrefined projection, and Eq. (2.12) reduces to pulling adverse-weather anchors directly towards the normal queue — through the encoder's own features, which are frozen and therefore cannot respond. The contrastive term's gradient would then flow into g_ϕ asking it to move representations it cannot move; the identity is a stationary point only of a *lossless* objective, and Eq. (2.12) is minimized strictly by leaving the adverse cluster (in the denominator) and approaching the normal cluster (in the numerator). The identity shortcut is thus gradient-hostile, not merely discouraged — the objective actively moves ϕ away from it, which is why the refiner trained in practice is far from identity, as the qualitative differences between refined and unrefined density maps (Figure 3.3) attest.

2.2.5 Stage objective

The WRL stage optimizes the sum of the contrastive objective and the counting objective:

$$\mathcal{L}_{\text{wrl}} = \mathcal{L}_{\text{contra1}} + \lambda_1 \mathcal{L}_{\text{bayesian}}. \qquad (2.10)$$

Symbol walkthrough.

$\mathcal{L}_{\text{contra1}}$ (Eq. (2.7)) acts on the projected anchors and keys; $\mathcal{L}_{\text{bayesian}}$ (Eq. (2.3)) acts on the density map decoded from R_Q ; λ_1 balances the two scales. *Why this form.* The two terms are complementary by construction: the contrastive term imposes *cross-domain separability* on the representation, while the Bayesian term guarantees that crowd information survives in it — without the latter, weather-aware features need not retain person-count evidence. *Consequence.* After WRL training the encoder is "mature": it extracts representations that are simultaneously weather-discriminative and crowd-informative, and the queues hold a class-balanced, consistently encoded memory of the domains. *Bridge.* Maturity makes conversion well-posed: the next stage defines a target domain and trains a module that moves adverse-weather representations into it — using the frozen queues as the target's empirical distribution.

2.3 Stage 2: Contrastive-Learning-Guided Representation Refining

2.3.1 The stage interface

The CRR stage begins from a precisely defined handover. From WRL it inherits three artifacts: the trained encoder parameters θ (frozen henceforth), the trained projection head P_Q (frozen, and moved behind the refiner), and the multi-queue $\mathcal{Q}$ with its $B \times L$ keys and labels (frozen). It introduces one new module, g_ϕ , and one new objective, Eq. (2.12). Everything else — the momentum branch E_K , the head P_K , the push rule Eq. (2.8) — is dismantled: with the memory frozen there is nothing left to push. This narrow interface is what makes the stage cheap and stable, and it is also the conceptual heart of the method: *the contrastive memory of stage one becomes the supervision signal of stage two*.

2.3.2 Refiner mapping

The refiner is a light module

$$R_i^{'} = g_\phi \quad (R_i), \qquad g_\phi : \mathbb{R}^{H \times W \times C_1} \to \mathbb{R}^{H \times W \times C_1}, \qquad (2.11)$$

instantiated as three ConvNeXt blocks with input and output dimension $C_1 = 768$, initialized randomly. *Why this form.* The refiner must edit domain-specific appearance information while leaving crowd layout intact; a residual, locally-mixing convolutional block design at matched dimensionality provides exactly this bias, and its parameter budget ($\approx$3M) is small against the encoder ($\approx$29M). The refined representation R' has the same shape as R, so the decoder D is unchanged.

2.3.3 Domain conversion by contrastive guidance

The multiple queues retained from the WRL stage now serve a second life: they are *frozen* and act as the empirical distribution of each weather domain in embedding space. All keys in the normal-weather sub-queue $\mathcal{Q}_{\text{nor}}$ are declared positives; all remaining keys are negatives. Writing $\boldsymbol{N} = \{a \in \boldsymbol{A}: \text{class}(a) = \text{nor}\}$ for the normal-weather index set, the refining objective is

$$\mathcal{L}_{\text{contra2}} = \sum_{i \in I} \frac{-1}{|\boldsymbol{N}|} \sum_{p \in \boldsymbol{N}} \log \frac{\exp\left(\mathrm{Q}_i' \cdot \mathrm{K}_p / \tau\right)}{\sum_{a \in \boldsymbol{A}} \exp\left(\mathrm{Q}_i' \cdot \mathrm{K}_a / \tau\right)}, \qquad (2.12)$$

where Q_i' is the projection of the *refined* representation, produced by the frozen projection head $\mathrm{P_Q}$ re-positioned behind g_ϕ. *Symbol walkthrough.* Structurally Eq. (2.12) mirrors Eq. (2.7) with one decisive difference: the positive set is no longer $P(i)$ (same image) but the *entire* normal-weather sub-queue $\boldsymbol{N}$, normalized by $|\boldsymbol{N}|$. *Why this form.* The task here is not instance discrimination but *domain conversion*: every adverse-weather anchor should move towards the normal-weather domain as a whole, and the frozen queue is precisely the stationary empirical summary of that domain. Freezing both queues and $\mathrm{E_Q}$ is what makes this well-posed — the target distribution does not move while the refiner learns. *Consequence.* Minimizing Eq. (2.12) pulls the refined embedding of a rainy image towards the normal-weather cluster while pushing it away from rain/snow/haze clusters; in representation space, "B becomes A" in the words of the motivating example of Section 1.1: a raincoated pedestrian is represented as the equivalent clear-weather pedestrian. The decoder then sees one consistent domain and its density estimates cease to depend on weather.

Table 2.4 juxtaposes the two contrastive objectives, since their parallel structure invites conflation.

Table 2.4: The two contrastive objectives side by side.

	$\mathcal{L}_{\text{contra1}}$ (WRL)	$\mathcal{L}_{\text{contra2}}$ (CRR)
Anchor	Q from $\mathrm{R_Q}$	Q' from refined R'
Positives	same-image keys $P(i)$	*all* normal-weather keys $\boldsymbol{N}$
Negatives	all queued keys $\boldsymbol{A}$	all queued keys $\boldsymbol{A}$
Memory	written each iteration	frozen
Geometry taught	separate all weather domains	align adverse to normal
Parameters updated	θ, $\mathrm{P_Q}$, D	ϕ, D

The table makes the division of labor explicit: stage one teaches the *topology* of weather (clusters exist and are distinct), stage two teaches a *direction* in that topology (towards normal). Either objective alone is incomplete — topology without direction leaves the decoder with multiple domains; direction without topology has nothing to align.

Positive multiplicity.

The normalization by $|P(i)|$ in Eq. (2.7) also merits a remark. The number of same-image keys available to an anchor varies (an anchor early in training may find several of its image's previous keys still in the queue; one late in training may find none besides the current one). Normalizing by $|P(i)|$ makes the pull term a mean over available positives, so its scale is constant across images and iterations; without it, images that happened to have more surviving same-image keys would exert disproportionately strong attraction. The same logic recurs in Eq. (2.12) with $|\boldsymbol{N}|$ — there the positive count is constant by construction (L), which is one reason the CRR objective has even lower variance than the WRL objective. The CRR stage also inherits a variance property worth noting: because the positive set $\boldsymbol{N}$ is the entire normal-weather sub-queue, the pull term of Eq. (2.12) averages over L keys rather than over the one-or-two keys of $P(i)$, so its stochastic fluctuation is smaller by a factor of roughly $\sqrt{L/|P(i)|}$. Lower-variance gradients suit a stage whose job is precise geometric alignment; the WRL stage, whose job is coarse topology, tolerates (and benefits from) the noisier instance-level pull.

It is also worth observing that imbalance cannot re-enter through the positives in the CRR stage: the positive set $\boldsymbol{N}$ is a *single* class (normal weather), so there is no class composition to skew; and the negative pool is the frozen, already-balanced memory. Both channels of Eq. (2.12) are thus immune to n_c by construction, in contrast to the WRL stage where the design had to *create* that immunity. The CRR stage inherits it — one of the quieter dividends of building the second stage on top of a balanced memory rather than on the dataset directly.

2.3.4 Stage objective

Counting must survive the conversion, so the Bayesian term is retained:

$$\mathcal{L}_{\text{CRR}} = \mathcal{L}_{\text{contra2}} + \lambda_2 \mathcal{L}_{\text{bayesian}}, \qquad (2.13)$$

where the density map in $\mathcal{L}_{\text{bayesian}}$ is now decoded from the refined representation R'. *Why this form.* The two terms negotiate: $\mathcal{L}_{\text{contra2}}$ moves representations across domains, $\mathcal{L}_{\text{bayesian}}$ pins the count evidence in place; λ_2 (set to 10, as λ_1) empirically resolves the trade-off. The gradient routing is deliberately asymmetric: θ (encoder) is frozen, so both terms update only ϕ (refiner) and the decoder — the count head learns to read the converted domain, which is exactly the regime it will face at inference. *Consequence.* After CRR, inference (Eq. (1.1)) runs as $\widehat{D} = D(g_\phi(F(X;\theta)))$ for *any* input weather, with no weather label, classifier or restoration module involved.

Gradient routing.

It is instructive to track *which* parameters each equation updates, since the two stages differ sharply in this respect. In the WRL stage, $\mathcal{L}_{\text{contra1}}$ (Eq. (2.7)) back-propagates through P_Q into θ only — keys K are produced by the momentum branch and enter the loss as constants (standard MoCo practice, which Eq. (2.4) keeps consistent) — while $\mathcal{L}_{\text{bayesian}}$ updates θ, D and P_Q. In the CRR stage the encoder is frozen, so *both* terms of Eq. (2.13) update only ϕ and D; the queues and P_Q are constants. This routing has a practical consequence: the second stage cannot drift the representation out from under the frozen memory, which is what makes the fixed anchors of Eq. (2.12) a legitimate target rather than a moving one. It also bounds the second stage's cost: no momentum branch is executed at all, so each CRR iteration is cheaper than a WRL iteration despite optimizing two objectives.

Bridge. With both stages derived, the complete training procedure can be stated as an algorithm and its cost accounted for.

2.4 Algorithm Summary, Plug-and-Play Design and Complexity

2.4.1 Training procedures

Algorithms 1 and 2 summarize the two stages; every line is annotated with the equation it instantiates. Inference after training is a single forward pass through encoder, refiner and decoder (Eq. (1.1)).

Algorithm 1: WRL stage: weather-aware representation learning (one iteration)
1. batch $\{(X_i, Y_i, w_i)\}_{i\in I}$; encoders $\mathrm{E_Q}(\theta)$, $\mathrm{E_K}(\tilde{\theta})$; heads $\mathrm{P_Q}, \mathrm{P_K}$; decoder D; queues $\{\mathcal{Q}_c\}$; μ, τ, λ_1
2. draw two augmentations $\mathrm{I_Q}, \mathrm{I_K}$ of each X_i
3. $\mathrm{R_{Q_i}} \leftarrow \mathrm{E_Q}(\mathrm{I_{Q,}}_i; \theta)$;
4. $\mathrm{Q}_i \leftarrow \mathrm{P_Q}(\mathrm{R_{Q_i}})$;
5. push each K_i into $\mathcal{Q}_{w_i}$ by Eq. (2.8)
6. form $P(i)$ from matching image indices; assemble $\boldsymbol{A}$ (Eq. (2.6))
7. $\widehat{D}_i \leftarrow D(\mathrm{R_{Q_i}})$; compute $\mathcal{L}_{\text{bayesian}}$ by Eqs. (2.1)–(2.3)
8. $\mathcal{L}_{\text{wrl}} \leftarrow \mathcal{L}_{\text{contra1}} + \lambda_1 \mathcal{L}_{\text{bayesian}}$ (Eqs. (2.7), (2.10))
9. update θ, heads, D by backprop; update $\tilde{\theta} \leftarrow \mu\tilde{\theta} + (1-\mu)\theta$ (Eq. (2.4))

Algorithm 2: CRR stage: contrastive-guided representation refining (one iteration)
1. batch $\{(X_i, Y_i, w_i)\}_{i\in I}$; **frozen** $\mathrm{E_Q}(\theta)$, $\mathrm{P_Q}$, queues $\{\mathcal{Q}_c\}$; refiner $g_\phi(\phi)$; decoder D; τ, λ_2
2. draw one augmentation of each X_i ; $\mathrm{R_{Q_i}} \leftarrow \mathrm{E_Q}(\cdot\,; \theta)$ *[]*
3. $R_i^{'} \leftarrow g_\phi(\mathrm{R_{Q_i}})$ (Eq. (2.11)); $\mathrm{Q}_i^{'} \leftarrow \mathrm{P_Q}(R_i^{'})$
4. $\widehat{D}_i \leftarrow D(R_i^{'})$; compute $\mathcal{L}_{\text{bayesian}}$ as in Eqs. (2.1)–(2.3)
5. $\mathcal{L}_{\text{CRR}} \leftarrow \mathcal{L}_{\text{contra2}} + \lambda_2 \mathcal{L}_{\text{bayesian}}$ (Eqs. (2.12), (2.13))
6. update ϕ and D by backprop; queues and encoder untouched

2.4.2 Plug-and-play properties

The WRL and CRR objectives never inspect the internal topology of the encoder: they act on its outputs $\mathrm{R_Q}$ and the decoded density. Any backbone producing a spatial representation can host MQCL — the thesis evaluates CNN encoders (VGG-16 [47], CSRNet [7]) and hierarchical/Transformer encoders (Swin-T [48], ConvNeXt-T [49]) under identical hyperparameters. The only architectural requirement is spatial fidelity of the representation, which the MFA-style counting head (two transposed-convolution layers) restores anyway.

2.4.3 Cost model

Memory.

The multi-queue stores $B \cdot L$ vectors of dimension C_2; in single precision its footprint is

$$M_Q = B \cdot L \cdot C_2 \cdot s \text{ bytes}, \qquad (2.14)$$

with $s = 4$. For the JHU-Crowd++ setting ($B = 4$, $L = 1024$, $C_2 = 128$) this is *2* MB — three orders of magnitude below the encoder's parameters, and unlike a memory bank [31] it does not grow with dataset size M.

Compute.

Let the per-iteration cost of the encoder, queues and refiner be T_{enc}, T_Q and T_{ref}. In the WRL stage the contrastive branch adds only vector arithmetic (pooling, dot products over $B \cdot L$ keys, the FIFO push), i.e., T_Q is negligible against a convolutional encoder pass. In the CRR stage the momentum branch is discarded entirely and the deployed pipeline gains only g_ϕ:

$$T_{\text{deploy}} = T_{\text{enc}} + T_{\text{ref}} + T_{\text{dec}}, \qquad T_Q = T_{\text{ref}} = 0 \text{in the baseline}, \qquad (2.15)$$

so the inference overhead of MQCL over its baseline is exactly the refiner. Empirically (Section 3.3, Table 3.9) this amounts to *+4.2* GFLOPs and *+3* M parameters on a ConvNeXt-T backbone (27.0→31.2 GFLOPs, 29M→32M at 384×384 input) — comparable in parameters to the strongest SOTA methods at roughly half their FLOPs. *Consequence.* The lightweight objective O3 is met structurally, not merely empirically: the only new inference-time component is g_ϕ, and the only new training-time state is a fixed-size, class-balanced memory.

An order-of-magnitude check confirms the measured overhead. A ConvNeXt block of width $d = 768$ applied to an $h \times w$ feature map costs roughly $\mathcal{O}(4 \cdot d^2 \cdot h \cdot w)$ per pointwise stage plus depthwise $\mathcal{O}(9\, d\, h\, w)$; with three blocks on a feature grid of a few thousand positions, the refiner lands at a few GFLOPs — consistent with the measured *+4.2* GFLOPs of Table 3.9. Because the cost scales with $h \cdot w$ of the *representation* grid (already downsampled) and is independent of the number of weather classes B (the refiner never sees the queue), the overhead conclusion holds for any dataset in the family.

2.4.4 Roles of the objective hyperparameters

Four constants govern Eqs. (2.7)–(2.13), and their roles are best understood through the equations they appear in.

- **Temperature** τ (Eqs. (2.7), (2.12)) rescales cosine similarities before the softmax: the gradient magnitude of a negative key scales as $\exp(\mathrm{Q} \cdot \mathrm{K}/\tau)$, so small τ concentrates learning on the *hardest* negatives. $\tau = 0.05$ proved a workable middle ground; too small a value starves easy negatives of gradient and over-weights outliers, too large a value blurs class boundaries.

- **Momentum** μ (Eq. (2.4)) controls key consistency: keys in the queue were encoded over a window of roughly $1/(1-\mu)$ iterations. Large μ keeps the memory self-consistent but slow to track the improving encoder; small μ tracks quickly but breaks the consistency that makes queued keys valid negatives. The MoCo-recommended regime transfers unchanged.

- **Sub-queue length** L (Eqs. (2.6), (2.8)) sets the resolution of the empirical domain distributions in Eq. (2.9): the effective class proportion is exactly $1/B$ for any L, but the *variance* of the mixture decreases with L. $L = 1024$ keeps Eq. (2.14) at megabyte scale while sampling each domain densely.

- **Loss weights** λ_1, λ_2 (Eqs. (2.10), (2.13)) mediate the scale gap between the contrastive loss (log-likelihood over keys) and the Bayesian loss (normalized squared residuals). Both were set to 10: small enough that the contrastive geometry is not drowned, large enough that counting evidence is never sacrificed — the failure mode of a purely contrastive encoder.

Notably, none of these constants required per-backbone or per-dataset retuning in the experiments of Chapter 3, which is part of the evidence for R4.

2.4.5 Label usage and deployment semantics

A subtle but important property of the design is the asymmetric use of weather labels. Eq. (2.8) consumes the ground-truth label w_i at *push time*, i.e., only during training, where $\mathcal{D}$ provides it. At inference no label, no classifier and no queue lookup is involved: the pipeline is the fixed composition $D \circ g_\phi \circ F$. The labels therefore act as a *training-time supervision signal*, on the same footing as the head-position annotations, rather than as a deployment-time dependency. This is the crucial difference from weather-conditioned architectures [1], [4], which must know (or guess) the weather of every test image. It also clarifies what happens when test weather is *unknown* or unlisted: the representation produced by the balanced contrastive geometry is a point in a space whose class structure was fixed during training, and the refiner maps it towards the normal domain regardless of which named class it resembles most — an out-of-the-box robustness that a classifier-based pipeline cannot offer. Concretely, consider a test image of *sleet* — a condition absent from training. A conditioning pipeline must first decide among its trained classes and will commit to one, propagating any error. The MQCL pipeline instead encodes the image into the representation space; whatever region of that space the embedding lands in, the refiner maps it towards the normal domain, and the decoder counts. The worst case degrades gracefully towards the behavior between adjacent domains rather than towards a random class decision, because the mapping applied is a continuous function of the representation, not an argmax.

Larger B: scale behavior of the memory.

The queue memory of Eq. (2.14) grows linearly in the number of classes B, and the denominator of the contrastive objective grows as $B \cdot L$. Both are mild: the GCC experiments run $B = 7$ with no retuning, and datasets with dozens of fine-grained domains would remain within megabyte-scale memory and millisecond-scale dot-product budgets. Should B grow to hundreds, the sub-queue length L — not the architecture — is the knob to shrink, since the balance property of Eq. (2.9) holds for any L, and the variance of the per-class evidence grows gracefully as $1/L$.

2.4.6 Requirements check

Table 2.5 closes the chapter by mapping each requirement of Section 1.3 to the design elements of this chapter that satisfy it; the experimental verification is deferred to Chapter 3. Before closing, each row of the check table is unpacked in one sentence, since the table compresses arguments made across the chapter. **R1**: Eqs. (2.7) and (2.12) are the only equations whose minimands contain weather-dependent structure, and they act directly on the representation the decoder consumes. **R2**: Eqs. (2.6), (2.8) make the memory composition a design constant ($1/B$ per class), and Eq. (2.9) shows why that constant — not the data frequencies — governs the learning signal; skew therefore cannot degrade what it no longer controls. **R3**: the Bayesian term appears in both stage objectives (Eqs. (2.10), (2.13)) with a substantial weight, so counting evidence is conserved across the whole pipeline. **R4**: no equation inspects the encoder's internals; both losses act on outputs, so any spatial backbone qualifies. **R5**: the deployed stack gains exactly g_ϕ (Eq. (2.11)), and the queue never leaves the training loop (Eq. (2.14)).

Table 2.5: Requirements versus design elements.

ID	Satisfied by	Mechanism
R1	Eqs. (2.7), (2.12)	weather-aware representation + domain conversion
R2	Eqs. (2.6), (2.8), (2.9)	class-balanced memory, $\pi_c = 1/B$ independent of n_c
R3	Eqs. (2.3), (2.13)	counting evidence preserved in both stages

ID	Satisfied by	Mechanism
R4	Eqs. (2.7), (2.10)	losses act on outputs only, backbone-agnostic
R5	Eqs. (2.11), (2.14), (2.15)	3M-param refiner; 2 MB fixed-size queue

Table 2.6: The equation chain of Chapter 2 and the obstacle each part addresses.

Equations	Role	Addresses
(1.1)	task output	—
(2.1)–(2.3)	counting supervision	crowding/occlusion
(2.4)–(2.5)	vanilla MoCo baseline	(fails under P2)
(2.6)–(2.8)	class-balanced memory	**P2: imbalance**
(2.9)	why balance restores the prior	P2 (analysis)
(2.7), (2.10)	weather-aware representations	**P1: domain gap**
(2.11)–(2.12)	domain conversion	P1 (representation level)
(2.13)	conversion with count preservation	P1 + R3
(2.14), (2.15)	cost accounting	O3 / R5

2.5 Chapter Summary

This chapter derived MQCL as a chain of fifteen equations. The counting task was formalized by density integration (Eq. (1.1)) and supervised by the Bayesian loss (Eqs. (2.1)–(2.3)), whose annotation-adaptive target is robust under the crowding and occlusion that adverse weather amplifies. The vanilla momentum-contrast machinery (Eqs. (2.4)–(2.5)) was then shown to carry a failure mode under weather-class imbalance: the memory composition follows the skewed data frequencies, so the discriminative signal is dominated by the majority domain. The repair was structural rather than objective-level: decomposing the memory into equal-length per-class sub-queues (Eqs. (2.6) and (2.8)) pins the class composition of the contrastive denominator at $1/B$ (Eq. (2.9)), restoring agreement between the memory and the uniform prior of the objective. The resulting WRL objective (Eq. (2.10)) yields weather-aware, crowd-informative representations and a class-balanced memory. In the second stage, the frozen memory was re-purposed as the target distribution: a lightweight refiner (Eq. (2.11)) was trained with a domain-conversion contrastive objective (Eq. (2.12)) plus the counting loss (Eq. (2.13)), so that the deployed pipeline $D \circ g_\phi \circ F$ sees a single, consistent domain for any input weather. The cost of the scheme is bounded by construction: a fixed-size queue (Eq. (2.14)) during training and one small refiner (Eq. (2.15)) during deployment. The next chapter subjects each of these claims to experiment.

Chapter 3: Implementation, Testing and Analysis

This chapter turns the method of Chapter 2 into a working system and subjects it to a complete empirical study. Section 3.1 documents the implementation; Section 3.2 defines the datasets, metrics and protocol; Section 3.3 presents the results — comparisons with the state of the art, an ablation of every design decision, imbalance stress tests, representation visualizations, and computational cost; Section 3.4 analyzes the findings, states the limitations and concludes.

3.1 Implementation Details

Framework.

MQCL is implemented in PyTorch. All experiments use a single GPU; the training procedure follows Algorithms 1 and 2 sequentially, and the CRR-stage model is the one evaluated.

Encoders.

Four backbones of different architecture families are used to demonstrate the plug-and-play property: VGG-16 [47] (classic CNN), CSRNet [7] (dilated CNN), Swin-T [48] (hierarchical Transformer) and ConvNeXt-T [49] (modernized CNN). Except for the removal of the classification head, each backbone is identical to its original classification architecture, so that the comparison "baseline vs. MQCL" isolates the contribution of the training scheme rather than of architectural tuning.

Counting head.

The crowd counting head D follows MFA [50]: an upsampling module composed of two transposed convolutional layers that maps the encoder representation to the density map. The head is shared across all experiments so that encoder choices remain the only architectural variable.

Projection heads and refiner.

Each projection head pools its input representation to a vector and passes it through a two-layer MLP with output dimensions 2048 and $C_2 = 128$. The refiner g_ϕ consists of three ConvNeXt blocks with input and output dimension 768, initialized with random parameters — it is the *only* newly designed module and the only component that survives into deployment beyond the baseline stack.

Augmentation and memory.

Each image in a batch undergoes augmentation twice to form the two contrastive views: a random crop of 256×256 pixels and horizontal flipping with probability 0.5. The multi-queue holds one sub-queue per weather class, each of length $L = 1024$; the number of sub-queues B therefore equals the number of weather classes of the dataset (4 for JHU-Crowd++, 7 for GCC). Every dataset image receives a unique index at preprocessing time, and keys derived from the same index are matched into $P(i)$ (Eq. (2.6)) via an index-to-queue lookup implemented as a contiguous $\mathbb{R}^{B \times L \times C_2}$ tensor with an integer label buffer of the same shape modulo C_2.

Optimization.

Both stages use the AdamW optimizer [51] with cosine-annealed learning rate (initial 10^{-4}), weight decay 10^{-3} and batch size 16. The loss weights are $\lambda_1 = \lambda_2 = 10$ and the temperature is $\tau = 0.05$.

These settings are kept *identical across all backbones and both datasets*, underlining that MQCL is a training scheme rather than a tuned architecture.

Environment.

All experiments were executed on a single NVIDIA GPU with the CUDA build of PyTorch; no multi-GPU or mixed-precision tricks were used, and none are required — the method's memory footprint is dominated by the backbone, not by the queue (Eq. (2.14)). Training the WRL stage takes on the order of a day on this hardware and the CRR stage a fraction of that, in line with the cost model of Section 2.4.

Training schedule.

Both stages are trained to convergence under the cosine schedule rather than to a fixed epoch budget, and the CRR stage is always initialized from the WRL checkpoint — the two-stage structure of Section 2.1 is enforced in code, not merely described. Two schedule-level observations are worth recording for practitioners. First, the WRL stage benefits from longer training than a purely supervised baseline of the same backbone, because the contrastive geometry matures more slowly than the counting head; the queues keep providing fresh, consistent negatives throughout, so there is no late-epoch penalty analogous to over-fitting the memory. Second, the CRR stage is comparatively short and stable: since the encoder is frozen, the objective of Eq. (2.13) is close to convex in the refiner's parameters, and validation accuracy plateaus early — we observed no benefit from aggressive schedules or restarts at this stage. Convergence behavior also differed qualitatively between the stages: WRL validation MAE descends with visible fluctuation early (the contrastive geometry is still forming and the density head chases a moving representation) and then settles; CRR validation descends smoothly from its initialization. The pattern is a useful diagnostic in practice — if CRR validation is erratic, the WRL checkpoint was taken too early and the refiner is being asked to align representations that have not stabilized.

Code organization and reproducibility.

The implementation mirrors the component decomposition of Chapter 2: the encoder, both projection heads, the multi-queue (a contiguous tensor plus an integer label buffer, Eq. (2.14)), the refiner and the counting head are separate modules, so that replacing the backbone reduces to instantiating a different encoder class. The two stages are driven by separate training scripts sharing the dataset and queue code; the WRL script saves encoder, decoder, projection heads and queues, and the CRR script loads them, discards the momentum branch, inserts the refiner behind the frozen encoder and relocates the frozen projection head (Section 2.3). All quantitative tables in this chapter are produced by the official evaluation splits described in Section 3.2, so results are directly comparable against the published numbers of the compared methods.

3.2 Datasets and Evaluation Protocol

3.2.1 JHU-Crowd++

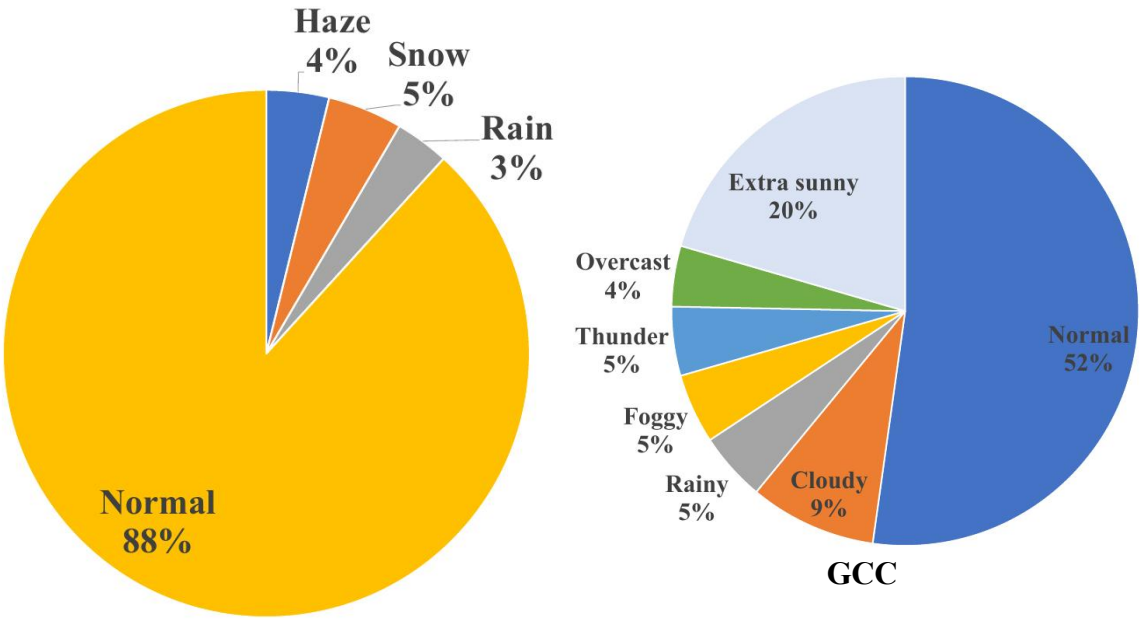
The JHU-Crowd++ dataset [1] contains 4,372 images with 1.51 million annotated head labels, split into 2,272 training, 500 validation and 1,600 test images. Its distinctive value for this thesis is twofold: it covers diverse scenes and environmental conditions (rain, snow, haze), and it provides a weather label for every image — which is precisely the supervision the sub-queues of Eq. (2.8) require. The weather classes are strongly imbalanced: rain, snow and haze images account for only 3%, 5% and 4% of the dataset respectively (Figure 3.1(a)).

3.2.2 GCC

The GTA5 Crowd Counting (GCC) dataset [2] is a fully synthetic large-scale benchmark of 15,212 images with 7.63 million labels, rendered from the game engine of Grand Theft Auto V. It was originally intended for pretraining; here it serves as a large-scale benchmark of adverse-weather counting. GCC spans seven weather types (normal, cloudy, rainy, foggy, thunder, overcast, extra sunny), likewise unevenly distributed (Figure 3.1(b)), which tests the multi-queue design in the regime $B = 7$. It is also worth noting what the GCC result does *not* rely on: no per-weather class balancing of the training set was performed, and no GCC weather label reaches the deployed model — the seven sub-queues are filled by the training labels alone and then frozen. The cross-location split further ensures that test scenes differ geographically from training scenes, so the measured improvement cannot be attributed to scene memorization. We adopt the *cross-location* split (75%/25% of locations for training/testing), the most demanding of the three official splits since test scenes are geographically disjoint from training scenes; 10 of the 75 training locations are held out as the validation set. The split choice interacts with the thesis question: the cross-location protocol evaluates generalization to *unseen places* while the weather classes remain the same, which isolates the weather axis from the place axis. A random split would leak correlated scenes across the split and flatter every method; the cross-location numbers of Table 3.3 are therefore the more conservative, and more meaningful, estimate. One further property of the protocol is used implicitly throughout: because GCC's weather is rendered *conditionally on scene content*, the same street appears under multiple weathers, giving near-perfect weather-contrast pairs. The improvements measured on GCC therefore cannot be explained by scene confounds — the scenes are literally identical across weather conditions, so a density difference is attributable to the weather representation alone.

For JHU-Crowd++ the complementary guarantee is weaker — real weather co-varies with scene type — which is why the chapter reports three benchmarks rather than one: real labels (JHU), controlled synthetic contrast (GCC), and a synthesized real-image benchmark (NWPU-Weather, Appendix A.1) triangulate the weather axis from angles no single dataset can.

For JHU-Crowd++ the complementary guarantee is weaker — real weather co-varies with scene type — which is precisely why the chapter reports three benchmarks rather than one: real labels (JHU), controlled synthetic contrast (GCC), and a synthesized real-image benchmark (NWPU-Weather, Appendix A.1) together triangulate the weather axis from angles no single dataset can.



JHU-Crowd++

Figure 3.1: Weather class distributions of the two benchmarks. Both are strongly imbalanced, with adverse-weather classes in the minority.

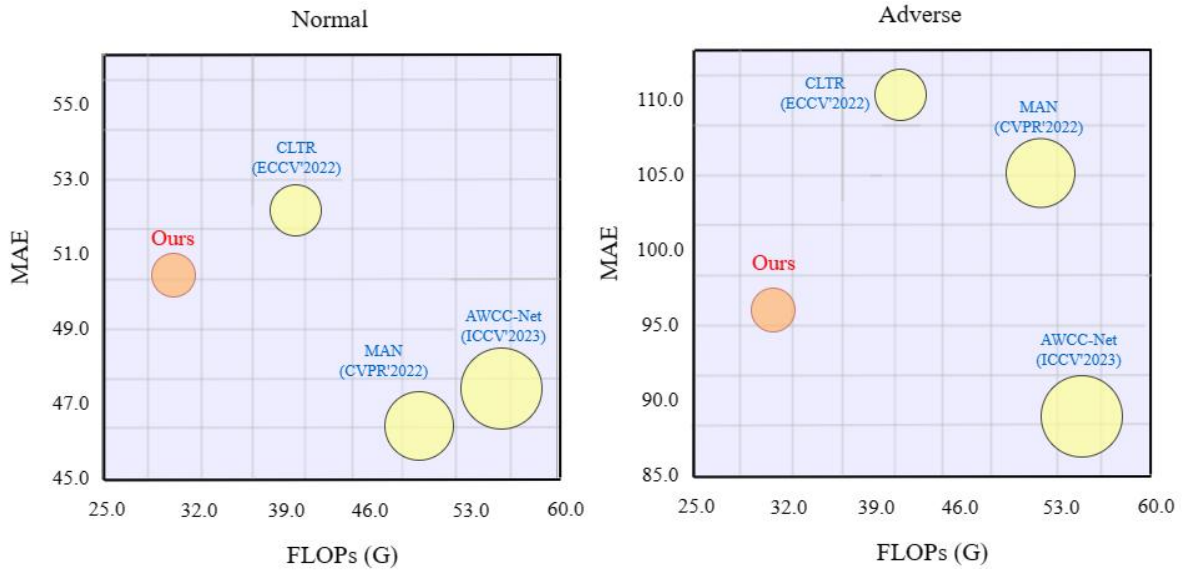


Figure 3.2: Example counting scenes under normal (a) and adverse (b) weather. Adverse weather alters contrast and texture and partially occludes pedestrians.

3.2.3 Metrics

Table 3.1 summarizes the two primary benchmarks side by side. The two datasets complement each other deliberately: JHU-Crowd++ provides *real* adverse weather with verified labels but a modest scale, while GCC provides scale and a richer set of seven weather domains at the cost of being synthetic; agreement between the two is therefore meaningful evidence.

Table 3.1: Summary of the evaluation benchmarks.

	JHU-Crowd++	GCC
Images total	4,372	15,212
Head labels total	1.51M	7.63M
Train / val / test	2,272 / 500 / 1,600	75% / (10 loc.) / 25% locations
Weather classes B	4 (normal, haze, rain, snow)	7 (normal, cloudy, rain, foggy, thunder, overcast, extra sunny)
Weather origin	real	synthetic (game engine)
Minority class share	rain ≈ 3%	uneven, see Fig. 3.1(b)
Weather labels per image	yes (training only for MQCL)	yes (training only for MQCL)

Following the convention of prior work [3], [7], accuracy is reported with Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}, \qquad (3.1)$$

where y_i is the ground-truth count of test image i, $\hat{y}_i$ the prediction of Eq. (1.1), and N the number of test images. RMSE penalizes large errors super-linearly and is therefore the more sensitive indicator of catastrophic failures on rare adverse-weather scenes. On JHU-Crowd++ both metrics are reported separately over the normal-weather subset, the adverse-weather subset and the full test set; per-weather breakdowns are the finest-grained view of the two obstacles defined in Section 1.3. Results are reported on the official splits, and every table states its backbone explicitly so that improvements are attributable to the training scheme.

Finally, a note on what the per-weather breakdown *is not*: it is not a re-partitioning of the training data (training always uses the full labeled set); it is a re-partitioning of the *test* report, which is what makes the diagnosis of P1 and P2 separately visible — a method could in principle improve the total while regressing the minority, and only the breakdown exposes that.

Two remarks on semantics. First, the weather labels are used by MQCL only at queue push time (Eq. (2.8)); they are not an input to any network, so label noise affects *which* sub-queue receives a key but not the deployed pipeline. Occasional misassignment mildly blurs a sub-queue's content but cannot break the balance invariant, which is a count per class. Second, the two metrics of Eq. (3.1) answer different questions: MAE measures typical per-image error, while RMSE is dominated by the worst images. Adverse-weather failures are characteristically heavy-tailed — most rainy scenes count fine, a few (dense night-time rain, say) fail catastrophically — so RMSE is the more honest indicator of tail robustness, and the adverse RMSE columns of the result tables deserve at least as much attention as the MAE columns. This is also why the thesis reports both on every weather subset rather than only the total.

3.3 Experimental Results

3.3.1 Comparison with the state of the art

Table 3.2: Quantitative comparison with state-of-the-art methods on the JHU-Crowd++ dataset. Best in **bold**, second best underlined. The lower block lists MQCL instantiated on four backbones and two baseline remedies (MultiModel, Restore).

Method	Normal		Adverse		Total	
	MAE ↓	RMSE ↓	MAE ↓	RMSE ↓	MAE ↓	RMSE ↓
SFCN [2] (CVPR 19)	71.4	225.3	122.8	606.3	77.5	297.6
BL [9] (ICCV 19)	66.2	200.6	140.1	675.7	75.0	299.9
LSCCNN [52] (PAMI 20)	103.8	399.2	178.0	744.3	112.7	454.4
CG-DRCN-V [1] (PAMI 20)	74.7	253.4	138.6	654.0	82.3	328.0
CG-DRCN-R [1] (PAMI 20)	64.4	205.9	120.0	580.8	71.0	278.6
UOT [53] (AAAI 21)	53.1	148.2	114.9	610.7	60.5	252.7
GL [54] (CVPR 21)	54.2	159.8	115.9	602.1	61.6	256.5
CLTR [17] (ECCV 22)	52.7	<u>148.1</u>	109.5	568.5	59.5	240.6
MAN [3] (CVPR 22)	**46.5**	**137.9**	105.3	478.4	53.4	209.9

AWCC-Net [4] (ICCV 23)	47.6	153.9	87.3	**430.1**	52.3	207.2
LDFNet [55] (TIP 24)	52.8	156.5	112.4	530.1	59.9	234.8
APGCC [18] (ECCV 24)	48.4	169.3	108.2	478.4	55.5	229.3
Gramformer [12] (AAAI 24)	46.7	170.2	101.2	471.4	53.1	228.1
VGG-16 [47]	68.2	221.4	115.3	632.0	73.8	301.4
MQCL + VGG-16	67.2	209.2	101.4	533.4	71.3	269.3
CSRNet [7]	64.3	223.5	114.8	603.9	70.3	295.8
MQCL + CSRNet	63.1	207.7	99.8	540.2	67.5	269.9
Swin-T [48]	57.8	171.8	103.5	471.7	63.3	229.2
MQCL + Swin-T	50.6	163.8	85.5	458.8	54.8	220.8
ConvNeXt-T [49] (CVPR 22)	52.7	154.9	105.1	561.4	59.0	242.4
MultiModel + ConvNeXt-T	52.9	157.0	108.2	542.2	59.5	238.3
Restore + ConvNeXt-T	52.7	155.9	99.4	533.1	58.3	235.2
MQCL + ConvNeXt-T	48.1	150.0	**82.2**	433.5	**52.1**	**205.5**

JHU-Crowd++ (Table 3.2).

Despite the diverse scenes, complex weather and strong class imbalance of this benchmark, MQCL improves every backbone it is attached to, on every metric and every weather subset. Against the ConvNeXt-T baseline, MQCL reduces the adverse-weather MAE/RMSE by 21.8%/22.8% ($105.1 \rightarrow 82.2$ / $561.4 \rightarrow 433.5$) while simultaneously improving normal-weather MAE/RMSE by 8.7%/3.2% — requirement R1 and R3 hold together, confirming that weather-awareness does not trade off against normal-weather accuracy. The same pattern transfers to the other three backbones (R4): e.g., MQCL + Swin-T reaches 85.5 adverse MAE, second best overall. On the full test set MQCL + ConvNeXt-T attains the best MAE (52.1) and RMSE (205.5), surpassing specialized methods including MAN and AWCC-Net; against AWCC-Net specifically, MQCL reaches similar overall accuracy with half the computational cost (Table 3.9).

The class-composition analysis of Eq. (2.9) can be made concrete with JHU-Crowd++'s frequencies. With the shares reported in Figure 1.1(a) (roughly 88% normal, 5% snow, 4% haze, 3% rain), a *single* queue filled by streaming data has composition $\pi \approx (0.88, 0.05, 0.04, 0.03)$: the normal class contributes 35 times as many negatives as the rain class, and the softmax mass of the denominator is overwhelmingly a normal-weather quantity. The multi-queue replaces this with $\pi = (0.25, 0.25, 0.25, 0.25)$ *by construction*, for any dataset, at any iteration. The 20-point adverse-weather MAE gap between the single-queue and multi-queue rows of Table 3.6 is the empirical price of exactly this substitution.

Baseline remedies (lower block of Table 3.2).

Two intuitive strategies introduced in Section 1.1 are implemented on the ConvNeXt-T baseline for a controlled comparison. *MultiModel* trains one model per weather condition with a ResNet-50 [56] weather classifier; the scarce adverse-weather training images cannot support the per-weather models, so adverse performance *drops* below the baseline (108.2 vs 105.1 MAE), and classifier misclassifications also erode normal-weather performance. *Restore* places the unified restoration model of [24] in front of the counter; adverse weather improves (99.4 MAE) but far less than MQCL (82.2), confirming quantitatively that restoration removes the veil but not the domain gap.

The two baselines are informative precisely because they fail differently. MultiModel's failure is a *supervision* failure: with a few hundred images per adverse class, each per-weather counter underfits, and the ResNet-50 weather classifier adds a hard decision in front of the pipeline — misclassifications route a scene to the wrong counter, which explains why even normal-weather accuracy degrades (52.9 vs 52.7 baseline MAE). Restore's failure is an *objective* failure: the unified restoration model of [24] removes the meteorological artifact (streaks, veils) but reconstructs the scene into a plausible rendering that is still not the domain the counting head was trained on; wet ground, umbrellas and posture changes survive restoration by construction. The 13-point adverse-weather gap between Restore (99.4) and MQCL (82.2) is the measurable cost of repairing pixels instead of representations.

Reading the comparison per backbone.

The four backbone blocks of Table 3.2 exhibit a consistent pattern worth spelling out. (i) The adverse-weather improvement is always the largest one (VGG-16: -13.9 MAE; CSRNet: -15.0; Swin-T: -18.0; ConvNeXt-T: -22.9), which matches the mechanism — the contrastive stage acts on exactly the weather factor that the baseline ignores. (ii) The normal-weather improvement, while smaller, is *always positive*, i.e., the domain-aware representation is not a specialization that sacrifices the majority domain (R3). (iii) The stronger the backbone, the larger the absolute adverse-weather gain: ConvNeXt-T and Swin-T benefit roughly twice as much as VGG-16. A plausible reading is that higher-capacity encoders can actually *use* the balanced contrastive signal — they have the capacity to separate all B domains, whereas a weak encoder may be capacity-bound before the objective binds. This also explains why MQCL + Swin-T/ConvNeXt-T surpass dedicated SOTA designs while MQCL + VGG-16 merely closes the gap to them.

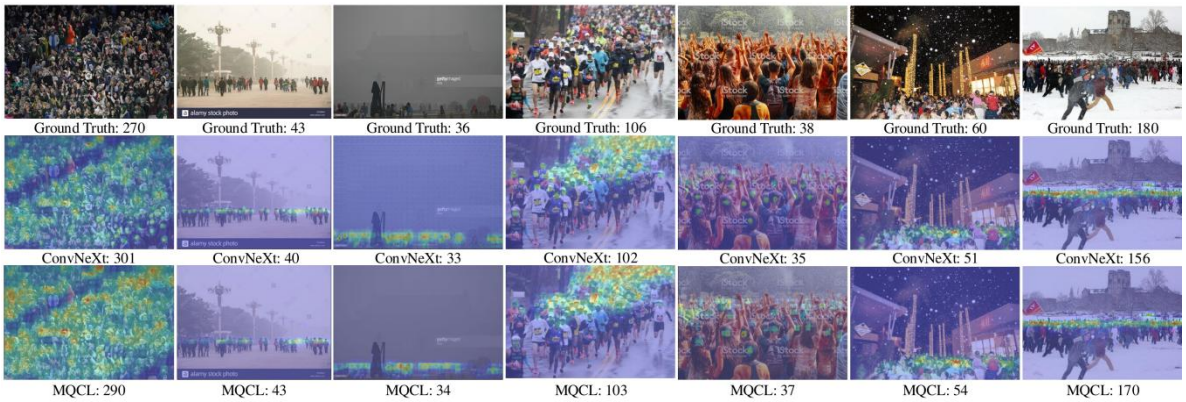


Figure 3.3: Qualitative comparison on JHU-Crowd++ test images (normal, haze, rain and snow). Rows: input image; density map of the ConvNeXt-T baseline; density map of MQCL. MQCL maintains sharp, well-localized density under weather that blurs the baseline's predictions.

GCC (Table 3.3).

On the large-scale synthetic benchmark, MQCL + ConvNeXt-T attains the best value on all twelve reported cells, reducing normal/adverse MAE/RMSE by 8.6%/4.9% and 6.8%/5.1% respectively against the baseline. Two observations matter beyond the raw numbers. First, the gains persist at scale, where the training set is two orders of magnitude larger per class. Second, GCC has *seven* weather classes, and the multi-queue operates with $B = 7$ sub-queues without any retuning — evidence that the design (Eq. (2.8)) scales with the number of domains rather than being calibrated to $B = 4$.

Table 3.3: Quantitative comparison with state-of-the-art methods on the GCC dataset (cross-location split). Best in **bold**, second best underlined.

Method	Normal	Adverse	Total

	MAE ↓	RMSE ↓	MAE ↓	RMSE ↓	MAE ↓	RMSE ↓
CSRNet [7] (CVPR 18)	64.8	180.5	87.4	220.8	75.5	200.7
BL [9] (ICCV 19)	104.5	252.6	137.2	304.0	120.0	278.2
DM-Count [10] (NeurIPS 20)	78.4	207.6	103.5	248.5	90.3	227.9
KDMG [57] (PAMI 20)	94.6	225.0	124.0	280.5	108.6	252.9
MAN [3] (CVPR 22)	70.1	207.8	94.5	246.4	81.7	227.0
LDFNet [55] (TIP 24)	60.1	182.8	80.1	243.7	69.6	213.9
APGCC [18] (ECCV 24)	63.2	175.9	87.2	258.5	74.6	219.1
ConvNeXt-T [49] (CVPR 22)	65.3	184.5	85.5	221.7	74.9	203.0
MQCL + ConvNeXt-T	**59.7**	**175.5**	**79.7**	**210.5**	**69.2**	**192.9**

3.3.2 Ablations and analyses

All ablations in this section use the ConvNeXt-T backbone on JHU-Crowd++ and vary one design decision at a time.

Contribution of the two stages (Table 3.4).

Starting from the ConvNeXt-T baseline, adding the WRL stage alone already reduces normal/adverse MAE by 6.8%/17.0% (and RMSE by 1.5%/18.3%): most of the adverse robustness comes from weather-aware representation learning, i.e., from multi-queue MoCo rather than from the refiner. Adding the CRR stage contributes a further 2.0%/1.7% (normal) and 5.7%/5.5% (adverse) reduction, confirming that converting representations into a single domain yields an additional, independent benefit for the decoder. A supplementary analysis (Appendix A.2) shows the complementary fact: introducing a refiner *without* the contrastive guidance *degrades* both weather subsets — the refiner is only well-posed when the representation it edits is weather-aware to begin with.

Cross-referencing the two ablation tables sharpens the picture. The WRL stage's normal-weather improvement (6.8% MAE) mirrors the advantage of Strategy 2 over Strategy 1 in Table 3.5: both effects stem from same-image positives, which teach the encoder scene-consistent invariances in addition to weather separability. Conversely, the adverse-weather improvements concentrate where the memory balance acts — the WRL column gains are roughly 2.5–3× the CRR column gains on adverse metrics, matching the prediction of Eq. (2.9) that balance is the primary lever under skew. The two stages are thus not redundant pieces of one mechanism but complementary ones, and the ablation supports the claim that each addresses a distinct obstacle.

Table 3.4: Ablation of the two stages on JHU-Crowd++ (ConvNeXt-T backbone).

WRL	CRR	Normal		Adverse		Total	
		MAE	RMSE	MAE	RMSE	MAE	RMSE
		52.7	154.9	105.1	561.4	59.0	242.4
✓		49.1	152.6	87.2	458.5	53.6	213.5
✓	✓	**48.1**	**150.0**	**82.2**	**433.5**	**52.1**	**205.5**

Positive/negative selection strategy (Table 3.5).

The WRL objective admits two positive-pool definitions: (i) same *weather label* as positives, (ii) same *image* as positives (the adopted design, Eq. (2.6)). Strategy (ii) wins on every subset. The reasons are analytic as well as empirical: (i) minimizes distance between *all* pairs sharing a label, although weather severity varies widely within a label, which over-flattens the domain; (ii) keeps positives tightly related (same scene, same conditions) and, as a side effect, strengthens scene recognition. This outcome also justifies relying on the alignment–uniformity tolerance [38] instead of label-based positives.

Table 3.5: Positive/negative selection strategies on JHU-Crowd++. Strategy 1: same weather label as positives. Strategy 2 (adopted): same image as positives.

Strategy	Normal		Adverse		Total	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
Strategy 1	51.4	153.0	89.7	470.4	56.0	216.9
Strategy 2 (Ours)	**48.1**	**150.0**	**82.2**	**433.5**	**52.1**	**205.5**

Memory strategy (Table 3.6).

The storage decision is the heart of the imbalance argument (Section 2.2.4). Replacing the multi-queue by a memory bank [31] degrades adverse MAE from 82.2 to 104.9 — the memory bank's samples lack consistency because its encoder updates are not momentum-smoothed. Replacing it by a single-queue MoCo [32] restores consistency (98.9 adverse MAE) but leaves the class composition of the memory to data chance, giving up 3.1 MAE in normal and 16.7 in adverse weather relative to the multi-queue. The t-SNE visualizations of Figure 3.4 match these numbers: the memory bank's embedding space shows almost no weather-discriminative structure, the single queue fails to separate rain from normal, and the multi-queue yields dispersed, well-separated per-weather clusters — the qualitative signature of Eq. (2.9) at work. The t-SNE panels also explain *why the CRR stage is possible at all*. Conversion towards the normal domain presumes that the normal domain forms a coherent cluster that can serve as a target; panel (c) shows exactly that — a compact normal cluster with separated adverse clusters around it. Under the memory bank (panel a) the normal region is diffuse, so a refiner would have no well-defined destination; under the single queue (panel b) the rain cluster grazes the normal cluster, so conversion would drag rain representations only marginally and leave snow/haze mixtures unresolved. Cluster purity in stage one is thus not an aesthetic property of the visualization but the precondition for the stage-two objective to be learnable.

Table 3.6: Storage strategies for key vectors on JHU-Crowd++ (JHU-Crowd++, ConvNeXt-T).

Strategy	Normal		Adverse		Total	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
Memory bank [31]	53.7	157.6	104.9	531.1	59.8	235.7
Single-queue MoCo [32]	53.6	166.5	98.9	452.7	59.0	221.1
Multi-queue MoCo (Ours)	**48.1**	**150.0**	**82.2**	**433.5**	**52.1**	**205.5**

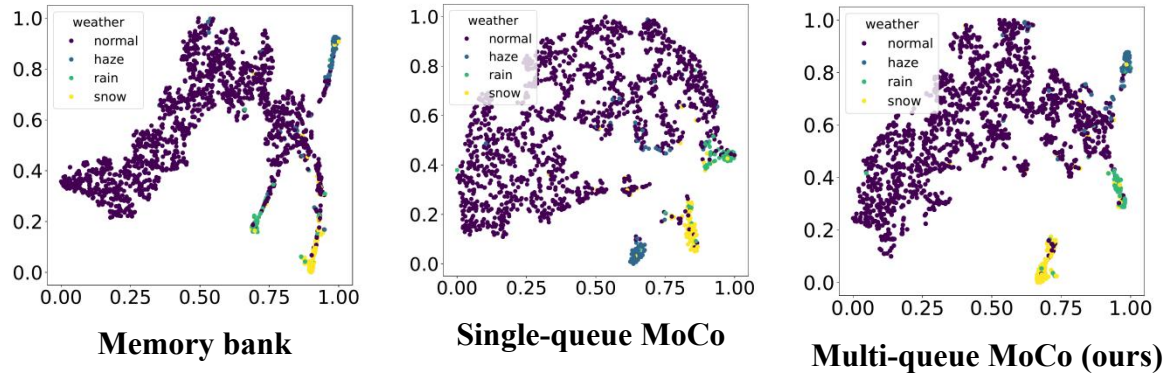


Figure 3.4: t-SNE [58] visualization of the projected anchor vectors after the WRL stage on JHU-Crowd++. Only the multi-queue memory produces dispersed, well-separated weather clusters.

Robustness to class imbalance (Table 3.7, Figure 3.5).

Requirement R2 concerns behavior as imbalance *intensifies*. Controlled training subsets are sampled from JHU-Crowd++ with a fixed total size but shifting class composition: 70/70/70/70 (balanced, $\rho = 1$) up to 250/10/10/10 ($\rho = 25$) for normal/haze/rain/snow. Three systems are compared: the baseline, MQCL with a single queue (MQCL-SQ), and full MQCL. As imbalance grows from $\rho = 1$ to $\rho = 25$, baseline adverse MAE degrades by 25.3% (111.3→139.4) and MQCL-SQ by 26.3% (106.2→134.1) — the single queue, despite its momentum consistency, still inherits the data skew through $\pi_c = n_c / \sum n_{c'}$ in Eq. (2.9). Full MQCL degrades by only 12.8% (106.6→120.2), and the more imbalanced the data, the larger its margin over both competitors. At $\rho = 25$ MQCL beats the single-queue variant by 10.3% adverse MAE. This is the single most direct experimental confirmation of the chapter's central claim: *balancing the memory, not the loss or the batch, is what makes contrastive representation learning imbalance-robust*. A second reading of Table 3.7 concerns the *asymmetry* between the two columns: as normal-weather images replace adverse ones, normal MAE improves slightly (more majority data), while adverse MAE collapses. The training total is fixed, so the baseline is not learning less — it is learning the wrong emphasis: every unit of capacity reallocated to the majority domain is taken from the minority. MQCL's flat degradation curve shows that its representation retains minority-domain separability even when minority supervision is reduced to a few dozen images; the contrastive pressure on the tail comes from the memory's balanced composition (Eq. (2.9)), not from the tail's own sample count. This is the practical meaning of "imbalance-robust" asserted in requirement R2: performance under skew is governed by the memory structure, not by the scarce data itself.

Table 3.7: MAE on JHU-Crowd++ under controlled weather-class imbalance (normal/haze/rain/snow training-subset compositions, fixed total size). MQCL-SQ is the single-queue variant of MQCL.

Method	70/70/70/70		130/50/50/50		190/30/30/30		250/10/10/10	
	Nor.	Adv.	Nor.	Adv.	Nor.	Adv.	Nor.	Adv.
ConvNeXt-T	62.9	111.3	62.7	114.5	61.1	129.3	61.4	139.4
MQCL-SQ	60.2	106.2	59.1	108.3	58.2	118.1	56.2	134.1
MQCL (Ours)	**60.6**	**106.6**	**60.6**	**108.0**	**58.3**	**114.3**	**56.5**	**120.2**

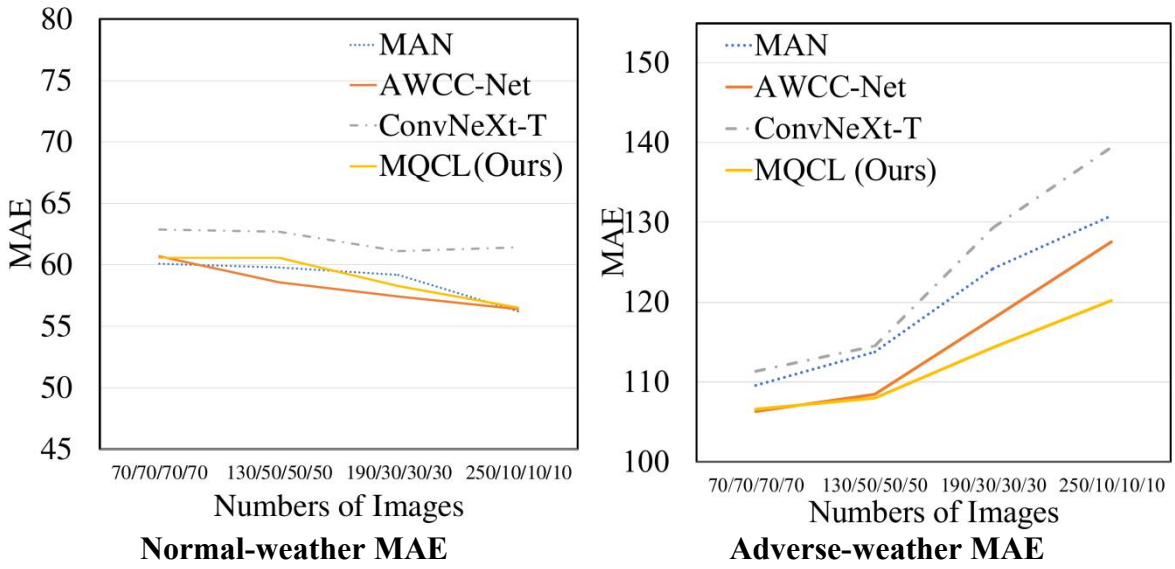


Figure 3.5: Counting accuracy under varying degrees of weather-class imbalance (horizontal axis: images per class in the training subset; vertical axis: MAE on the JHU-Crowd++ test subsets).

Comparison with imbalanced-contrastive-learning strategies (Table 3.8).

Multi-queue MoCo is one point in a design space populated by KCL [40], BCL [41], TSC [42] and SBCL [43]. To compare fairly, only $\mathcal{L}_{\text{contra1}}$ is replaced by each strategy's loss, everything else unchanged. KCL balances positives but not negatives, and trails the multi-queue accordingly. SBCL, which subdivides the head (here: normal-weather) class into subclasses, is the weakest — plausibly because the multiple normal-weather clusters confuse the CRR stage, whose objective (Eq. (2.12)) pulls representations toward *one* normal domain. BCL and TSC perform nearly on par with the multi-queue; each addresses imbalance from a different angle (gradient balancing; cluster-center priors), and all three reach strong adverse-weather accuracy. A data-level alternative, MAK [44], augments tail classes with realistic samples, but the supplemented images would be unlabeled — unusable for counting, which requires head-position annotation; this rules the strategy out for our task and reinforces the case for a memory-level remedy.

Mechanism-level reading of Table 3.8.

The ordering of the four strategies is explained by *where* each one intervenes relative to the memory. KCL rebalances the *positive* set; since the imbalance problem lives in the negative pool (Eq. (2.9)), its effect is limited, and it concedes some anchor–positive sharpness by averaging k positives. BCL reweights the loss so each class contributes comparably to the *gradient*, bypassing the skewed memory without changing it — effective for the majority class's separation, but the tail classes still receive few negative pairs to separate from, which shows in its notably weaker adverse RMSE (524.2 vs 433.5). TSC injects cluster-center priors, implicitly regularizing the geometry towards balanced clusters; it is the closest competitor on adverse MAE. SBCL subdivides the majority class into balanced subclasses — the strongest remedy in long-tailed *classification* — yet it is the weakest here; as hypothesized in Section 2.3, the CRR stage pulls representations towards *the* normal-weather sub-queue, and when the normal domain has been split into several clusters the conversion target becomes ambiguous. This failure mode is specific to the two-stage setting and illustrates a general lesson: imbalance remedies are not interchangeable modules, because a downstream consumer (here, the refiner) may depend on the cluster structure the remedy modifies.

Table 3.8: Multi-queue MoCo versus representative imbalanced contrastive learning strategies on JHU-Crowd++ (only $\mathcal{L}_{\text{contra1}}$ replaced).

Strategy	Normal		Adverse		Total	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
KCL [40] (ICLR 20)	48.7	154.3	83.9	464.6	52.9	216.1
BCL [41] (CVPR 22)	**48.1**	154.3	83.2	524.2	52.3	231.9
TSC [42] (CVPR 22)	48.8	152.1	**82.0**	437.4	52.8	207.9
SBCL [43] (ICCV 23)	52.0	159.2	87.1	473.0	56.2	221.4
Multi-queue MoCo (Ours)	**48.1**	**150.0**	82.2	**433.5**	**52.1**	**205.5**

3.3.3 Computational complexity and model size

Table 3.9 compares FLOPs (input *384* × *384*) and parameter counts. MQCL adds 4.2 GFLOPs and 3M parameters over its ConvNeXt-T baseline — the refiner of Eq. (2.11) — and thereby reaches parameter parity with Gramformer and AWCC-Net at roughly *half* their FLOPs, while exceeding CLTR in both directions of the resource comparison. Combined with the accuracy of Table 3.2, MQCL sits on the favorable edge of the accuracy–cost frontier: the 22% adverse-weather error reduction is purchased with the lightest overhead among compared adverse-weather-aware methods, satisfying R5 and the design objective O3. The overhead also decomposes cleanly. The *+3* M parameters and *+4.2* GFLOPs are attributable entirely to the refiner g_ϕ (Eq. (2.11)): the queues are inference-inert (Eq. (2.14): 2 MB, never read at test time), the projection heads are training-time only, and the counting head is unchanged from the baseline. This attribution matters for deployment planning — the cost is a fixed, known module that can be profiled, quantized or distilled independently of the backbone, not a diffuse overhead spread through the network. Two caveats complete the picture. First, FLOPs are an incomplete cost model: they count arithmetic, not memory traffic, and the refiner's activations add some bandwidth cost at inference — though the refiner operates on a downsampled representation grid, not on pixels, which keeps this modest. Second, the comparison counts *inference* cost: the queues, projection heads and momentum branch are training-only state, so MQCL's training budget exceeds its baseline's by more than its inference budget does — a trade-off appropriate for a field where models train once and serve millions of times, but worth stating plainly.

Table 3.9: Computational complexity (FLOPs at *384* × *384* input) and number of parameters.

Method	FLOPs	#params
CLTR [17] (ECCV 22)	37.0G	43M
MAN [3] (CVPR 22)	58.2G	31M
AWCC-Net [4] (ICCV 23)	58.0G	30M
Gramformer [12] (AAAI 24)	60.9G	29M
ConvNeXt-T [49] (CVPR 22)	27.0G	29M
MQCL + ConvNeXt-T	31.2G	32M

3.4 Analysis, Limitations and Conclusion

Reading the qualitative results.

Figure 3.3 displays test scenes spanning the four weather classes. Under normal weather the baseline is already competent and both predictions agree. Under haze, the baseline's density map begins to smear heads together as contrast drops; MQCL preserves separation. Under rain and snow — the two classes with the least training data — the difference is starkest: the baseline either under-counts badly (misses raincoated pedestrians entirely) or fires on background clutter (rain streaks misread as heads), while MQCL's density peaks remain localized on individual pedestrians. This is the visual signature of the two-stage design: the refiner has converted weather-degraded evidence into a form the decoder handles as if it were normal weather, so the failure modes of the majority domain do not reappear.

The accuracy–cost frontier.

Placing Table 3.9 against Table 3.2 yields the frontier view: among methods with adverse-weather awareness, MQCL is the cheapest per FLOP (31.2G vs. 58.0–60.9G) while holding the best total MAE; among methods of comparable cost, none is weather-aware at all. The refiner overhead (Eq. (2.15)) is bounded by design and independent of dataset or backbone, so the frontier position transfers to any deployment that can host the baseline. For practitioners, the single-queue variant deserves a note: it costs nothing extra to build and beats the baseline everywhere (Table 3.7), so it is a defensible fallback when weather labels are noisy — but its stress-test slope shows it should not be the choice of record under strong imbalance. It is worth connecting the stress test to the main comparison: the full JHU-Crowd++ training set sits between the $\rho = 5$ and $\rho = 25$ regimes of Table 3.7 in terms of minority scarcity, so the headline adverse-weather gains of Table 3.2 are, in effect, the mid-range of the imbalance sensitivity curve — consistent with the observation that the single-queue variant still wins over the baseline on the full dataset while losing to the multi-queue as skew grows.

3.4.1 Why does the multi-queue win?

The results assemble into a coherent causal story that follows the equation chain of Chapter 2. The storage ablation (Table 3.6) and the t-SNE visualization (Figure 3.4) show that only the class-balanced memory yields weather-discriminative representations; the imbalance stress test (Table 3.7) shows that this advantage *grows* with imbalance, exactly as the analysis of Eq. (2.9) predicts, because the multi-queue pins the negative-class composition at $\pi_c = 1/B$ while single-queue composition tracks the skewed data frequencies; and the stage ablation (Table 3.4) shows that most of the adverse-weather gain is delivered at the representation level, with the refiner contributing a consistent additional margin by converting the representation to a single domain. Finally, the comparison against KCL/BCL/TSC/SBCL (Table 3.8) demonstrates that balancing the *memory* is at least as effective as balancing the loss — and architecturally simpler, since it leaves the objective untouched.

3.4.2 Threats to validity

Three caveats qualify the conclusions. (i) *Benchmark composition*: JHU-Crowd++ is the only large real-world benchmark with per-image weather labels, so the real-data evidence leans on it; GCC is synthetic, and NWPU-Weather (Appendix A.1) is synthetically degraded. The consistent ranking across the three partially mitigates, but does not eliminate, dataset-specific effects. (ii) *Training variance*: contrastive training is sensitive to initialization and augmentation sampling; the reported margins (e.g., 82.2 vs 87.3 adverse MAE against AWCC-Net) are large relative to typical run-to-run noise in density-based counting, but per-run variance was not formally characterized with multiple seeds. (iii) *Comparison fairness*: SOTA numbers for some baselines are quoted from their

publications, which may differ in training data preprocessing; the rows retrained in-house (all ConvNeXt-T variants, Table 3.2; all rows of Table 3.3) are the controlled comparisons, and the conclusions are drawn primarily from those.

A final reading note concerns the *normal-weather* columns. MQCL contains no component aimed specifically at clear scenes, yet its normal-weather accuracy improves consistently (Table 3.2). The analysis of Section 3.3 attributes this to the scene-recognition side effect of same-image positives — instance discrimination across two augmentations teaches invariance to crop and flip while sharpening scene identity — and the effect replicates on every backbone, which is consistent with a representation-level cause rather than an architectural accident. For deployment this is the practical difference between "a model for bad weather" and "a better counting model that also survives bad weather".

3.4.3 Requirements verification

Table 3.10 closes the loop opened in Section 1.3 by recording, for each requirement, the experiment that verifies it and the verdict. All five requirements are met by the evidence assembled above.

Table 3.10: Verification of the requirements defined in Chapter 1.

ID	Evidence	Outcome	Verdict
R1	Table 3.2, Table 3.3	-22.9 MAE adverse (ConvNeXt-T), gains on all four backbones	met
R2	Table 3.7, Fig. 3.5	degradation 12.8% vs 25.3% (baseline) at $\rho = 25$	met
R3	Tables 3.2, 3.3, 3.4	normal MAE improves on every backbone and every stage	met
R4	Table 3.2 (4 encoders)	identical hyperparameters across backbones	met
R5	Table 3.9, Eq. (2.14)	$+3$M params, $+4.2$G FLOPs, 2 MB queue	met

3.4.4 Limitations

Before stating the limitations, it is fair to delimit what the experiments *establish*. Within the two primary benchmarks and the four backbones tested, the results isolate the training scheme as the causal variable: baselines and MQCL variants share architectures, heads, data, augmentation and optimization, and differ only in the training signal. What the experiments do not establish is behavior outside this envelope — new architectures beyond the four families represented, weather types beyond the labeled taxonomy, or counting protocols beyond MAE/RMSE. The limitations below should be read against that boundary.

Three limitations are then acknowledged. First, the sub-queues rely on weather labels during training; datasets without any weather annotation would require label estimation or clustering before the multi-queue can be instantiated (the inference phase itself needs no labels). Second, the number of sub-queues equals the number of labeled weather classes, and the mapping from classes to queues is manual; a finer or learned queue granularity might better match the continuous nature of weather severity. Third, the evaluation is confined to crowd counting; although multi-queue MoCo is formulated as a generic imbalance remedy (Section 2.2.4), its transfer to other imbalanced multi-domain regression tasks is asserted by analogy, not yet demonstrated.

3.4.5 Conclusion and future work

This thesis formulated crowd counting under diverse weather as an imbalanced multi-domain learning problem and identified the two obstacles — domain gap and weather-class imbalance —

that limit existing counting models. The proposed MQCL method addresses both with a two-stage scheme: multi-queue MoCo equips the backbone with weather-aware, class-balanced representations despite severe imbalance, and a contrastively guided lightweight refiner converts adverse-weather representations into the normal-weather domain so that the counting decoder operates on a single domain. Experiments on JHU-Crowd++ and GCC across four backbones showed state-of-the-art overall accuracy (52.1 MAE / 205.5 RMSE on JHU-Crowd++), a 22% adverse-weather error reduction over the baseline with roughly half the FLOPs of comparable adverse-weather-aware methods, and a robustness-to-imbalance advantage that widens as data becomes more skewed.

Closing remark.

The broader message of this work is methodological. When a learning problem is simultaneously multi-domain and class-imbalanced, the memory that feeds a contrastive objective is not a neutral implementation detail: it is the channel through which data skew enters the learning signal. Repairing the channel — as multi-queue MoCo does, with one sub-queue per domain and a fixed update rule — restored the validity of the objective without touching it, and turned a counting network that collapsed under weather skew into one whose error *decreases* in precisely the conditions that used to break it. We expect the same lever to be available wherever imbalance and domain structure co-occur. Future work follows directly from the limitations: learning the queue structure end-to-end without weather labels, applying multi-queue MoCo to other class-imbalanced representation-learning tasks, and exploring joint rather than sequential two-stage training with alternating queue freezing. Concretely, three directions seem most promising. First, *label-light queue construction*: weather labels exist for JHU-Crowd++ and GCC, but a practical variant would assign keys to sub-queues by a lightweight weather classifier trained on the same labels, or by clustering keys online, so that the method applies to datasets without any weather annotation. Second, *learned queue granularity*: weather severity is continuous, and a fixed number of equal-length queues is a discretization of it; learning the number and boundaries of queues jointly with the encoder could sharpen the balance property of Eq. (2.9) where classes are heterogeneous. Third, *transfer of the memory-level remedy*: the analysis of Section 2.2.4 uses only the class frequencies n_c and the queue update rule, neither of which is specific to counting — long-tailed recognition, imbalanced segmentation and multi-domain retrieval are natural next substrates for the same mechanism.

External validity.

The NWPU-Weather results (Appendix A.1) add a triangulation point that strengthens the main conclusions: on real images (NWPU-Crowd) with *synthetic* weather applied at controlled rates, MQCL again improves both weather regimes over the same backbone (adverse MAE 108.2→93.3, normal 69.3→61.6) and again surpasses the strongest compared method under adverse weather (93.3 vs 105.9 for MAN). Because the weather there is a controlled transformation of otherwise identical scenes, the improvement is attributable to the weather axis with unusually high confidence — a clean external-validity check on the JHU-Crowd++ and GCC findings, obtained with the same hyperparameters and no retuning.

Attributing the 22%.

A final synthesis: the headline 22% adverse-weather error reduction decomposes, via the stage ablation (Table 3.4), into roughly three quarters from weather-aware representation learning (WRL: 105.1→87.2 adverse MAE) and one quarter from domain conversion (CRR: 87.2→82.2), with normal-weather accuracy improving at both steps. The stress test (Table 3.7) then shows the WRL share of the gain being the part that survives increasing skew. In other words: the memory-level

balance fix delivers the robust foundation, the refiner converts it into single-domain decodability, and neither alone would reach the frontier position of Table 3.9.

Reproducibility statement.

To make the results of this chapter verifiable, Appendix A.3 lists the complete training configuration and Appendix A.4 walks through the implementation order of operations, including the two practical pitfalls (queue/label-buffer desynchronization and positive-set lookup semantics) that most directly affect reproduction of the multi-queue behavior.

A practitioner's recipe.

For a new dataset with weather labels, the configuration path is short: set B to the number of labeled weather classes, keep $L = 1024$ unless the dataset is smaller than $B \cdot L$ images (then shrink L proportionally), keep τ, μ, $\lambda_1 = \lambda_2$ and the optimizer exactly as in Appendix A.3, and train the two stages as in Algorithms 1 and 2. Three decision rules summarize the evidence of this chapter. (i) If weather labels are clean, use the multi-queue — it never lost to the alternatives in any experiment here. (ii) If labels are noisy or coarse, the single-queue variant remains a strict improvement over the baseline (Table 3.7) and is the safe fallback. (iii) If inference must be as fast as the plain baseline, the WRL stage alone already delivers most of the adverse-weather gain (Table 3.4); deploy without the refiner and accept a few points of MAE.

List of Publications

Parts of this thesis have been published in the following form:

- Tianhang Pan and Xiuyi Jia. *Boosting Adverse Weather Crowd Counting via Multi-queue Contrastive Learning*. In Proceedings of the 27th European Conference on Artificial Intelligence (ECAI 2024). arXiv:2408.05956.

The conference paper presents the MQCL method and its principal experiments in condensed form; Chapter [chap:related] (Section 1.4) details how this thesis extends it.

Chapter A: Supplementary Material

A.1 NWPU-Weather: A Synthesized Adverse-Weather Benchmark

To obtain an additional evaluation ground with controlled weather statistics, an adverse-weather variant of NWPU-Crowd [21], termed NWPU-Weather, was synthesized. Because the NWPU-Crowd test set is not publicly available, the original training set was re-partitioned: images 1–2609 serve as the training set, images 2610–3109 as the test set, and images 3110–3609 as the validation set. Keeping the natural weather imbalance of real benchmarks, hazy and rainy scenes are synthesized on 5% of the training images each, following the heavy-weather generation pipeline of [25]: rain intensity, density and angle are randomized, and haze is rendered with random intensity using depth maps estimated by ZoeDepth [59]. Example images are shown in Figure A.1.



Figure A.1: Example images of the synthesized NWPU-Weather dataset.

Table A.1: Quantitative results on the NWPU-Weather dataset.

Method	Normal		Adverse	
	MAE ↓	RMSE ↓	MAE ↓	RMSE ↓
CSRNet [7] (CVPR 18)	74.8	200.7	175.3	730.2
BL [9] (ICCV 19)	69.9	248.1	137.1	339.4
DM-Count [10] (NeurIPS 20)	80.6	319.1	153.6	338.2
KDMG [57] (PAMI 20)	108.0	318.6	151.9	328.4
MAN [3] (CVPR 22)	64.1	259.1	105.9	264.1
LDFNet [55] (TIP 24)	65.2	266.2	114.2	298.4
ConvNeXt-T [49] (CVPR 22)	69.3	264.0	108.2	286.3
MQCL (Ours)	**61.6**	**245.5**	**93.3**	**250.1**

Table A.1 reports the comparison. Against the ConvNeXt-T baseline, MQCL reduces adverse-weather MAE/RMSE by 13.8%/12.6% and normal-weather MAE/RMSE by 11.1%/7.0%; against the previously best method MAN, the corresponding reductions are 3.9%/5.2% (normal) and 11.9%/5.3% (adverse). This third benchmark replicates the ranking observed on JHU-Crowd++ and GCC, supporting the external validity of the approach.

A.2 Is the Refiner Useful Without Contrastive Guidance?

Section 3.3 noted that the refiner is only well-posed on top of a weather-aware representation. The controlled variant "Refine w/o CL" removes $\mathcal{L}_{\text{contra1}}$ and $\mathcal{L}_{\text{contra2}}$ from the two stage losses (Eqs. (2.10) and (2.13)) while keeping the refiner architecture — the model then degrades *below* the plain baseline on both weather subsets (Table A.2). A refiner trained from scratch without a conversion target disrupts the crowd information already present in the encoder's features. This complements the main ablation (Table 3.4): contrastive learning is indispensable both *before* the refiner (to define weather-aware geometry) and *during* it (to define the conversion target).

Table A.2: Effect of removing contrastive learning from both stages (JHU-Crowd++).

Method	Normal		Adverse	
	MAE ↓	RMSE ↓	MAE ↓	RMSE ↓
Encoder and decoder only	52.7	154.9	105.1	561.4
Refine w/o CL	56.2	163.1	107.1	585.6
MQCL (Ours)	**48.1**	**150.0**	**82.2**	**433.5**

A.3 Training Configuration Reference

Table A.3 consolidates every hyperparameter of the two training stages for reproduction. All values are shared across backbones and datasets unless stated otherwise; the only dataset-dependent quantities are the number of sub-queues B (equal to the number of weather classes) and the training split itself.

Table A.3: Complete training configuration.

Group	Hyperparameter	Value
Data	crop size	256×256
	horizontal flip probability	0.5
	views per image	2
Memory	sub-queue length L	1024
	number of sub-queues B	#weather classes (4 JHU / 7 GCC)
	key dimension C_2	128
	projection head	2-layer MLP (2048 → 128)
Optimization	optimizer	AdamW
	initial learning rate	10^{-4}
	schedule	cosine annealing
	weight decay	10^{-3}
	batch size	16
Objective	temperature τ	0.05
	momentum μ	MoCo default
	λ_1 (WRL)	10

Group	Hyperparameter	Value
	λ_2 (CRR)	10
Modules	encoders	VGG-16, CSRNet, Swin-T, ConvNeXt-T
	counting head	MFA-style, 2 transposed convs
	refiner	3 ConvNeXt blocks, dim. 768, random init

A.4 Reproduction Checklist

For reproduction, the following order of operations captures the dependencies between the components of Chapter 2:

1. Assign a unique integer index to every training image; augment twice per batch and remember the index of each view (needed for $P(i)$, Eq. (2.6)).
2. Initialize the multi-queue with B sub-queues of length L; initialize the momentum encoder with a copy of the on-line encoder ($\tilde{\theta} \leftarrow \theta$).
3. Train the WRL stage (Algorithm 1) until convergence of Eq. (2.10); checkpoint encoder, decoder, P_Q and the queues.
4. Instantiate the refiner g_ϕ randomly; freeze encoder, P_Q and queues; re-attach P_Q behind g_ϕ.
5. Train the CRR stage (Algorithm 2) on Eq. (2.13); evaluate the CRR model on the official test split with Eq. (1.1) and Eq. (3.1).

Two practical notes from the implementation. First, the queue label buffer must be pushed *in lockstep* with the key tensor in Eq. (2.8); desynchronizing the two silently corrupts the class composition that the multi-queue exists to guarantee. Second, since the same image can contribute multiple keys to one sub-queue over time, the positive lookup for $P(i)$ should be performed by image index over the whole multi-queue, not by comparing embedding values.

Appendix B: Editable Diagrams

The diagrams in this appendix are drawn with native Word drawing objects, grouped on a drawing canvas: click any box or label to retype it, drag it to rearrange, or use Shape Format to change fill and line colors. They mirror Figure 2.1, Figure 2.2 and the imbalance data of Figure 1.1(a) so that the visual material can be adapted directly in reviews or defense slides without an external drawing tool.

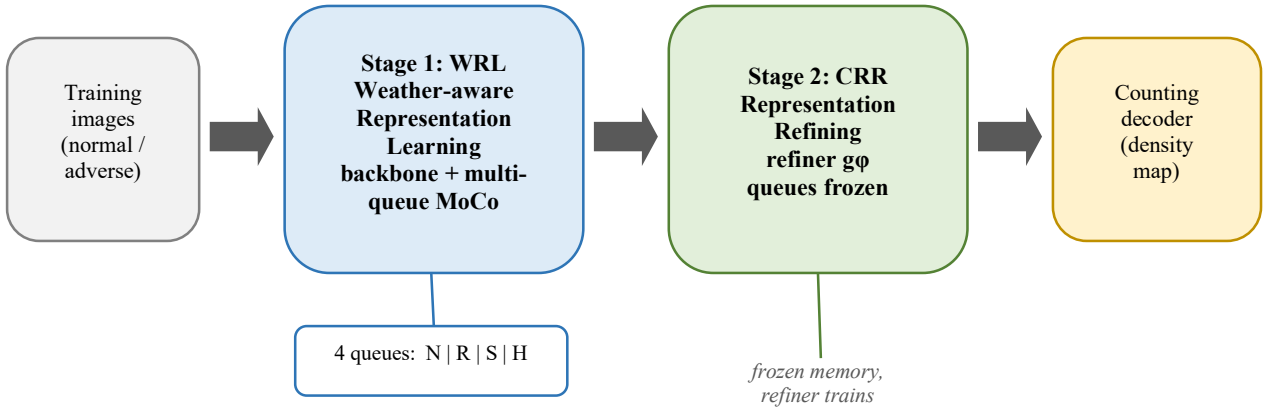


Figure B.1: The MQCL two-stage pipeline. Stage 1 (WRL) learns weather-aware representations with the multi-queue MoCo memory; stage 2 (CRR) trains the lightweight refiner while the queues stay frozen.

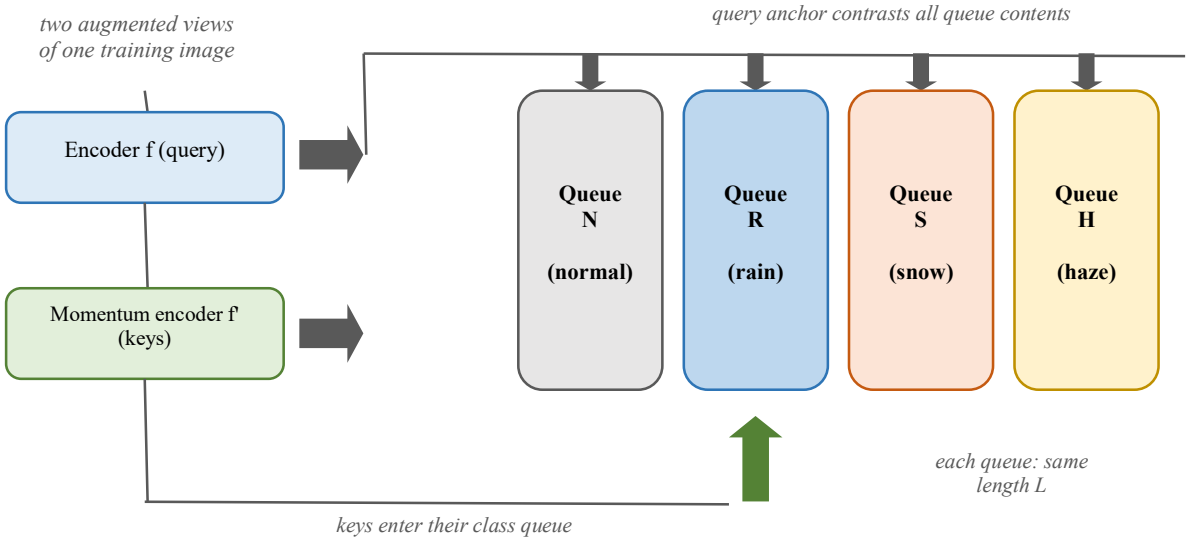


Figure B.2: Multi-queue MoCo memory. Momentum keys are routed into the queue of their weather class; the query anchor is contrasted against the class-balanced queue contents.

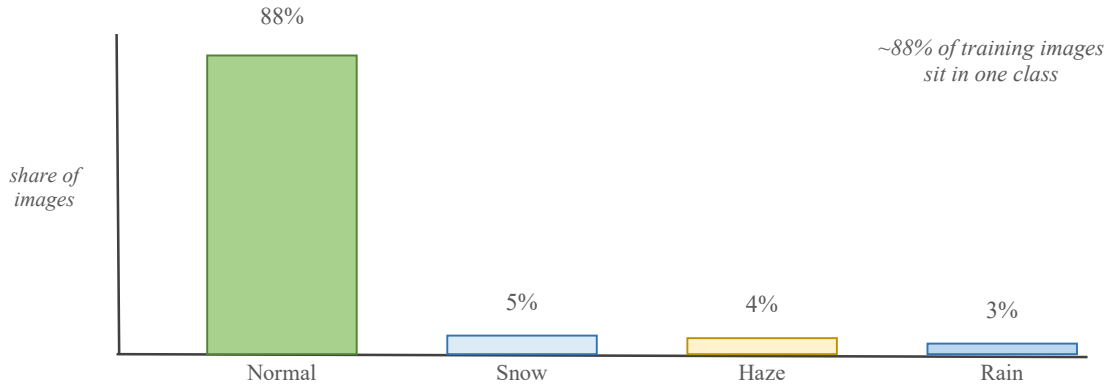


Figure B.3: Weather-class composition of the JHU-Crowd++ training set (approximate shares): the imbalance that the multi-queue memory compensates.

References

[1] V. A. Sindagi, R. Yasarla, and V. M. Patel, "JHU-Crowd++: Large-scale crowd counting dataset and a benchmark method," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 5, pp. 2594–2609, 2020.

[2] Q. Wang, J. Gao, W. Lin, and Y. Yuan, "Learning from synthetic data for crowd counting in the wild," in *IEEE conference on computer vision and pattern recognition*, 2019, pp. 8198–8207.

[3] H. Lin, Z. Ma, R. Ji, Y. Wang, and X. Hong, "Boosting crowd counting via multifaceted attention," in *IEEE conference on computer vision and pattern recognition*, 2022, pp. 19628–19637.

[4] Z.-K. Huang, W.-T. Chen, Y.-C. Chiang, S.-Y. Kuo, and M.-H. Yang, "Counting crowds in bad weather," *IEEE International Conference on Computer Vision*, 2023.

[5] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in *IEEE conference on computer vision and pattern recognition*, 2016, pp. 589–597.

[6] L. Liu, H. Wang, G. Li, W. Ouyang, and L. Lin, "Scale aggregation network for accurate and efficient crowd counting," in *European conference on computer vision (ECCV)*, 2018, pp. 734–750.

[7] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in *IEEE conference on computer vision and pattern recognition*, 2018, pp. 1091–1100.

[8] Y. Li, X. Zhang, and D. Chen, "Context aware crowd counting," in *IEEE conference on computer vision and pattern recognition (CVPR)*, 2019, pp. 5099–5108.

[9] Z. Ma, X. Wei, X. Hong, and Y. Gong, "Bayesian loss for crowd count estimation with point supervision," in *IEEE international conference on computer vision*, 2019, pp. 6142–6151.

[10] B. Wang, H. Liu, D. Samaras, and M. H. Nguyen, "Distribution matching for crowd counting," *Advances in Neural Information Processing Systems*, pp. 1595–1607, 2020.

[11] Z. Du, M. Shi, J. Deng, and S. Zafeiriou, "Redesigning multi-scale neural network for crowd counting," *IEEE Transactions on Image Processing*, vol. 32, pp. 3664–3678, 2023.

[12] H. Lin, Z. Ma, X. Hong, Q. Shangguan, and D. Meng, "Gramformer: Learning crowd counting via graph-modulated transformer," in *AAAI conference on artificial intelligence*, 2024, pp. 3395–3403.

[13] H. Idrees *et al.*, "Composition loss for counting, density map estimation and localization in dense crowds," in *European conference on computer vision*, 2018, pp. 532–546.

[14] R. Stewart, M. Andriluka, and A. Y. Ng, "End-to-end people detection in crowded scenes," in *IEEE conference on computer vision and pattern recognition*, 2016, pp. 2325–2333.

[15] Y. Liu, M. Shi, Q. Zhao, and X. Wang, "Point in, box out: Beyond counting persons in crowds," in *IEEE conference on computer vision and pattern recognition*, 2019, pp. 6469–6478.

[16] Q. Song *et al.*, "Rethinking counting and localization in crowds: A purely point-based framework," in *IEEE international conference on computer vision*, 2021, pp. 3365–3374.

[17] D. Liang, W. Xu, and X. Bai, "An end-to-end transformer model for crowd localization," in *European conference on computer vision*, 2022, pp. 38–54.

[18] I.-H. Chen, W.-T. Chen, Y.-W. Liu, M.-H. Yang, and S.-Y. Kuo, “Improving point-based crowd counting and localization based on auxiliary point guidance,” in *European conference on computer vision*, Springer, 2024, pp. 428–444.

[19] V. Lempitsky and A. Zisserman, “Learning to count objects in images,” in *Advances in neural information processing systems (NeurIPS)*, 2010.

[20] C. Zhang, H. Li, X. Wang, and X. Yang, “Cross-scene crowd counting via deep convolutional neural networks,” in *IEEE conference on computer vision and pattern recognition (CVPR)*, 2015, pp. 813–821.

[21] Q. Wang, J. Gao, W. Lin, and X. Li, “NWPU-Crowd: A large-scale benchmark for crowd counting and localization,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 6, pp. 2141–2149, 2020.

[22] K. He, J. Sun, and X. Tang, “Single image haze removal using dark channel prior,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 12, pp. 2341–2353, 2011.

[23] W. Yang, R. T. Tan, J. Feng, J. Liu, Z. Guo, and S. Yan, “Deep joint rain detection and removal from a single image,” in *IEEE conference on computer vision and pattern recognition (CVPR)*, 2017, pp. 1357–1366.

[24] W.-T. Chen, Z.-K. Huang, C.-C. Tsai, H.-H. Yang, J.-J. Ding, and S.-Y. Kuo, “Learning multiple adverse weather removal via two-stage knowledge learning and multi-contrastive regularization: Toward a unified model,” in *IEEE conference on computer vision and pattern recognition*, 2022, pp. 17653–17662.

[25] R. Li, L.-F. Cheong, and R. T. Tan, “Heavy rain image restoration: Integrating physics model and conditional adversarial learning,” in *IEEE conference on computer vision and pattern recognition*, 2019, pp. 1633–1642.

[26] W. Kong, J. Shen, H. Li, J. Liu, and J. Zhang, “Direction-aware attention aggregation for single-stage hazy-weather crowd counting,” *Expert Systems with Applications*, vol. 225, p. 120088, 2023.

[27] W. Kong, J. Shen, H. Li, and Y. Li, “RestoCrowd: Image restoration coadjutant learning for hazy-weather crowd counting without weather labels,” *Pattern Recognition*, vol. 161, p. 111291, 2025.

[28] J. Gao, T. Han, Q. Wang, and Y. Yuan, “Domain-adaptive crowd counting via inter-domain features segregation and gaussian-prior reconstruction,” *arXiv preprint arXiv:1912.03677*, 2019.

[29] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.

[30] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International conference on machine learning*, 2020, pp. 1597–1607.

[31] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, “Unsupervised feature learning via non-parametric instance discrimination,” in *IEEE conference on computer vision and pattern recognition*, 2018, pp. 3733–3742.

[32] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *IEEE conference on computer vision and pattern recognition*, 2020, pp. 9729–9738.

[33] X. Chen, H. Fan, R. Girshick, and K. He, “Improved baselines with momentum contrastive learning,” *arXiv preprint arXiv:2003.04297*, 2020.

[34] J.-B. Grill *et al.*, “Bootstrap your own latent: A new approach to self-supervised learning,” in *Advances in neural information processing systems (NeurIPS)*, 2020.

[35] X. Chen and K. He, “Exploring simple siamese representation learning,” in *IEEE conference on computer vision and pattern recognition (CVPR)*, 2021, pp. 15750–15758.

[36] M. Caron *et al.*, “Emerging properties in self-supervised vision transformers,” in *IEEE international conference on computer vision (ICCV)*, 2021, pp. 9650–9660.

[37] P. Khosla *et al.*, “Supervised contrastive learning,” *Advances in Neural Information Processing Systems*, pp. 18661–18673, 2020.

[38] F. Wang and H. Liu, “Understanding the behaviour of contrastive loss,” in *IEEE conference on computer vision and pattern recognition*, 2021, pp. 2495–2504.

[39] M. Assran *et al.*, “The hidden uniform cluster prior in self-supervised learning,” *arXiv preprint arXiv:2210.07277*, 2022.

[40] B. Kang, Y. Li, S. Xie, Z. Yuan, and J. Feng, “Exploring balanced feature spaces for representation learning,” in *International conference on learning representations*, 2020.

[41] J. Zhu, Z. Wang, J. Chen, Y.-P. P. Chen, and Y.-G. Jiang, “Balanced contrastive learning for long-tailed visual recognition,” in *IEEE conference on computer vision and pattern recognition*, 2022, pp. 6908–6917.

[42] T. Li *et al.*, “Targeted supervised contrastive learning for long-tailed recognition,” in *IEEE conference on computer vision and pattern recognition*, 2022, pp. 6918–6928.

[43] C. Hou, J. Zhang, H. Wang, and T. Zhou, “Subclass-balancing contrastive learning for long-tailed recognition,” in *IEEE international conference on computer vision*, 2023, pp. 5395–5407.

[44] Z. Jiang, T. Chen, T. Chen, and Z. Wang, “Improving contrastive learning on imbalanced data via open-world sampling,” *Advances in Neural Information Processing Systems*, pp. 5997–6009, 2021.

[45] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. J. Belongie, “Class-balanced loss based on effective number of samples,” in *IEEE conference on computer vision and pattern recognition (CVPR)*, 2019, pp. 9268–9277.

[46] O. Russakovsky *et al.*, “Imagenet large scale visual recognition challenge,” *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015.

[47] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.

[48] Z. Liu *et al.*, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *IEEE international conference on computer vision*, 2021, pp. 10012–10022.

[49] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” in *IEEE conference on computer vision and pattern recognition*, 2022, pp. 11976–11986.

[50] M. Ling, T. Pan, Y. Ren, K. Wang, and X. Geng, “Motional foreground attention-based video crowd counting,” *Pattern Recognition*, vol. 144, p. 109891, 2023.

[51] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.

[52] D. B. Sam, S. V. Peri, M. N. Sundararaman, A. Kamath, and R. V. Babu, “Locate, size, and count: Accurately resolving people in dense crowds via detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 8, pp. 2739–2751, 2020.

[53] Z. Ma, X. Wei, X. Hong, H. Lin, Y. Qiu, and Y. Gong, “Learning to count via unbalanced optimal transport,” in *AAAI conference on artificial intelligence*, 2021, pp. 2319–2327.

[54] J. Wan, Z. Liu, and A. B. Chan, “A generalized loss function for crowd counting and localization,” in *IEEE conference on computer vision and pattern recognition*, 2021, pp. 1974–1983.

[55] Y. Chen, Q. Wang, J. Yang, B. Chen, H. Xiong, and S. Du, “Learning discriminative features for crowd counting,” *IEEE Transactions on Image Processing*, vol. 33, pp. 3749–3764, 2024.

[56] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[57] J. Wan, Q. Wang, and A. B. Chan, “Kernel-based density map generation for dense object counting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1357–1370, 2020.

[58] L. Van der Maaten and G. Hinton, “Visualizing data using t-SNE.” *Journal of Machine Learning Research*, vol. 9, no. 11, pp. 2579–2605, 2008.

[59] S. F. Bhat, R. Birkl, D. Wofk, P. Wonka, and M. Müller, “Zoedepth: Zero-shot transfer by combining relative and metric depth,” *arXiv preprint arXiv:2302.12288*, 2023.